

Linux 系统使用手册

Revision History

Draft	Date	Revision No.	Description
	2025/04/03	V1.0	1. 初始版本。

目 录

前 言.....	4
1. LinuxSDK 安装.....	6
1.1 安装依赖软件.....	6
1.2 解压 LinuxSDK.....	9
1.3 解压 dl.tar.gz 压缩包.....	11
1.4 系统开发环境配置.....	12
1.5 应用开发环境配置.....	13
2. Linux 系统镜像编译、生成.....	15
2.1 配置编译选项.....	16
2.1.1 配置 LinuxSDK 编译选项.....	16
2.1.2 配置设备树	16
2.1.3 配置内核选项	18
2.1.4 配置 Buildroot 编译选项	22
2.2 编译 LinuxSDK.....	24
2.3 局部编译	26
2.3.1 U-Boot 编译.....	26
2.3.2 内核编译	27
2.3.3 文件系统编译	29
2.3.4 生成 update.img 镜像.....	30
3. rootfs 根文件系统修改说明.....	31
3.1 修改 rootfs 根文件系统	31
3.2 修改 rootfs 根文件镜像系统大小	33
3.3 基于 rootfs.img 提取 rootfs.tar.gz.....	37
3.4 通过 rootfs.tar.gz 压缩包制作 rootfs.img.....	38
4. Linux 系统文件替换说明.....	40
4.1 通过 Linux 命令行固化	40
4.1.1 替换 U-Boot 镜像.....	41

因我们的存在，让嵌入式应用更简单

4.1.2 替换内核镜像	41
4.2 通过瑞芯微开发工具 RKDevTool 固化	42
5. U-Boot 命令和环境变量说明	46
6. 文件系统中文支持说明	49
7. 系统信息查询	49
8. 文件系统组件安装说明	54
9. 内存分配说明	57
10. 程序开机自启动说明	61
11. CPU 主频调节说明	63
11.1 CPU 主频说明	64
11.2 查看 CPU 主频	65
11.2.1 通过 policy 方式查看	65
11.2.2 通过单个 CPU 核心方式查看	66
11.3 CPU 主频配置	67
11.3.1 通过内核配置方法	68
11.3.2 通过命令行配置方法	71
11.3.3 通过设备树配置方法	72
12. TFTP 使用说明	73
12.1 TFTP 服务器搭建	74
12.2 TFTP 文件传输测试	77
13. NFS 使用说明	80
13.1 NFS 服务器搭建	80
13.2 NFS 共享目录挂载测试	83
14. 基于 TFTP + NFS 的系统启动说明	87
15. Linux 设备驱动说明	94
更多帮助	96

前言

请先按照《Linux 开发环境搭建》文档进行开发环境搭建。由于系统编译过程需下载相关软件工具包，因此请确保上位机 Ubuntu 系统可正常访问互联网。

本说明文档适用开发环境如下：

- (1) Windows 开发环境：Windows 7 64bit、Windows 10 64bit
- (2) Linux 开发环境：VMware16.2.5、Ubuntu22.04.5 64bit
- (3) U-Boot：U-Boot-2017.09
- (4) Kernel：Linux-6.1.115
- (5) Buildroot 文件系统版本：Buildroot-2024.02
- (6) LinuxSDK：LinuxSDK-[版本号]（基于 rk3576_linux6.1_release_v1.1.0）
- (7) 交叉编译工具链如下：

Buildroot 文件系统、应用开发：aarch64-buildroot-linux-gnu-gcc-12.4.0

U-Boot、内核开发：gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu

Linux 系统相关软件包位于产品资料“4-软件资料\Linux\”目录下，包含 U-Boot、Kernel、Filesystem 和 LinuxSDK 文件夹（具体如下表）。

表 1

U-Boot	image\xxx	U-Boot 镜像文件	
	src	U-Boot 源码压缩包	
	U-Boot 特性支持说明文件		
Kernel	image\xxx	内核镜像文件	
	src	Kernel 源码压缩包	
	Linux 内核特性支持说明文件		
Filesystem	buildroot-2024.02	image\xxx	Buildroot 文件系统镜像文件
		src	Buildroot 文件系统源码压缩包
		Buildroot 文件系统特性支持说明文件	
updateimg	update-[版本号].img，Linux 系统镜像		

因我们的存在，让嵌入式应用更简单

LinuxSDK	LinuxSDK-[版本号]。我司基于官方 rk3576_linux6.1_release_v1.1.0 开发的 LinuxSDK 开发包
	rk3576-buildroot-2024.02-sysroot-[版本号].tar.gz 压缩包，包含程序运行时所使用的库（常用开发库、Qt 库等），以及交叉编译工具链
	dl.tar.gz 压缩包。从官网下载的开源软件包，可直接解压，缩减编译时间
	LinuxSDK 特性支持说明文件
Tools	LZ4_old1-dev.zip 压缩包，用于实现数据的快速压缩和解压缩，提高数据传输速度和存储效率
	gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu.tar.gz 压缩包。U-Boot、内核交叉编译工具链

我司提供的相关文件支持系统开发和应用开发，具体说明如下：

表 2

	系统开发	应用开发
文件	LinuxSDK-[版本号].tar.gz	LinuxSDK-[版本号].tar.gz
	gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu.tar.gz	rk3576-buildroot-2024.02-sysroot-[版本号].tar.gz

备注：

- （1） 我司同时单独提供 U-Boot、内核交叉编译工具链：gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu，位于产品资料“4-软件资料\Linux\Tools\”目录下。
- （2） 应用开发交叉编译工具链包含在 LinuxSDK 开发包中，需经编译 LinuxSDK 后才会生成。亦可将 rk3576-buildroot-2024.02-sysroot-[版本号].tar.gz 解压至 LinuxSDK 协同使用。

因我们的存在，让嵌入式应用更简单

1. LinuxSDK 安装

1.1 安装依赖软件

(1) 安装库和工具集

执行如下命令，安装 LinuxSDK 开发包依赖的库和工具集。

备注：安装过程中，请保证 Ubuntu 可正常访问互联网，若提示"*** is already the newest version ***"表示该软件已安装，请忽略。

```
Host# sudo apt-get install -y git ssh make gcc libssl-dev liblz4-tool expect expect-dev g++  
patchelf chrpath gawk texinfo chrpath diffstat binfmt-support qemu-user-static live-build  
bison flex fakeroot cmake gcc-multilib g++-multilib unzip device-tree-compiler ncurses-dev  
libgucharmap-2-90-dev bzip2 expat gpgv2 cpp-aarch64-linux-gnu libgmp-dev libmpc-dev bc  
python-is-python3 python2
```

```
tronlong@tronlong-virtual-machine:~$ sudo apt-get install -y git ssh make gcc libssl-  
dev liblz4-tool expect expect-dev g++ patchelf chrpath gawk texinfo chrpath diffstat  
binfmt-support qemu-user-static live-build bison flex fakeroot cmake gcc-multilib g++  
-multilib unzip device-tree-compiler ncurses-dev libgucharmap-2-90-dev bzip2 expat gp  
gv2 cpp-aarch64-linux-gnu libgmp-dev libmpc-dev bc python-is-python3 python2  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
Note, selecting 'expect' instead of 'expect-dev'  
Note, selecting 'libncurses-dev' instead of 'ncurses-dev'  
bc is already the newest version (1.07.1-3build1).  
bc set to manually installed.  
bzip2 is already the newest version (1.0.8-5build1).  
bzip2 set to manually installed.  
unzip is already the newest version (6.0-26ubuntu3.2).  
unzip set to manually installed.  
The following additional packages will be installed:  
  binutils binutils-common binutils-x86-64-linux-gnu cmake-data  
  cpp-11-aarch64-linux-gnu debootstrap dh-elpa-helper g++-11 g++-11-multilib gcc-11  
  gcc-11-aarch64-linux-gnu-base gcc-11-multilib gir1.2-gucharmap-2.90 git-man  
  icu-devtools lib32asan6 lib32atomic1 lib32gcc-11-dev lib32gcc-s1 lib32gomp1  
  lib32itm1 lib32quadmath0 lib32stdc++-11-dev lib32stdc++6 lib32ubsan1 libasan6  
  libatk-bridge2.0-dev libatk1.0-dev libatspi2.0-dev libbinutils libblkid-dev  
  libbrotli-dev libc-dev-bin libc-devtools libc6-dev libc6-dev-i386 libc6-dev-x32  
  libc6-i386 libc6-x32 libcairo2-dev libcc1-0 libcrypt-dev libctf-nobfd0 libctf0  
  libdatrie-dev libdbus-1-dev libdeflate-dev libdpkg-perl libegl-dev  
  libegl1-mesa-dev libepoxy-dev liberror-perl libexpat1-dev libfakeroot libfdt1
```

图 1

因我们的存在，让嵌入式应用更简单

```
Setting up libxext-dev:amd64 (2:1.3.4-1build1) ...
Setting up libglx-dev:amd64 (1.4.0-1) ...
Setting up libxi-dev:amd64 (2:1.8-1build1) ...
Setting up libxrender-dev:amd64 (1:0.9.10-1build4) ...
Setting up libgl-dev:amd64 (1.4.0-1) ...
Setting up libxft-dev:amd64 (2.3.4-1) ...
Setting up libxtst-dev:amd64 (2:1.2.3-1build4) ...
Setting up libxdamage-dev:amd64 (1:1.1.5-2build2) ...
Setting up libatspi2.0-dev:amd64 (2.44.0-3) ...
Setting up libegl-dev:amd64 (1.4.0-1) ...
Setting up libxcomposite-dev:amd64 (1:0.4.5-1build2) ...
Setting up libxcursor-dev:amd64 (1:1.2.0-2build4) ...
Setting up libepoxy-dev:amd64 (1.5.10-1) ...
Setting up libatk-bridge2.0-dev:amd64 (2.38.0-3) ...
Setting up libxrandr-dev:amd64 (2:1.5.2-1build1) ...
Setting up libxinerama-dev:amd64 (2:1.1.4-3) ...
Setting up libcairo2-dev:amd64 (1.16.0-5ubuntu2) ...
Setting up libgles-dev:amd64 (1.4.0-1) ...
Setting up libglvnd-dev:amd64 (1.4.0-1) ...
Setting up libpango1.0-dev:amd64 (1.50.6+ds-2ubuntu1) ...
Setting up libegl1-mesa-dev:amd64 (23.2.1-1ubuntu3.1~22.04.2) ...
Setting up libgtk-3-dev:amd64 (3.24.33-1ubuntu2.2) ...
Setting up libgucharmap-2-90-dev:amd64 (1:14.0.3-1) ...
tronlong@tronlong-virtual-machine:~$
```

图 2

(2) 配置 Python 版本

执行如下命令，创建 Python 的软链接。

Host# `sudo rm /usr/bin/python`

Host# `sudo ln -s /usr/bin/python3 /usr/bin/python`

```
tronlong@tronlong-virtual-machine:~$ sudo rm /usr/bin/python
tronlong@tronlong-virtual-machine:~$ sudo ln -s /usr/bin/python3 /usr/bin/python
tronlong@tronlong-virtual-machine:~$
```

图 3

执行如下命令，查看 Python 版本，确认是否链接成功。

备注：Python 版本需 3.6.0 及以上。

Host# `ls -al /usr/bin/python*`

Host# `python --version`

```
tronlong@tronlong-virtual-machine:~$ ls -al /usr/bin/python*
lrwxrwxrwx 1 root root 16 11月 6 17:04 /usr/bin/python -> /usr/bin/python3
lrwxrwxrwx 1 root root 9 7月 28 2021 /usr/bin/python2 -> python2.7
-rwxr-xr-x 1 root root 3588440 10月 16 2023 /usr/bin/python2.7
lrwxrwxrwx 1 root root 10 11月 1 18:21 /usr/bin/python3 -> python3.10
-rwxr-xr-x 1 root root 5909000 9月 11 23:47 /usr/bin/python3.10
tronlong@tronlong-virtual-machine:~$ python --version
Python 3.10.12
tronlong@tronlong-virtual-machine:~$
```

图 4

因我们的存在，让嵌入式应用更简单

(3) 安装 LZ4

请将产品资料“4-软件资料\Linux\Tools\”目录下的 LZ4_old1-dev.zip 压缩包拷贝至 Ubuntu，执行如下命令进行解压。

Host# unzip LZ4_old1-dev.zip

```
tronlong@tronlong-virtual-machine:~$ ls LZ4_old1-dev.zip
LZ4_old1-dev.zip
tronlong@tronlong-virtual-machine:~$ unzip LZ4_old1-dev.zip
Archive: LZ4_old1-dev.zip
149d17e2b00c806406c651f1747dc3ce14855247
  creating: LZ4_old1-dev/
  creating: LZ4_old1-dev/.circleci/
  inflating: LZ4_old1-dev/.circleci/config.yml
  creating: LZ4_old1-dev/.circleci/images/
  creating: LZ4_old1-dev/.circleci/images/primary/
  inflating: LZ4_old1-dev/.circleci/images/primary/Dockerfile
  inflating: LZ4_old1-dev/.cirrus.yml
  inflating: LZ4_old1-dev/.gitattributes
  creating: LZ4_old1-dev/.github/
  creating: LZ4_old1-dev/.github/ISSUE_TEMPLATE/
  inflating: LZ4_old1-dev/.github/ISSUE_TEMPLATE/bug_report.md
  inflating: LZ4_old1-dev/.github/ISSUE_TEMPLATE/feature_request.md
  inflating: LZ4_old1-dev/.github/dependabot.yaml
  creating: LZ4_old1-dev/.github/workflows/
  inflating: LZ4_old1-dev/.github/workflows/README.md
  inflating: LZ4_old1-dev/.github/workflows/ci.yml
  inflating: LZ4_old1-dev/.github/workflows/scorecard.yml
  inflating: LZ4_old1-dev/.gitignore
  inflating: LZ4_old1-dev/INSTALL
  inflating: LZ4_old1-dev/LICENSE
  inflating: LZ4_old1-dev/Makefile
  inflating: LZ4_old1-dev/Makefile.inc
  inflating: LZ4_old1-dev/NEWS
  inflating: LZ4_old1-dev/README.md
```

图 5

执行如下命令，进行安装 LZ4。

Host# cd LZ4_old1-dev/

Host# make

Host# sudo make install

Host# sudo install -m 0755 lz4 /usr/bin/lz4

```
tronlong@tronlong-virtual-machine:~$ cd LZ4_old1-dev/
tronlong@tronlong-virtual-machine:~/LZ4_old1-dev$ ls
appveyor.yml  doc      lib      Makefile.inc  programs  tests
build         examples LICENSE NEWS        README.md
contrib       INSTALL  Makefile  ossfuzz       SECURITY.md
tronlong@tronlong-virtual-machine:~/LZ4_old1-dev$ make
make[1]: Entering directory '/home/tronlong/LZ4_old1-dev/lib'
compiling static library
compiling dynamic library 1.9.4
creating versioned links
```

图 6

因我们的存在，让嵌入式应用更简单

```
tronlong@tronlong-virtual-machine:~/LZ4_old1-dev$ sudo make install
[sudo] password for tronlong:
make[1]: Entering directory '/home/tronlong/LZ4_old1-dev/lib'
Installing libraries in /usr/local/lib
Installing headers in /usr/local/include
lz4 libraries installed
make[1]: Leaving directory '/home/tronlong/LZ4_old1-dev/lib'
make[1]: Entering directory '/home/tronlong/LZ4_old1-dev/programs'
Installing binaries in /usr/local/bin
Installing man pages in /usr/local/share/man/man1
lz4 installation completed
make[1]: Leaving directory '/home/tronlong/LZ4_old1-dev/programs'
tronlong@tronlong-virtual-machine:~/LZ4_old1-dev$ sudo install -m 0755 lz4 /usr/bin/lz4
tronlong@tronlong-virtual-machine:~/LZ4_old1-dev$
```

图 7

执行如下命令，查看 LZ4 版本信息。

备注：LZ4 版本需 v1.7.3 及以上。

Host# lz4 -v

```
tronlong@tronlong-virtual-machine:~/LZ4_old1-dev$ lz4 -v
*** LZ4 command line interface 64-bits v1.9.4, by Yann Collet ***
refusing to read from a console
tronlong@tronlong-virtual-machine:~/LZ4_old1-dev$
```

图 8

1.2 解压 LinuxSDK

由我司提供的 LinuxSDK-[版本号].tar.gz 开发包位于产品资料“4-软件资料\Linux\Linux SDK\”目录下，基于瑞芯微的 rk3576_linux6.1_release_v1.1.0 进行开发。

打开 Ubuntu，执行如下命令，创建 RK3576 工作目录。

Host# mkdir -p /home/tronlong/RK3576

```
tronlong@tronlong-virtual-machine:~$ mkdir -p /home/tronlong/RK3576
tronlong@tronlong-virtual-machine:~$ ls
Desktop  Downloads  mnt        Pictures  RK3576  Templates
Documents LZ4_old1-dev Music      Public    snap     Videos
tronlong@tronlong-virtual-machine:~$
```

图 9

请将 LinuxSDK 开发包拷贝至 RK3576 工作目录，执行如下命令，将 LinuxSDK 开发包解压至 RK3576 工作目录。耗时 5min 后解压完成，生成 rk3576_linux6.1_release 文件夹。

备注：LinuxSDK 版本号以实际情况为准。

因我们的存在，让嵌入式应用更简单

```
Host# cd /home/tronlong/RK3576/

Host# tar -zxf LinuxSDK-v1.0.tar.gz
```

```
tronlong@tronlong-virtual-machine:~$ cd /home/tronlong/RK3576/
tronlong@tronlong-virtual-machine:~/RK3576$ ls
LinuxSDK-v1.0.tar.gz
tronlong@tronlong-virtual-machine:~/RK3576$ tar -zxf LinuxSDK-v1.0.tar.gz
tronlong@tronlong-virtual-machine:~/RK3576$ ls
LinuxSDK-v1.0.tar.gz  rk3576_linux6.1_release
tronlong@tronlong-virtual-machine:~/RK3576$
```

图 10

执行如下命令，进入并查看 LinuxSDK 开发包目录。

```
Host# cd rk3576_linux6.1_release/

Host# ls -l
```

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$ ls -l
total 56
drwxrwxr-x 7 tronlong tronlong 4096 2月 18 10:34 app
drwxrwxr-x 16 tronlong tronlong 4096 2月 18 10:34 buildroot
lrwxrwxrwx 1 tronlong tronlong 39 2月 18 10:34 build.sh -> device/rockchip/common/scripts/build.sh
lrwxrwxrwx 1 tronlong tronlong 21 2月 18 10:34 Copyright_Statement.md -> docs/licenses/LICENSE
drwxrwxr-x 9 tronlong tronlong 4096 2月 18 10:35 debian
drwxrwxr-x 3 tronlong tronlong 4096 2月 18 10:34 device
drwxrwxr-x 7 tronlong tronlong 4096 2月 18 10:37 docs
lrwxrwxrwx 1 tronlong tronlong 27 2月 18 10:34 envsetup.sh -> buildroot/build/envsetup.sh
drwxrwxr-x 34 tronlong tronlong 4096 2月 18 10:35 external
drwxrwxr-x 10 tronlong tronlong 4096 2月 18 10:37 hal
lrwxrwxrwx 1 tronlong tronlong 10 2月 18 10:36 kernel -> kernel-6.1
drwxrwxr-x 26 tronlong tronlong 4096 2月 18 10:36 kernel-6.1
lrwxrwxrwx 1 tronlong tronlong 31 2月 18 10:34 Makefile -> device/rockchip/common/Makefile
drwxrwxr-x 3 tronlong tronlong 4096 2月 18 10:35 prebuilts
drwxrwxr-x 9 tronlong tronlong 4096 2月 18 10:36 rkbin
lrwxrwxrwx 1 tronlong tronlong 41 2月 18 10:34 rkflash.sh -> device/rockchip/common/scripts/rkflash.sh
drwxrwxr-x 14 tronlong tronlong 4096 2月 18 10:37 rtos
drwxrwxr-x 5 tronlong tronlong 4096 2月 18 10:37 tools
drwxrwxr-x 24 tronlong tronlong 4096 2月 18 10:37 u-boot
drwxrwxr-x 8 tronlong tronlong 4096 2月 18 10:37 yocto
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$
```

图 11

表 3 LinuxSDK 源码目录结构

app	存放上层应用 APP，主要是一些应用 Demo
buildroot	存放 Buildroot 根文件系统
build.sh	用于编译镜像的脚本文件
Copyright_Statement.md	版权说明文件
debian	存放 Debian 根文件系统

因我们的存在，让嵌入式应用更简单

device/rockchip	存放芯片板级配置以及一些编译和打包固件的脚本和文件
docs	存放官方参考资料
external	存放第三方相关仓库，包括显示、音视频、摄像头、网络、安全等
hal	Bare-metal 系统源码
kernel	kernel-6.1 软链接文件
kernel-6.1	存放 Linux 内核源码
Makefile	LinuxSDK 的顶层 Makefile
output	存放编译生成的固件信息、编译信息、XML、主机环境等
prebuilts	存放交叉编译工具链
rkbin	存放 Rockchip 相关二进制和工具
rkflash.sh	Linux 环境下的系统固化脚本
rtos	存放 RTOS 系统源码
tools	存放 Linux 和 Windows 操作系统下的常用工具
u-boot	存放 2017.09 版本的 U-Boot 源码
yocto	存放 Yocto 根文件系统

1.3 解压 dl.tar.gz 压缩包

dl.tar.gz 用于存放从官网下载的开源软件包，Buildroot 系统编译时可从该目录获取开源包，以节约下载时间，提高编译效率，且能避免因网络环境差异导致编译失败的问题。

请将产品资料“4-软件资料\LinuxSDK\”目录下的 dl.tar.gz 压缩包拷贝至 Ubuntu 系统 "RK3576/rk3576_linux6.1_release/buildroot/" 目录。执行如下命令，将 dl.tar.gz 压缩包解压至当前目录。

```
Host# cd buildroot/
```

```
Host# tar -zxf dl.tar.gz
```

因我们的存在，让嵌入式应用更简单

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$ cd buildroot/
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release/buildroot$ ls
arch      CHANGES      COPYING      envsetup.sh  Makefile.legacy  scripts      utils
archives  Config.in      DEVELOPERS   fs           package          support
board     Config.in.legacy dl.tar.gz    linux        README          system
boot      configs       docs        Makefile     README.rockchip  toolchain
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release/buildroot$ tar -xzf dl.tar.gz
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release/buildroot$ ls
arch      CHANGES      COPYING      docs         Makefile      README.rockchip  toolchain
archives  Config.in      DEVELOPERS   envsetup.sh  Makefile.legacy  scripts          utils
board     Config.in.legacy dl          fs           package        support
boot      configs       dl.tar.gz   linux        README        system
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release/buildroot$
```

图 12

1.4 系统开发环境配置

如需进行 U-Boot、内核开发，请在 LinuxSDK 源码目录下，执行如下命令，配置环境变量。

备注：配置系统开发环境变量前，请勿执行配置应用环境变量，否则会导致系统镜像编译失败。

```
Host# export PATH=/home/tronlong/RK3576/rk3576_linux6.1_release/prebuilts/gcc/linux-
x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu/bin/:$PATH
```

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$ export PATH=/home/tronlong/RK3576/rk3576_linux6.1_release/prebuilts/gcc-
/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu/bin/:$PATH
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$
```

图 13

```
Host# aarch64-none-linux-gnu-gcc -v
```

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$ aarch64-none-linux-gnu-gcc -v
Using built-in specs.
COLLECT_GCC=aarch64-none-linux-gnu-gcc
COLLECT_LTO_WRAPPER=/home/tronlong/RK3576/rk3576_linux6.1_release/prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64-none-
linux-gnu/bin/../libexec/gcc/aarch64-none-linux-gnu/10.3.1/lto-wrapper
Target: aarch64-none-linux-gnu
Configured with: /data/jenkins/workspace/GNU-toolchain/arm-10/src/gcc/configure --target=aarch64-none-linux-gnu --prefix= --with-sysroot=/a
aarch64-none-linux-gnu/libc --with-build-sysroot=/data/jenkins/workspace/GNU-toolchain/arm-10/build-aarch64-none-linux-gnu/install//aarch64-
none-linux-gnu/libc --with-bugurl=https://bugs.linaro.org/ --enable-gnu-indirect-function --enable-shared --disable-libssp --disable-libmud
flap --enable-checking=release --enable-languages=c,c++,fortran --with-gmp=/data/jenkins/workspace/GNU-toolchain/arm-10/build-aarch64-none-
linux-gnu/host-tools --with-mpfr=/data/jenkins/workspace/GNU-toolchain/arm-10/build-aarch64-none-linux-gnu/host-tools --with-mpc=/data/jenk
ins/workspace/GNU-toolchain/arm-10/build-aarch64-none-linux-gnu/host-tools --with-isl=/data/jenkins/workspace/GNU-toolchain/arm-10/build-aa
rch64-none-linux-gnu/host-tools --enable-fix-cortex-a53-843419 --with-pkgversion='GNU Toolchain for the A-profile Architecture 10.3-2021.07
(arm-10.29)'
Thread model: posix
Supported LTO compression algorithms: zlib
gcc version 10.3.1 20210621 (GNU Toolchain for the A-profile Architecture 10.3-2021.07 (arm-10.29))
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$
```

图 14

因我们的存在，让嵌入式应用更简单

1.5 应用开发环境配置

我司提供的 sysroot 压缩包主要包含应用开发时所需的库（常用开发库、Qt 库等），适用于 Linux 系统应用开发，搭配我司 LinuxSDK 协同使用。

请将产品资料“4-软件资料\Linux\LinuxSDK\”目录下的 sysroot 压缩包拷贝至 Ubuntu 系统"RK3576/rk3576_linux6.1_release/"目录。执行如下命令，将其解压，解压完成后会在 buildroot 目录下增加一个 output 目录，此目录包含 sysroot 相关文件。

备注：如需进行系统开发，请将 output 目录删除，否则会导致 LinuxSDK 编译失败。

```
Host# tar -zxf rk3576-buildroot-2024.02-sysroot-v1.0.tar.gz
```

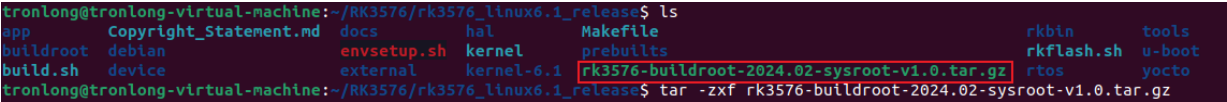


图 15

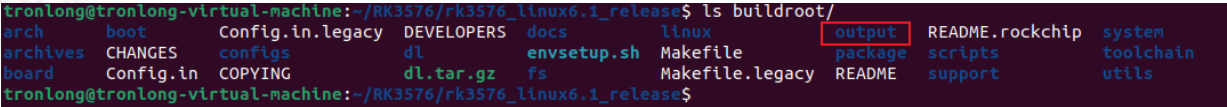


图 16

应用开发的交叉编译工具链位于"buildroot/output/rockchip_rk3576/host/bin/"目录下。

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/host/bin$ ls
2to3                                attr                                glib-mkenums                      msgmerge                          tiff2bw
2to3-3.11                          autoconf                           go                                names                             tiff2pdf
aarch64-buildroot-linux-gnu-addr2line  autoheader                        gobject-query                    ncurses6-config                 tiff2ps
aarch64-buildroot-linux-gnu-ar        autom4te                          gofmt                             nlnja                           tiff2rgba
aarch64-buildroot-linux-gnu-as        automake                           gperf                            ntfs-3g                          tiffcp
aarch64-buildroot-linux-gnu-c++       autotake-1.16                    gresource                        ntfs-3g.probe                   tiffcrop
aarch64-buildroot-linux-gnu-c++_br_real  autotools                         gsettings                        ntfscluster                     tiffddither
aarch64-buildroot-linux-gnu-cc_br_real  autoscan                         gtester-report                  ntfsdump                        tiffinfo
aarch64-buildroot-linux-gnu-c++filt    autoupdate                       gunzip                           ntfsinfo                        tiffmedian
aarch64-buildroot-linux-gnu-cpp        awk                               gzip                             ntfsls                           tiffset
aarch64-buildroot-linux-gnu-elfedit     bison                             hardlink                         openssl                          tiffsplit
aarch64-buildroot-linux-gnu-g++        bzip2                             hatchling                       pal2rgb                          toe
aarch64-buildroot-linux-gnu-g++_br_real  bzip                             hexdump                         patchelf                        toolchain-wrapper
aarch64-buildroot-linux-gnu-gcc        bzcat                             host-make                       pcrc2-config                    tput
aarch64-buildroot-linux-gnu-gcc-12.4.0  bzegrep                          l386                            pcrc2grep                       tracegen
aarch64-buildroot-linux-gnu-gcc-12.4.0.br_real  bzfgrep                         ifnames                        pcrc2test                       tset
aarch64-buildroot-linux-gnu-gcc-ar      bzgrep                           infocmp                         pipesz                           udevadm
aarch64-buildroot-linux-gnu-gcc-br_real  bzlib2                           infotocap                      pkgconf                          uic
aarch64-buildroot-linux-gnu-gcc-nm      bzlib2recover                   intltool-extract                pkg-config                       umount
aarch64-buildroot-linux-gnu-gcc-ranlib  bzless                           intltoolize                     pngfix                           unname26
aarch64-buildroot-linux-gnu-gcov        bzmore                           intltool-merge                  png-fix-itxt                    uncompress
aarch64-buildroot-linux-gnu-gcov-dump    cal                               intltool-prepare                ppm2tiff                         unlz4
aarch64-buildroot-linux-gnu-gcov-tool    captoinfo                       intltool-update                 prlint                            unlzma
aarch64-buildroot-linux-gnu-gdb         chacl                             tpcrm                           pydoc3                          unsquashfs
aarch64-buildroot-linux-gnu-gdb-add-index  chatr                             tps                             pydoc3.11                       unxz
aarch64-buildroot-linux-gnu-gprof        choom                             tsoolize                       pyproject-build                 unzstd
aarch64-buildroot-linux-gnu-ld          clear                             jpegtran                        python                            utmpdump
aarch64-buildroot-linux-gnu-ld.bfd      clear                             kill                             python3                          uuidgen
aarch64-buildroot-linux-gnu-lto-dump     cmake                             last                             python3.11                      uuidparse
aarch64-buildroot-linux-gnu-nm          col                                lastb                           python3.11-config               waitpid
aarch64-buildroot-linux-gnu-objcopy      colcrt                           libpng16-config                 python3-config                   wayland-scanner
aarch64-buildroot-linux-gnu-objdump      colrm                             libpng-config                   python-config                     wheel
aarch64-buildroot-linux-gnu-gcc-ranlib  column                            libtool                         qdbuscpp2xml                    whereis
aarch64-buildroot-linux-gnu-readelf      cpack                            libtoolize                      qdbusxml2cpp                    wrjpgcom
aarch64-buildroot-linux-gnu-size         c_rehash                         linux32                         qemu-aarch64                    x86_64
aarch64-buildroot-linux-gnu-strings     ctest                            linux64                         qemu-edid                       xgettext
aarch64-buildroot-linux-gnu-strip       cygdb                            localedef                       qemu-img                        xkbcomp
aarch64-linux-addr2line                 cython                           logger                          qemu-io                          xml2-config
aarch64-linux-ar                        cythonize                       look                             qemu-nbd                        xmllint
aarch64-linux-as                        djpeg                            lowmfts-3g                      qemu-pr-helper                  xmllint
aarch64-linux-c++                       dmesg                            lsattr                          qemu-storage-daemon            xmlwf
aarch64-linux-c++_br_real               eject                             lsblk                           xslt-config                     xsltproc
aarch64-linux-cc                        event_rpcgen.py                  lscpu                           qmake                           xz
aarch64-linux-cc_br_real                 fadvice                         lsipc                           qmlcachegeen                   xz
```

图 17

```
aarch64-buildroot-linux-gnu-nm          col                                lastb                           python3.11-config               waitpid
aarch64-buildroot-linux-gnu-objcopy      colcrt                           libpng16-config                 python3-config                   wayland-scanner
aarch64-buildroot-linux-gnu-objdump      colrm                             libpng-config                   python-config                     wheel
aarch64-buildroot-linux-gnu-ranlib       column                            libtool                         qdbuscpp2xml                    whereis
aarch64-buildroot-linux-gnu-readelf      cpack                            libtoolize                      qdbusxml2cpp                    wrjpgcom
aarch64-buildroot-linux-gnu-size         c_rehash                         linux32                         qemu-aarch64                    x86_64
aarch64-buildroot-linux-gnu-strings     ctest                            linux64                         qemu-edid                       xgettext
aarch64-buildroot-linux-gnu-strip       cygdb                            localedef                       qemu-img                        xkbcomp
aarch64-linux-addr2line                 cython                           logger                          qemu-io                          xml2-config
aarch64-linux-ar                        cythonize                       look                             qemu-nbd                        xmllint
aarch64-linux-as                        djpeg                            lowmfts-3g                      qemu-pr-helper                  xmllint
aarch64-linux-c++                       dmesg                            lsattr                          qemu-storage-daemon            xmlwf
aarch64-linux-c++_br_real               eject                             lsblk                           xslt-config                     xsltproc
aarch64-linux-cc                        event_rpcgen.py                  lscpu                           qmake                           xz
aarch64-linux-cc_br_real                 fadvice                         lsipc                           qmlcachegeen                   xz
aarch64-linux-cc_c++_br_real             faked                            lsirq                           qmlformat                       xzcat
aarch64-linux-cpp                       fakeroot                         lslocks                         qmlimportscanner               xzcmp
aarch64-linux-cpp_br_real                fallocate                       lsmem                          qmlint                          xzdec
aarch64-linux-elfedit                   fax2ps                          lsns                            qmlin                           xzdiff
aarch64-linux-g++                       fax2tiff                       lz4                             qmltyperegistrar               xzegrep
aarch64-linux-g++_br_real                findcore                        lz4c                           qt.conf                         xzfgrep
aarch64-linux-gcc                       findmnt                         lz4cat                         qtwaylandscanner               xzgrep
aarch64-linux-gcc-12.4.0                 fixqt4headers.pl              lzcat                          qvkgen                         xzless
aarch64-linux-gcc-12.4.0.br_real          flex                            lzcnp                          raw2tiff                       xzmore
aarch64-linux-gcc-ar                     flex++                         lzdifff                        rcc                             yacc
aarch64-linux-gcc-br_real                flock                           lzegrep                        rdjpgcom                       zcat
aarch64-linux-gcc-nm                     gaplication                    lzfgrep                        rename                          zcmp
aarch64-linux-gcc-ranlib                 gawk                           lzgrep                         renice                          zdiff
aarch64-linux-gcov                       gawk-5.3.0                    lzlib                          reset                           zegrep
aarch64-linux-gcov-dump                  gawkbug                       lzless                         rev                             zfgrep
aarch64-linux-gcov-tool                  gdbus                          lzma                           rpcgen                         zforce
aarch64-linux-gdb                        gdbus-codegen                  lzmadec                        script                          zgrep
aarch64-linux-gprof                      gdk-pixbuf-csource             lzmainfo                       scriptlive                      zip
aarch64-linux-ld                         gdk-pixbuf-pixdata             lzmore                         scriptreplay                    zipcloak
aarch64-linux-ld.bfd                    gdk-pixbuf-query-loaders       m4                             setarch                         zipnote
aarch64-linux-ld-dump                   gdk-pixbuf-thumbnailer         nakedevs                       setfacl                         zipsplit
aarch64-linux-nm                         getfacl                        mako-render                    setfacl                         zless
aarch64-linux-objcopy                   getfattr                       mcookie                        setsid                          zmore
aarch64-linux-objdump                   getopt                          mk_cmds                        smtpd.py.11                     znw
aarch64-linux-ranlib                     gettextize                     mkpasswd                       sqfscat                         zstd
aarch64-linux-ranlib                     gto                             nksquashfs                     sqfstar                         zstdcat
aarch64-linux-readelf                   gto-querymodules              nksquashfs                     sqlite3                          zstdgrep
aarch64-linux-size                       glib-compile-resources         moc                             syncqt.pl                       zstdless
aarch64-linux-strings                    glib-compile-schemas          mount                           tabs                             zstdnt
aarch64-linux-strip                     glib-genmarshal               mountpoint                     tar                              zstdnt
aarch64-local-1.16                      glib-gettextize               msgfmt                         tic
```

图 18

如需进行应用开发，请确保安装应用交叉编译工具链软件包后，执行如下命令配置应用环境变量。

因我们的存在，让嵌入式应用更简单

备注：执行如下命令完成应用开发环境变量配置后，当前 Terminal 命令行终端将不再适用 U-Boot、内核开发。如需进行 U-Boot、内核开发，请另开一个 Terminal 命令行终端进行相关工作。

Host# source /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/host/environment-setup

```
tronlong@tronlong-virtual-machine:~$ source /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/host/environment-setup

[Buildroot]

Making embedded Linux easy!

Some tips:
* PATH now contains the SDK utilities
* Standard autotools variables (CC, LD, CFLAGS) are exported
* Kernel compilation variables (ARCH, CROSS_COMPILE, KERNELDIR) are exported
* To configure do "./configure $CONFIGURE_FLAGS" or use the "configure" alias
* To build CMake-based projects, use the "cmake" alias

tronlong@tronlong-virtual-machine:~$
```

图 19

执行如下命令，查看交叉编译工具链版本信息。

Host# aarch64-buildroot-linux-gnu-gcc -v

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$ aarch64-buildroot-linux-gnu-gcc -v
Using built-in specs.
COLLECT_GCC=/home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/host/bin/aarch64-buildroot-linux-gnu-gcc.
br_real
COLLECT_LTO_WRAPPER=/home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/host/libexec/gcc/aarch64-buildroot-linux-gnu/12.4.0/lto-wrapper
Target: aarch64-buildroot-linux-gnu
Configured with: ./configure --prefix=/home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/host --sysconfdir=/home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/host/etc --enable-static --target=aarch64-buildroot-linux-gnu --with-sysroot=/home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/host/aarch64-buildroot-linux-gnu/sysroot --enable-__cxa_atexit --with-gnu-ld --disable-libssp --disable-multilib --disable-decimal-float --enable-plugins --enable-lto --with-gmp=/home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/host --with-mpc=/home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/host --with-mpfr=/home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/host --with-pkgversion='Buildroot 2024.02' --with-bugurl=https://gitlab.com/buildroot.org/buildroot/-/issues --without-zstd --disable-libquadmath --disable-libquadmath-support --enable-tls --enable-threads --without-isl --without-cloog --with-abi=lp64 --with-cpu=cortex-a72.cortex-a53 --enable-languages=c,c++ --with-build-time-tools=/home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/host/aarch64-buildroot-linux-gnu/bin --enable-shared --disable-libgomp
Thread model: posix
Supported LTO compression algorithms: zlib
gcc version 12.4.0 (Buildroot 2024.02)
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$
```

图 20

2. Linux 系统镜像编译、生成

因我们的存在，让嵌入式应用更简单

2.1 配置编译选项

2.1.1 配置 LinuxSDK 编译选项

进入 LinuxSDK 源码目录, 配置 LinuxSDK 编译选项。

Host# ./build.sh lunch:tl3576_evm_defconfig

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$ ./build.sh lunch:tl3576_evm_defconfig
Log colors: message notice warning error fatal

Parsing supported commands...

Log saved at /home/tronlong/RK3576/rk3576_linux6.1_release/output/sessions/2024-12-30_17-45-23
Switching to defconfig: /home/tronlong/RK3576/rk3576_linux6.1_release/device/rockchip/.chip/tl3576_evm_defconfig
mkdir -p /home/tronlong/RK3576/rk3576_linux6.1_release/output/kconf/lxdialog
make CC="gcc" HOSTCC="gcc" \
  obj=/home/tronlong/RK3576/rk3576_linux6.1_release/output/kconf -C /home/tronlong/RK3576/rk3576_linux6.1_rele
ase/device/rockchip/common/kconfig -f Makefile.br conf
make[1]: Entering directory '/home/tronlong/RK3576/rk3576_linux6.1_release/device/rockchip/common/kconfig'
gcc -D_DEFAULT_SOURCE -D_XOPEN_SOURCE=600 -DCURSES_LOC="" -DNCURSES_WIDECHAR=1 -DLOCALE -I/home/tron
long/RK3576/rk3576_linux6.1_release/output/kconf -DCONFIG_="" -MM *.c > /home/tronlong/RK3576/rk3576_linux6.1
_release/output/kconf/.depend 2>/dev/null || :
gcc -D_DEFAULT_SOURCE -D_XOPEN_SOURCE=600 -DCURSES_LOC="" -DNCURSES_WIDECHAR=1 -DLOCALE -I/home/tron
long/RK3576/rk3576_linux6.1_release/output/kconf -DCONFIG_="" -c conf.c -o /home/tronlong/RK3576/rk3576_linu
x6.1_release/output/kconf/conf.o
gcc -D_DEFAULT_SOURCE -D_XOPEN_SOURCE=600 -DCURSES_LOC="" -DNCURSES_WIDECHAR=1 -DLOCALE -I/home/tron
long/RK3576/rk3576_linux6.1_release/output/kconf -DCONFIG_="" -I. -c /home/tronlong/RK3576/rk3576_linux6.1_re
lease/output/kconf/zconf.tab.c -o /home/tronlong/RK3576/rk3576_linux6.1_release/output/kconf/zconf.tab.o
gcc -D_DEFAULT_SOURCE -D_XOPEN_SOURCE=600 -DCURSES_LOC="" -DNCURSES_WIDECHAR=1 -DLOCALE -I/home/tron
long/RK3576/rk3576_linux6.1_release/output/kconf -DCONFIG_="" /home/tronlong/RK3576/rk3576_linux6.1_release/
output/kconf/conf.o /home/tronlong/RK3576/rk3576_linux6.1_release/output/kconf/zconf.tab.o -o /home/tronlong/RK
3576/rk3576_linux6.1_release/output/kconf/conf
rm /home/tronlong/RK3576/rk3576_linux6.1_release/output/kconf/zconf.tab.c
make[1]: Leaving directory '/home/tronlong/RK3576/rk3576_linux6.1_release/device/rockchip/common/kconfig'
#
# configuration written to /home/tronlong/RK3576/rk3576_linux6.1_release/output/.config
#
Using current kernel version(6.1)
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$
```

图 21

2.1.2 配置设备树

我司提供的设备树文件位于 LinuxSDK 源码"kernel/arch/arm64/boot/dts/rockchip/"目
录, 具体说明如下。

表 4

文件名称	说明
tl3576-evm.dts	基础设备树, 支持 TL3576-EVM 评估板基本外设, 不支持 LVDS L CD 显示。
tl3576-lvds-1024x768.dts	基于 tl3576-evm.dts 设备树增加支持 LVDS LCD 显示, 但不支持 MIPI LCD 显示。

评估板使用的设备树文件存在多级的依赖关系, 具体如下表所示。表中的下一级设备
树文件, 被上一级设备树文件所引用。

因我们的存在, 让嵌入式应用更简单

表 5

tl3576-lvds-1024x768.dts			
tl3576-evm.dts			
rk3576.dtsi	rk3576-evb1.dtsi		rk3576-linux.dtsi
rk3576-pinctrl.dtsi	rk3576-evb.dtsi	rk3576-rk806.dtsi	
rockchip-pinconf.dtsi	rk3576-evb-input-keymap.dtsi		

当前默认编译的设备树文件为 tl3576-evm.dts，如需替换为支持 LVDS LCD 显示的设备树，请参考以下步骤修改。

进入 LinuxSDK 源码目录, 执行如下命令, 打开配置文件, 修改内容并保存退出。

Host# vi device/rockchip/rk3576/tl3576_evm_defconfig

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$ vi device/rockchip/rk3576/tl3576_evm_defconfig
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$
```

图 22

修改内容如下：

```
#RK_KERNEL_DTS_NAME="tl3576-evm" //注释默认设备树文件 tl3576-evm
RK_KERNEL_DTS_NAME="tl3576-lvds-1024x768" //添加设备树文件 tl3576-lvds-1024x768
```

```
1 RK_UBOOT_SPL=y
2 #RK_KERNEL_DTS_NAME="tl3576-evm"
3 RK_KERNEL_DTS_NAME="tl3576-lvds-1024x768"
4 RK_USE_FIT_IMG=y
```

图 23

执行如下命令，重新配置 LinuxSDK。

Host# ./build.sh lunch:tl3576_evm_defconfig

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$ ./build.sh lunch:tl3576_evm_defconfig
Log colors: message notice warning error fatal

Log saved at /home/tronlong/RK3576/rk3576_linux6.1_release/output/sessions/2024-12-30_17-55-56
Switching to defconfig: /home/tronlong/RK3576/rk3576_linux6.1_release/device/rockchip/.chip/tl3576_evm_defconfig
#
# configuration written to /home/tronlong/RK3576/rk3576_linux6.1_release/output/.config
#
Using last kernel version(6.1)
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$
```

图 24

2.1.3 配置内核选项

我司默认配置的是 Linux 内核（推荐使用）。如需修改内核编译选项配置，请继续执行"make xxx menuconfig"命令打开图形配置界面。此命令通过读取当前目录的.config 文件，以图形界面的方式提供内核编译配置查看和修改方法。如无需修改内核编译选项配置，可跳过此步骤。

menuconfig 是一套图形化的配置工具，需要 ncurses 库支持。ncurses 库提供了一系列的 API 函数供调用者生成基于文本的图形界面，因此需要先执行如下命令通过网络安装 ncurses 库。

Host# sudo apt-get install libncurses5-dev

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$ sudo apt-get install libncurses5-dev
[sudo] password for tronlong:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
libncurses5-dev is already the newest version (6.3-2ubuntu0.1).
0 upgraded, 0 newly installed, 0 to remove and 44 not upgraded.
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$
```

图 25

Host# sudo apt-get install libncursesw5-dev

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$ sudo apt-get install libncursesw5-dev
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
libncursesw5-dev is already the newest version (6.3-2ubuntu0.1).
0 upgraded, 0 newly installed, 0 to remove and 44 not upgraded.
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$
```

图 26

ncurses 库安装完成后，执行如下命令打开图形配置界面。

Host# ./build.sh kernel-config

因我们的存在，让嵌入式应用更简单

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$ ./build.sh kernel-config
Log colors: message notice warning error fatal
Log saved at /home/tronlong/RK3576/rk3576_linux6.1_release/output/sessions/2024-12-30_17-57-23
Using last kernel version(6.1)

=====
Final configs
=====
RK_BOOT_FIT_ITS=/home/tronlong/RK3576/rk3576_linux6.1_release/device/rockchip/.chip/boot.its
RK_BOOT_IMG=boot.img
RK_BUILDROOT=y
RK_BUILDROOT_CFG=rockchip_rk3576
RK_CHIP=rk3576
RK_CHIP_FAMILY=rk3576
RK_DEBIAN=y
RK_DEBIAN_ARCH=arm64
RK_DEBIAN_BOOKWORM=y
RK_DEBIAN_MIRROR=mirrors.ustc.edu.cn
RK_DEBIAN_VERSION=bookworm
RK_DEFCONFIG=tl3576_evm_defconfig
RK_EXTRA_PARTITION_NUM=2
RK_EXTRA_PARTITION_STR=oem:oem:/oem:ext4:defaults:normal:auto:userdata:userdata:/userdata:ext4:defaults:normal:auto:@@@
RK_KERNEL=y
RK_KERNEL_ARCH=arm64
RK_KERNEL_CFG=rockchip_linux_defconfig
RK_KERNEL_DTS=kernel/arch/arm64/boot/dts/rockchip/tl3576-evm.dts
RK_KERNEL_IMG=kernel/arch/arm64/boot/Image
```

图 27

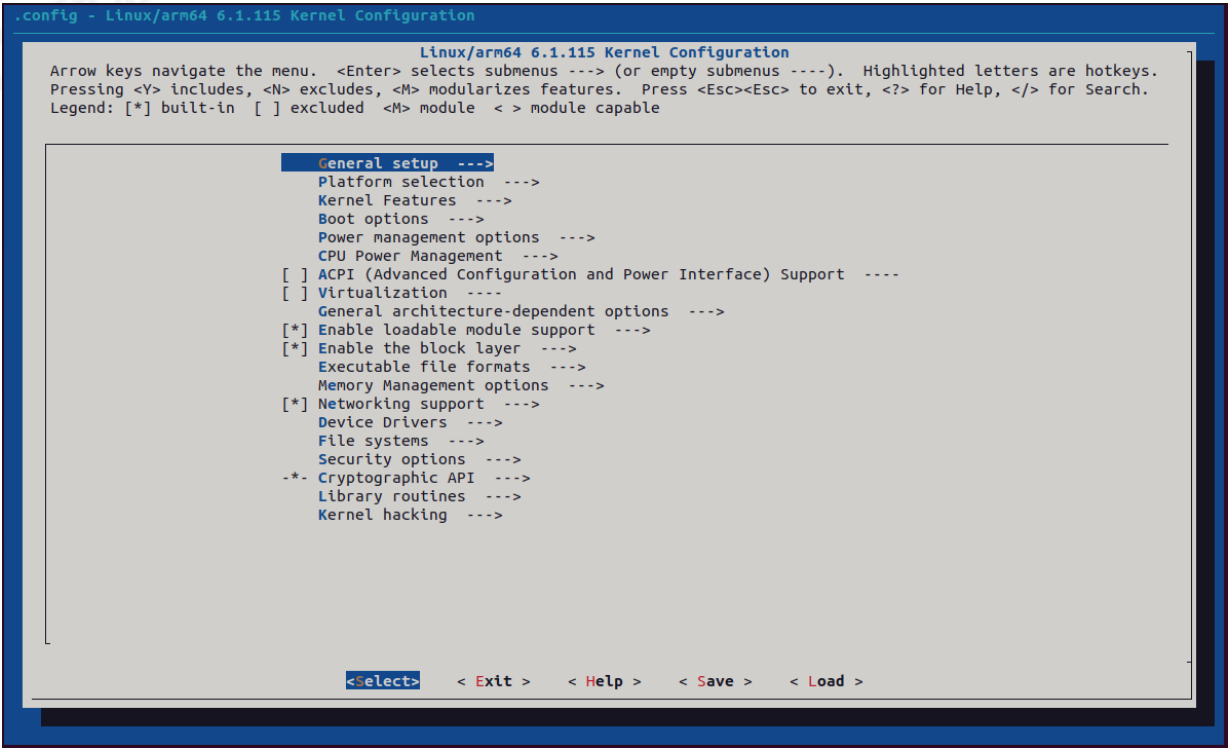


图 28

可通过键盘的方向键选中对应菜单栏。在<Select>被选中的情况下，可按 **Enter** 键进入子菜单。菜单选项中蓝色高亮的字母代表此菜单选项的快捷键，可在键盘上按下对应

因我们的存在，让嵌入式应用更简单

的字母快速选中对应的菜单选项。

每个菜单选项前的括号内容表示当前菜单选项的配置状态。选中对应的菜单选项后，按下 **Y** 键，会将相应的选项配置编译到内核中，同时菜单选项前面变为<*>。按下 **N** 键，不会将相应的选项配置编译到内核中。按下 **M** 键，会将相应的选项配置编译为内核模块，菜单选项前面变为<M>。如需搜索，可按下/键打开搜索框，然后输入要搜索的内容。

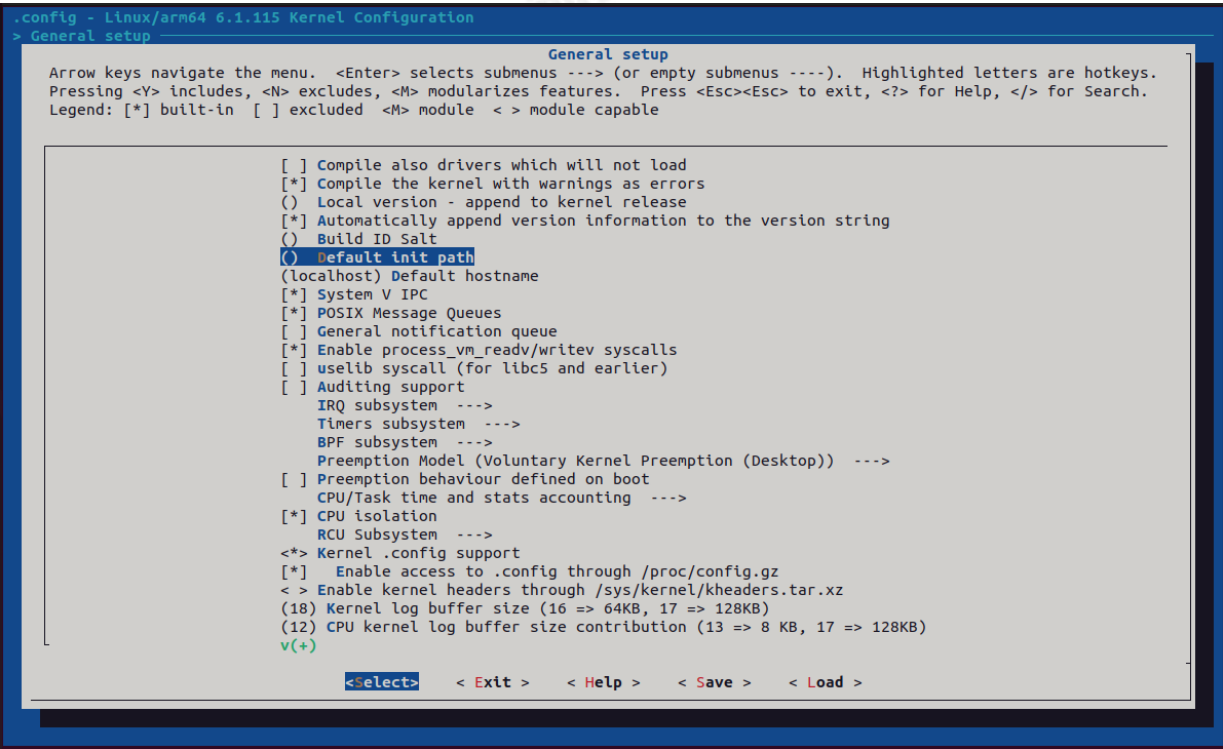


图 29

配置完毕后，选中<Save>，按 Enter 键保存配置选项。然后选中<Exit>，按 Enter 键退出。配置项将自动保存到"kernel/arch/arm64/configs/rockchip_linux_defconfig"。

如需配置为 Linux-RT 内核，请先执行执行如下命令备份 rockchip_linux_defconfig 文件。

```
Host# cd kernel
Host# cp arch/arm64/configs/rockchip_linux_defconfig arch/arm64/configs/rockchip_linux_defconfig-bak
```

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$ cd kernel
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release/kernel$ cp arch/arm64/configs/rockchip_linux_defconfig arch/arm64
/configs/rockchip_linux_defconfig-bak
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release/kernel$
```

图 30

执行如下命令，在 rockchip_linux_defconfig 基础上添加 rockchip_rt.config 的配置。

Host# make ARCH=arm64 rockchip_linux_defconfig rockchip_rt.config

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release/kernel$ make ARCH=arm64 rockchip_linux_defconfig rockchip_rt.conf
ig
#
# configuration written to .config
#
Using .config as base
Merging ./arch/arm64/configs/rockchip_rt.config
Value of CONFIG_ARM_PSCI_CPUIDLE is redefined by fragment ./arch/arm64/configs/rockchip_rt.config:
Previous value: CONFIG_ARM_PSCI_CPUIDLE=y
New value: # CONFIG_ARM_PSCI_CPUIDLE is not set

Value of CONFIG_ARM_ROCKCHIP_DMC_DEVFREQ is redefined by fragment ./arch/arm64/configs/rockchip_rt.config:
Previous value: CONFIG_ARM_ROCKCHIP_DMC_DEVFREQ=y
New value: # CONFIG_ARM_ROCKCHIP_DMC_DEVFREQ is not set

Value of CONFIG_CGROUP_CPUACCT is redefined by fragment ./arch/arm64/configs/rockchip_rt.config:
Previous value: CONFIG_CGROUP_CPUACCT=y
New value: # CONFIG_CGROUP_CPUACCT is not set

Value of CONFIG_CGROUP_SCHED is redefined by fragment ./arch/arm64/configs/rockchip_rt.config:
Previous value: CONFIG_CGROUP_SCHED=y
New value: # CONFIG_CGROUP_SCHED is not set

Value of CONFIG_NO_HZ_FULL is redefined by fragment ./arch/arm64/configs/rockchip_rt.config:
Previous value: # CONFIG_NO_HZ_FULL is not set
New value: CONFIG_NO_HZ_FULL=y
```

图 31

```
Value of CONFIG_CRYPTO_AUTHENC is redefined by fragment ./arch/arm64/configs/rockchip_rt.config:
Previous value: CONFIG_CRYPTO_AUTHENC=y
New value: CONFIG_CRYPTO_AUTHENC=n

Value of CONFIG_CRYPTO_SEQIV is redefined by fragment ./arch/arm64/configs/rockchip_rt.config:
Previous value: CONFIG_CRYPTO_SEQIV=y
New value: CONFIG_CRYPTO_SEQIV=n

Value of CONFIG_CRYPTO_ECHAINIV is redefined by fragment ./arch/arm64/configs/rockchip_rt.config:
Previous value: CONFIG_CRYPTO_ECHAINIV=y
New value: CONFIG_CRYPTO_ECHAINIV=n

#
# merged configuration written to .config (needs make)
#
.config:8072:warning: override: NO_HZ_FULL changes choice state
.config:8089:warning: override: HZ_PERIODIC changes choice state
.config:8090:warning: override: HZ_1000 changes choice state
.config:8091:warning: override: PREEMPT_RT changes choice state
#
# configuration written to .config
#
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release/kernel$
```

图 32

请执行如下命令，保存配置选项至 defconfig 文件，再将 defconfig 文件内容覆盖至 rockchip_linux_defconfig 文件。

Host# make ARCH=arm64 savedefconfig

因我们的存在，让嵌入式应用更简单

Host# mv defconfig arch/arm64/configs/rockchip_linux_defconfig

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release/kernel$ make ARCH=arm64 savedefconfig
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release/kernel$ mv defconfig arch/arm64/configs/rockchip_linux_defconfig
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release/kernel$
```

图 33

若编译 Linux-RT 内核后，如需恢复使用默认内核配置，可执行如下命令进行恢复，然后参考“局部编译”章节进行内核编译即可。

Host# cd kernel

Host# mv arch/arm64/configs/rockchip_linux_defconfig-bak arch/arm64/configs/rockchip_linux_defconfig

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$ cd kernel
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release/kernel$ mv arch/arm64/configs/rockchip_linux_defconfig-bak arch/a
rm64/configs/rockchip_linux_defconfig
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release/kernel$
```

图 34

2.1.4 配置 Buildroot 编译选项

如需修改 Buildroot 配置，请执行如下命令打开 Buildroot 配置界面。若无需修改 Buildroot 配置，可跳过此步骤。

Host# cd /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/

Host# ./envsetup.sh rockchip_rk3576

Host# make menuconfig

```
tronlong@tronlong-virtual-machine:~$ cd /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release/buildroot$ ./envsetup.sh rockchip_rk3576

##### Rockchip Buildroot SDK #####

make: Entering directory '/home/tronlong/RK3576/rk3576_linux6.1_release/buildroot'
mkdir -p /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config/lxdialog
PKG_CONFIG_PATH="" make CC="/usr/bin/gcc" HOSTCC="/usr/bin/gcc" \
  obj=/home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config -C support/k
config -f Makefile.br conf
/usr/bin/gcc -D_DEFAULT_SOURCE -D_XOPEN_SOURCE=600 -DCURSES_LOC="ncurses.h" -DNCURSES_WIDECHAR=1 -DLOCALE -I/home/tronl
ong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config -DCONFIG_="" -MM *.c > /home
/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config/.depend 2>/dev/null || :
/usr/bin/gcc -D_DEFAULT_SOURCE -D_XOPEN_SOURCE=600 -DCURSES_LOC="ncurses.h" -DNCURSES_WIDECHAR=1 -DLOCALE -I/home/tronl
ong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config -DCONFIG_="" -c conf.c -o /
home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config/conf.o
/usr/bin/gcc -D_DEFAULT_SOURCE -D_XOPEN_SOURCE=600 -DCURSES_LOC="ncurses.h" -DNCURSES_WIDECHAR=1 -DLOCALE -I/home/tronl
ong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config -DCONFIG_="" -I. -c /home/tr
onlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config/zconf.tab.c -o /home/tronl
ong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config/zconf.tab.o
/usr/bin/gcc -D_DEFAULT_SOURCE -D_XOPEN_SOURCE=600 -DCURSES_LOC="ncurses.h" -DNCURSES_WIDECHAR=1 -DLOCALE -I/home/tronl
ong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config -DCONFIG_="" /home/tronlong
/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config/conf.o /home/tronlong/RK3576/rk357
6_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config/zconf.tab.o -o /home/tronlong/RK3576/rk3576_li
nux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config/conf
rm /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config/zconf.tab.c
GEN /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/Makefile
Parsing defconfig: /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/configs/rockchip_rk3576_defconfig
Using /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/configs/rockchip_rk3576_defconfig as base
#
# merged configuration written to /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/.config.i
n (needs make)
#
#
# configuration written to /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/.config
#
make: Leaving directory '/home/tronlong/RK3576/rk3576_linux6.1_release/buildroot'
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release/buildroot$
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release/buildroot$ make menuconfig
```

图 35

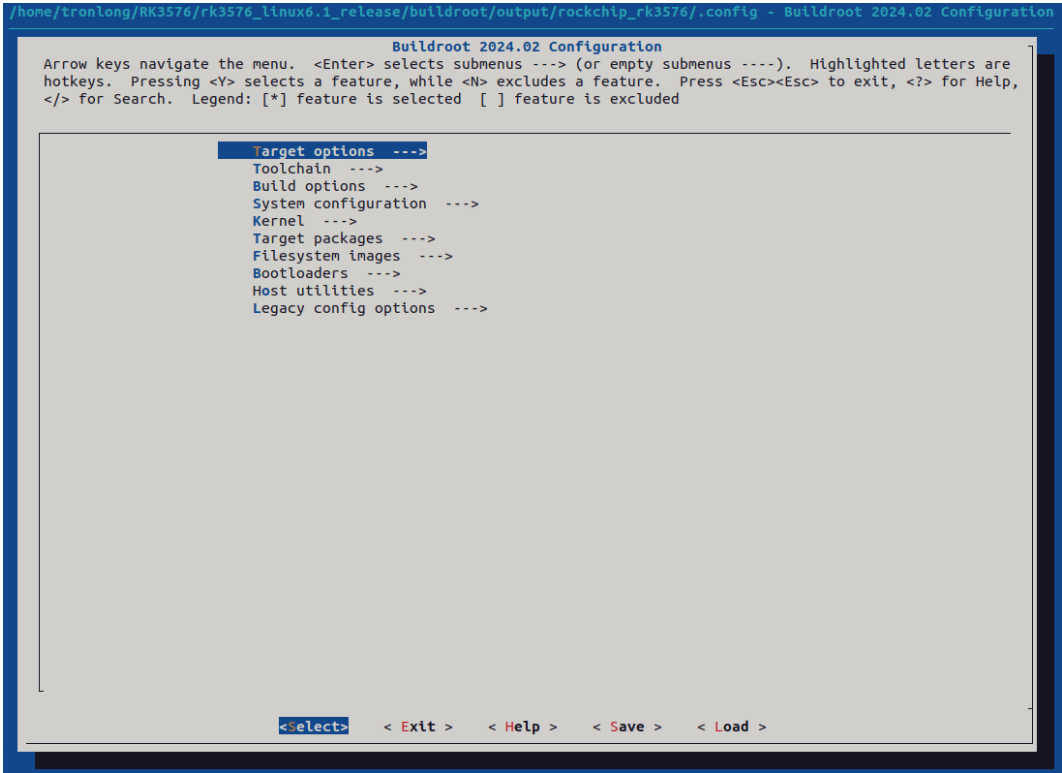


图 36

可通过键盘的方向键选中对应菜单栏。在<Select>被选中的情况下，可按 **Enter** 键进
因我们的存在，让嵌入式应用更简单

入子菜单。菜单选项中蓝色高亮的字母代表此菜单选项的快捷键，可在键盘上按下对应的字母快速选中对应的菜单选项。

每个菜单选项前的括号内容表示当前菜单选项的配置状态。选中对应的菜单选项后，按下 **Y** 键，会将相应的选项配置编译到 Buildroot 中，同时菜单选项前面变为 **< * >**。按下 **N** 键，不会将相应的选项配置编译到 Buildroot 中。如需搜索，可按下 **/** 键打开搜索框，然后输入要搜索的内容。

配置完毕后，选中 **<Save>**，按 **Enter** 键保存配置选项。然后选中 **<Exit>**，按 **Enter** 键退出。

执行如下命令，保存 Buildroot 配置项至 `"/home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/configs/rockchip_rk3576_defconfig"`。

Host# make savedefconfig

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release/buildroot$ make savedefconfig
GEN /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/Makefile
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release/buildroot$
```

图 37

2.2 编译 LinuxSDK

在 LinuxSDK 的 `rk3576_linux6.1_release` 目录下执行如下命令，一键编译生成 U-Boot、Kernel、Buildroot、Recovery 等镜像，并打包生成 Linux 系统镜像。

Host# ./build.sh

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$ ./build.sh
Log colors: message notice warning error fatal

Log saved at /home/tronlong/RK3576/rk3576_linux6.1_release/output/sessions/2024-12-30_18-27-05
Using last kernel version(6.1)

=====
Final configs
=====
RK_BOOT_FIT_ITS=/home/tronlong/RK3576/rk3576_linux6.1_release/device/rockchip/.chip/boot.its
RK_BOOT_IMG=boot.img
RK_BUILDROOT=y
RK_BUILDROOT_CFG=rockchip_rk3576
RK_CHIP=rk3576
RK_CHIP_FAMILY=rk3576
RK_DEBIAN=y
RK_DEBIAN_ARCH=arm64
RK_DEBIAN_BOOKWORM=y
RK_DEBIAN_MIRROR=mirrors.ustc.edu.cn
RK_DEBIAN_VERSION=bookworm
RK_DEFCONFIG=tl3576_evm_defconfig
RK_EXTRA_PARTITION_NUM=2
RK_EXTRA_PARTITION_STR=oem:oem:/oem:ext4:defaults:normal:auto:@userdata:userdata:/userdata:ext4:defaults:normal:auto:@@@
RK_KERNEL=y
RK_KERNEL_ARCH=arm64
RK_KERNEL_CFG=rockchip_linux_defconfig
RK_KERNEL_DTS=kernel/arch/arm64/boot/dts/rockchip/tl3576-evm.dts
```

图 38

预计耗时约 2 小时后编译完成，最终在 rockdev 目录下生成 U-Boot、Kernel、Buildroot、Recovery、Linux 系统等镜像如下所示。

```
Add file: ./package-file
package-file,Add file: ./package-file done,offset=0x800,size=0xe1,userspace=0x1
Add file: ./parameter.txt
parameter,Add file: ./parameter.txt done,offset=0x1000,size=0x227,userspace=0x1,flash_address=0x00000000
Add file: ./MiniloaderAll.bin
bootloader,Add file: ./MiniloaderAll.bin done,offset=0x1800,size=0xbb1f9,userspace=0x177
Add file: ./uboot.img
uboot,Add file: ./uboot.img done,offset=0xbd000,size=0x400000,userspace=0x800,flash_address=0x00004000
Add file: ./misc.img
misc,Add file: ./misc.img done,offset=0x4bd000,size=0xc000,userspace=0x18,flash_address=0x00006000
Add file: ./boot.img
boot,Add file: ./boot.img done,offset=0x4c9000,size=0x2963200,userspace=0x52c7,flash_address=0x00008000
Add file: ./recovery.img
recovery,Add file: ./recovery.img done,offset=0x2e2c800,size=0x3957800,userspace=0x72af,flash_address=0x00028000
Add file: ./rootfs.img
rootfs,Add file: ./rootfs.img done,offset=0x6784000,size=0x52100000,userspace=0xa4200,flash_address=0x00078000
Add file: ./oem.img
oem,Add file: ./oem.img done,offset=0x58884000,size=0x11ef000,userspace=0x23de,flash_address=0x01c78000
Add file: ./userdata.img
userdata,Add file: ./userdata.img done,offset=0x59a73000,size=0x800000,userspace=0x1000,flash_address=0x01cb8000
Add CRC...
Make firmware OK!
----- OK -----
*****rkImageMaker ver 2.29*****
Generating new image, please wait...
Writing head info...
Writing boot file...
Writing firmware...
Generating MD5 data...
MD5 data generated successfully!
New image generated successfully!

Run 'make edit-package-file' if you want to change the package-file.

Running mk-updateimg.sh - build_updateimg succeeded.
Images under /home/tronlong/RK3576/rk3576_linux6.1_release/output/firmware/ are ready!
Running mk-firmware.sh - build_firmware succeeded.
Running 99-all.sh - build_all succeeded.
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$
```

图 39

因我们的存在，让嵌入式应用更简单

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$ ls rockdev/
boot.img MiniLoaderAll.bin misc.img oem.img parameter.txt recovery.img rootfs.img uboot.img update.img userdata.img
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$
```

图 40

表 6 镜像文件说明

boot.img	内核镜像，包含内核和设备树
MiniLoaderAll.bin	DDR 初始化镜像和 U-Boot SPL 镜像
misc.img	misc 镜像
oem.img	oem 镜像，存放用户数据，挂载至文件系统的"/oem"目录
parameter.txt	分区表，update.img 按照此分区表存放各个镜像
recovery.img	recovery 镜像，用于系统升级
rootfs.img	文件系统镜像
uboot.img	U-Boot 镜像
userdata.img	userdata 镜像，挂载至文件系统的"/userdata"目录
update.img	Linux 系统镜像，包含了 MiniLoaderAll.bin、parameter.txt、uboot.img、misc.img、boot.img、recovery.img、rootfs.img、oem.img、userdata.img

请参考《Linux 系统启动卡制作及系统固化》文档，将 Linux 系统镜像文件固化至 Micro SD 卡或 eMMC。

2.3 局部编译

如需单独编译 U-Boot、Linux 内核或文件系统，可参照本章节内容进行局部编译。

2.3.1 U-Boot 编译

请先参考“配置 LinuxSDK 编译选项”章节配置 LinuxSDK。在 LinuxSDK 源码目录下执行如下命令，编译 U-Boot。

```
Host# cd /home/tronlong/RK3576/rk3576_linux6.1_release/
Host# ./build.sh uboot
```

```
tronlong@tronlong-virtual-machine:~$ cd /home/tronlong/RK3576/rk3576_linux6.1_release/
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$ ./build.sh kernel
Log colors: message notice warning error fatal

Log saved at /home/tronlong/RK3576/rk3576_linux6.1_release/output/sessions/2025-03-14_12-38-54
Using last kernel version(6.1)

=====
Final configs
=====
RK_BOOT_FIT_ITS=/home/tronlong/RK3576/rk3576_linux6.1_release/device/rockchip/.chip/boot.its
RK_BOOT_IMG=boot.img
RK_BUILDROOT=y
RK_BUILDROOT_CFG=rockchip_rk3576
RK_CHIP=rk3576
RK_CHIP_FAMILY=rk3576
RK_CHIP_HAS_GPU=y
RK_DEBIAN=y
RK_DEBIAN_ARCH=arm64
RK_DEBIAN_BOOKWORM=y
RK_DEBIAN_MIRROR=mirrors.ustc.edu.cn
RK_DEBIAN_VERSION=bookworm
RK_DEFCONFIG=tl3576_evm_defconfig
RK_EXTRA_PARTITION_NUM=2
RK_EXTRA_PARTITION_STR=auto:oem:auto:ext4:defaults:normal:auto:@auto:userdata:auto:ext4:defaults:normal:auto:@@@
RK_KERNEL=y
RK_KERNEL_ARCH=arm64
RK_KERNEL_CFG=rockchip_linux_defconfig
RK_KERNEL_DTS=kernel/arch/arm64/boot/dts/rockchip/tl3576-evm.dts
RK_KERNEL_IMG=kernel/arch/arm64/boot/Image
```

图 41

```
*****boot_merger ver 1.35*****
Info:Pack loader ok.
creating new idblock from loader...
idblock binary saving at rk3576_idblock_v1.09.107.img
pack loader(SPL) okay! Input: /home/tronlong/RK3576/rk3576_linux6.1_release/rkbin/RKBOOT/RK3576MINIALL.int

/home/tronlong/RK3576/rk3576_linux6.1_release/u-boot
pack loader with new: spl/u-boot-spl.bin

Image(no-signed, version=0): uboot.img (FIT with uboot, trust...) is ready
Image(no-signed): rk3576_idblock_v1.09.107.img (with spl, ddr...) is ready
pack uboot.img okay! Input: /home/tronlong/RK3576/rk3576_linux6.1_release/rkbin/RKTRUST/RK3576TRUST.int

Platform RK3576 is build OK, with new .config(make rk3576_defconfig -j16)
/home/tronlong/RK3576/rk3576_linux6.1_release/prebuilts/gcc/linux-x86/aarch64/gcc-arm-10.3-2021.07-x86_64-aarch64-none-linux-gnu/bin/
/aarch64-none-linux-gnu-
Fri Mar 14 10:16:22 CST 2025
+ cd ..
Running 70-loader.sh - build_uboot succeeded.
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$
```

图 42

编译完成后，在 rockdev 目录下生成 MiniLoaderAll.bin 文件和镜像 uboot.img。

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$ ls rockdev/MiniLoaderAll.bin
rockdev/MiniLoaderAll.bin
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$ ls rockdev/uboot.img
rockdev/uboot.img
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$
```

图 43

2.3.2 内核编译

请先参考“配置 LinuxSDK 编译选项”章节配置 LinuxSDK。在 LinuxSDK 源码目录下执行如下命令，编译内核。

因我们的存在，让嵌入式应用更简单

Host# cd /home/tronlong/RK3576/rk3576_linux6.1_release/

Host# ./build.sh kernel

```
tronlong@tronlong-virtual-machine:~$ cd /home/tronlong/RK3576/rk3576_linux6.1_release/
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$ ./build.sh kernel
Log colors: message notice warning error fatal

Log saved at /home/tronlong/RK3576/rk3576_linux6.1_release/output/sessions/2025-03-14_12-39-48
Using last kernel version(6.1)

=====
Final configs
=====
RK_BOOT_FIT_ITS=/home/tronlong/RK3576/rk3576_linux6.1_release/device/rockchip/.chip/boot.its
RK_BOOT_IMG=boot.img
RK_BUILDROOT=y
RK_BUILDROOT_CFG=rockchip_rk3576
RK_CHIP=rk3576
RK_CHIP_FAMILY=rk3576
RK_CHIP_HAS_GPU=y
RK_DEBIAN=y
RK_DEBIAN_ARCH=arm64
RK_DEBIAN_BOOKWORM=y
RK_DEBIAN_MIRROR=mirrors.ustc.edu.cn
RK_DEBIAN_VERSION=bookworm
RK_DEFCONFIG=tl3576_evm_defconfig
RK_EXTRA_PARTITION_NUM=2
RK_EXTRA_PARTITION_STR=auto:oem:auto:ext4:defaults:normal:auto:auto:userdata:auto:ext4:defaults:normal:auto:@@@
RK_KERNEL=y
RK_KERNEL_ARCH=arm64
RK_KERNEL_CFG=rockchip_linux_defconfig
RK_KERNEL_DTS=kernel/arch/arm64/boot/dts/rockchip/tl3576-evm.dts
RK_KERNEL_IMG=kernel/arch/arm64/boot/Image
```

图 44

```
Created: Fri Mar 14 10:59:45 2025
Image 0 (fdt)
Description: unavailable
Created: Fri Mar 14 10:59:45 2025
Type: Flat Device Tree
Compression: uncompressed
Data Size: 286679 Bytes = 279.96 KiB = 0.27 MiB
Architecture: AArch64
Load Address: 0xffffffff00
Hash algo: sha256
Hash value: d481782a47dd1c556abaacbfd188d04787658368ab4ba4e5dc1a6554c08bb842
Image 1 (kernel)
Description: unavailable
Created: Fri Mar 14 10:59:45 2025
Type: Kernel Image
Compression: uncompressed
Data Size: 42795520 Bytes = 41792.50 KiB = 40.81 MiB
Architecture: AArch64
OS: Linux
Load Address: 0xffffffff01
Entry Point: 0xffffffff01
Hash algo: sha256
Hash value: 3a7148ce7250ee2861c68233c86f58d6e5c3be65216a4743d20b479a79239b5b
Image 2 (resource)
Description: unavailable
Created: Fri Mar 14 10:59:45 2025
Type: Multi-File Image
Compression: uncompressed
Data Size: 324608 Bytes = 317.00 KiB = 0.31 MiB
Hash algo: sha256
Hash value: 83b3b3662c9312987b5e3cc3a3b5b5aef0c0266061306d7db122285cc5ae7480
Default Configuration: 'conf'
Configuration 0 (conf)
Description: unavailable
Kernel: kernel
FDT: fdt
+ ln -rsf kernel/boot.img /home/tronlong/RK3576/rk3576_linux6.1_release/output/firmware/boot.img
Not Found io-domains in kernel/arch/arm64/boot/dts/rockchip/tl3576-evm.dts
Running 10-kernel.sh - build_kernel succeeded.
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$
```

图 45

编译完成后，在 rockdev 目录下生成镜像 boot.img。

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$ ls rockdev/boot.img
rockdev/boot.img
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$
```

图 46

2.3.3 文件系统编译

在 LinuxSDK 源码目录下执行如下命令，编译文件系统。

备注：编译文件系统前，请确保已参考“解压 dl.tar.gz 压缩包”章节解压 dl.tar.gz 压缩包至 LinuxSDK 的"rk3576_linux6.1_release/buildroot/"目录。

Host# ./build.sh buildroot

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$ ./build.sh buildroot
Log colors: message notice warning error fatal

Log saved at /home/tronlong/RK3576/rk3576_linux6.1_release/output/sessions/2025-03-14_11-00-59
Using last kernel version(6.1)
Using rootfs system(buildroot) from cmdline

=====
Final configs
=====
RK_BOOT_FIT_ITS=/home/tronlong/RK3576/rk3576_linux6.1_release/device/rockchip/.chip/boot.its
RK_BOOT_IMG=boot.img
RK_BUILDROOT=y
RK_BUILDROOT_CFG=rockchip_rk3576
RK_CHIP=rk3576
RK_CHIP_FAMILY=rk3576
RK_CHIP_HAS_GPU=y
RK_DEBIAN=y
RK_DEBIAN_ARCH=arm64
RK_DEBIAN_BOOKWORM=y
RK_DEBIAN_MIRROR=mirrors.ustc.edu.cn
RK_DEBIAN_VERSION=bookworm
RK_DEFCONFIG=tl3576_evm_defconfig
RK_EXTRA_PARTITION_NUM=2
RK_EXTRA_PARTITION_STR=auto:oem:auto:ext4:defaults:normal:auto:@auto:userdata:auto:ext4:defaults:normal:auto:@@@
RK_KERNEL=y
RK_KERNEL_ARCH=arm64
RK_KERNEL_CFG=rockchip_linux_defconfig
RK_KERNEL_DTS=kernel/arch/arm64/boot/dts/rockchip/tl3576-evm.dts
RK_KERNEL_IMG=kernel/arch/arm64/boot/Image
```

图 47

```
Running 95-extra-parts.sh for /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/target (buildroot init=
busybox)...
Preparing extra partitions...
Running 97-ldcache.sh for /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/target (buildroot init=busy
box)...
creating ld.so.cache for ...
/home/tronlong/RK3576/rk3576_linux6.1_release/device/rockchip/common/tools/x86_64/ldconfig: Can't open configuration file /home/tron
long/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/target/etc/ld.so.conf: No such file or directory
/home/tronlong/RK3576/rk3576_linux6.1_release/device/rockchip/common/tools/x86_64/ldconfig: /lib/libstdc++.so.6.0.30-gdb.py is not a
n ELF file - it has the wrong magic bytes at the start.

saving cache in little endian encoding
Running 99-owner.sh for /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/target (buildroot init=busybo
x)...
Running 30-rootfs.sh - build_buildroot /home/tronlong/RK3576/rk3576_linux6.1_release/output/buildroot/images succeeded.
Running 30-rootfs.sh - build_rootfs buildroot succeeded.
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$
```

图 48

因我们的存在，让嵌入式应用更简单

编译完成后，在 rockdev 目录下生成镜像 rootfs.img。

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$ ls rockdev/rootfs.img
rockdev/rootfs.img
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$
```

图 49

2.3.4 生成 update.img 镜像

单独编译 U-Boot、内核、文件系统后，可以将所有镜像重新打包，重新生成 update.img。

备注：如需替换 U-Boot、内核、文件系统镜像，可替换"rk3576_linux6.1_release/output/firmware/"目录下对应的镜像，重新生成 update.img。

进入 LinuxSDK 源码目录，执行如下命令，重新生成 update.img。

```
Host# ./build.sh firmware
```

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$ ./build.sh firmware
Log colors: message notice warning error fatal

Log saved at /home/tronlong/RK3576/rk3576_linux6.1_release/output/sessions/2025-03-14_12-40-26
Using last kernel version(6.1)

=====
Final configs
=====
RK_BOOT_FIT_ITS=/home/tronlong/RK3576/rk3576_linux6.1_release/device/rockchip/.chip/boot.its
RK_BOOT_IMG=boot.img
RK_BUILDROOT=y
RK_BUILDROOT_CFG=rockchip_rk3576
RK_CHIP=rk3576
RK_CHIP_FAMILY=rk3576
RK_CHIP_HAS_GPU=y
RK_DEBIAN=y
RK_DEBIAN_ARCH=arm64
RK_DEBIAN_BOOKWORM=y
RK_DEBIAN_MIRROR=mirrors.ustc.edu.cn
RK_DEBIAN_VERSION=bookworm
RK_DEFCONFIG=tl3576_evm_defconfig
RK_EXTRA_PARTITION_NUM=2
RK_EXTRA_PARTITION_STR=auto:oem:auto:ext4:defaults:normal:auto:@auto:userdata:auto:ext4:defaults:normal:auto:@@@
RK_KERNEL=y
RK_KERNEL_ARCH=arm64
RK_KERNEL_CFG=rockchip_linux_defconfig
RK_KERNEL_DTS=kernel/arch/arm64/boot/dts/rockchip/tl3576-evm.dts
RK_KERNEL_IMG=kernel/arch/arm64/boot/Image
```

图 50

编译完成后，在 rockdev 目录下生成镜像 update.img。

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$ ls rockdev/update.img
rockdev/update.img
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release$
```

图 51

3. rootfs 根文件系统修改说明

本章节主要演示 rootfs 根文件系统镜像的修改、提取和镜像制作的方法。

我司已提供 Buildroot rootfs 根文件系统镜像位于“4-软件资料\Linux\Filesystem\buildroot-2024.02\image\buildroot-202402-[版本号]-[Git 系列号]\”目录下（版本号、Git 系列号请以实际情况为准），请将其拷贝至 Ubuntu 任意目录。

```
tronlong@tronlong-virtual-machine:~$ ls rootfs.img
rootfs.img
tronlong@tronlong-virtual-machine:~$
```

图 52

3.1 修改 rootfs 根文件系统

本次操作以增加文件为例，演示 rootfs 根文件系统的修改方法。

在 rootfs.img 文件所在目录下，执行如下命令创建 mount_dir 目录，挂载 rootfs 根文件系统。

Host# mkdir -p ~/mount_dir

Host# sudo mount -o loop rootfs.img ~/mount_dir

```
tronlong@tronlong-virtual-machine:~$ mkdir -p ~/mount_dir
tronlong@tronlong-virtual-machine:~$ sudo mount -o loop rootfs.img ~/mount_dir
[sudo] password for tronlong:
tronlong@tronlong-virtual-machine:~$
```

图 53

执行如下命令，进入 chroot 根文件系统。

Host# sudo chroot ~/mount_dir

```
tronlong@tronlong-virtual-machine:~$ sudo chroot ~/mount_dir
bash-5.2#
```

图 54

执行如下命令，在根文件系统新增 test 文件。

因我们的存在，让嵌入式应用更简单

备注：如需修改或删除文件，操作同理。

Host# touch test

```
tronlong@tronlong-virtual-machine:~$ sudo chroot ~/mount_dir
bash-5.2# touch test
bash-5.2# ls
bin          data  home  lib64  lost+found  oem  rockchip-test  sbin  system  udisk  var
boot         dev  info  libexec media      opt  root          sdcard test   userdata vendor
busybox.fragment etc  lib  linuxrc mnt        proc  run          sys   tmp    usr
```

图 55

执行如下命令，退出并卸载 rootfs 根文件系统。

Host# exit

Host# sudo umount -l ~/mount_dir

```
bash-5.2# exit
exit
tronlong@tronlong-virtual-machine:~$ sudo umount -l ~/mount_dir
tronlong@tronlong-virtual-machine:~$
```

图 56

执行如下命令，重新挂载 rootfs 根文件系统，可确认已成功新增 test 文件。

Host# sudo mount -o loop rootfs.img ~/mount_dir

Host# sudo chroot ~/mount_dir

```
tronlong@tronlong-virtual-machine:~$ sudo mount -o loop rootfs.img ~/mount_dir
tronlong@tronlong-virtual-machine:~$ sudo chroot ~/mount_dir
bash-5.2# ls
bin          data  home  lib64  lost+found  oem  rockchip-test  sbin  system  udisk  var
boot         dev  info  libexec media      opt  root          sdcard test   userdata vendor
busybox.fragment etc  lib  linuxrc mnt        proc  run          sys   tmp    usr
```

图 57

执行如下命令，退出并卸载 rootfs 根文件系统。

Host# exit

Host# sudo umount -l ~/mount_dir

因我们的存在，让嵌入式应用更简单

```
bash-5.2# exit
exit
tronlong@tronlong-virtual-machine:~$ sudo umount -l ~/mount_dir
tronlong@tronlong-virtual-machine:~$
```

图 58

3.2 修改 rootfs 根文件系统镜像系统大小

基于 rootfs 根文件系统镜像增加文件时，需确认镜像剩余空间大小，若剩余空间不足，请按如下方法扩大根文件系统镜像的空间。

(1) 查看 rootfs 根文件系统镜像剩余空间

执行如下命令挂载 rootfs 根文件系统。

Host# sudo mount -o loop rootfs.img ~/mount_dir

```
tronlong@tronlong-virtual-machine:~$ sudo mount -o loop rootfs.img ~/mount_dir
tronlong@tronlong-virtual-machine:~$
```

图 59

执行如下命令，挂载 proc 文件系统。

Host# sudo mount -t proc /proc ~/mount_dir/proc

```
tronlong@tronlong-virtual-machine:~$ sudo mount -t proc /proc ~/mount_dir/proc
tronlong@tronlong-virtual-machine:~$
```

图 60

执行如下命令，使用 chroot 进入 rootfs 根文件系统，查看根文件系统镜像剩余空间。本次查看 rootfs 根文件系统镜像大小为 1.3GByte，剩余空间为 74MByte。

Host# sudo chroot ~/mount_dir

Host# df -lh

```
tronlong@tronlong-virtual-machine:~$ sudo chroot ~/mount_dir
bash-5.2# df -lh
Filesystem      Size  Used Avail Use% Mounted on
/dev/loop12     1.3G  1.1G   74M  94% /
bash-5.2#
```

图 61

因我们的存在，让嵌入式应用更简单

执行如下命令，退出 rootfs 根文件系统目录，并取消挂载。

Host# exit

Host# sudo umount ~/mount_dir/proc

Host# sudo umount -l ~/mount_dir

```
bash-5.2# exit
exit
tronlong@tronlong-virtual-machine:~$ sudo umount ~/mount_dir/proc
tronlong@tronlong-virtual-machine:~$ sudo umount -l ~/mount_dir
tronlong@tronlong-virtual-machine:~$
```

图 62

(2) 修改 rootfs 根文件系统镜像大小

本次操作演示将原容量为 1.3GByte 的 rootfs 根文件系统镜像扩大为 2GByte。

执行如下命令，创建一个内容为空的 rootfs-expanded.img 文件。

Host# dd if=/dev/zero of=rootfs-expanded.img bs=1G count=2

```
tronlong@tronlong-virtual-machine:~$ dd if=/dev/zero of=rootfs-expanded.img bs=1G count=2
2+0 records in
2+0 records out
2147483648 bytes (2.1 GB, 2.0 GiB) copied, 2.03822 s, 1.1 GB/s
tronlong@tronlong-virtual-machine:~$ ls rootfs-expanded.img
rootfs-expanded.img
tronlong@tronlong-virtual-machine:~$
```

图 63

执行如下命令，将 rootfs-expanded.img 格式化为 EXT4 格式。

Host# mkfs.ext4 rootfs-expanded.img

```
tronlong@tronlong-virtual-machine:~$ mkfs.ext4 rootfs-expanded.img
mke2fs 1.46.5 (30-Dec-2021)
Discarding device blocks: done
Creating filesystem with 524288 4k blocks and 131072 inodes
Filesystem UUID: 52761ca1-1c37-40f0-a6c4-db9af1db4065
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

tronlong@tronlong-virtual-machine:~$
```

图 64

执行如下命令，挂载 rootfs.img 根文件系统镜像。

Host# sudo mount -o loop rootfs.img ~/mount_dir/

```
tronlong@tronlong-virtual-machine:~$ sudo mount -o loop rootfs.img ~/mount_dir/
tronlong@tronlong-virtual-machine:~$
```

图 65

执行如下命令，创建 mount_expanded 目录，并挂载 rootfs-expanded.img。

Host# mkdir -p ~/mount_expanded

Host# sudo mount -o loop rootfs-expanded.img ~/mount_expanded/

```
tronlong@tronlong-virtual-machine:~$ mkdir -p ~/mount_expanded
tronlong@tronlong-virtual-machine:~$ sudo mount -o loop rootfs-expanded.img ~/mount_expanded/
tronlong@tronlong-virtual-machine:~$
```

图 66

执行如下命令将 rootfs.img 根文件系统镜像的所有文件迁移至 rootfs-expanded.img，制作成新的 rootfs-expanded.img 根文件系统镜像。

Host# sudo mv ~/mount_dir/* ~/mount_expanded/

Host# sync

```
tronlong@tronlong-virtual-machine:~$ sudo mv ~/mount_dir/* ~/mount_expanded/
tronlong@tronlong-virtual-machine:~$ sync
tronlong@tronlong-virtual-machine:~$
```

图 67

因我们的存在，让嵌入式应用更简单

执行如下命令，挂载 proc 文件系统，并使用 chroot 进入 rootfs-expanded.img。

Host# sudo mount -t proc /proc ~/mount_expanded/proc/

Host# sudo chroot ~/mount_expanded

```
tronlong@tronlong-virtual-machine:~$ sudo mount -t proc /proc ~/mount_expanded/proc/
tronlong@tronlong-virtual-machine:~$ sudo chroot ~/mount_expanded
bash-5.2#
```

图 68

执行如下命令，可查看新制作的 rootfs-expanded.img 根文件系统镜像容量为 2.0GByte。

Host# df -lh

```
bash-5.2# df -lh
Filesystem      Size  Used Avail Use% Mounted on
/dev/loop13     2.0G  1.1G  725M  61% /
bash-5.2#
```

图 69

执行如下命令退出文件系统并卸载。

Host# exit

Host# sudo umount ~/mount_expanded/proc

Host# sudo umount -l ~/mount_expanded

Host# sudo umount -l ~/mount_dir

```
bash-5.2# exit
exit
tronlong@tronlong-virtual-machine:~$ sudo umount ~/mount_expanded/proc
tronlong@tronlong-virtual-machine:~$ sudo umount -l ~/mount_expanded
tronlong@tronlong-virtual-machine:~$ sudo umount -l ~/mount_dir
tronlong@tronlong-virtual-machine:~$
```

图 70

执行如下命令删除原 rootfs.img 镜像，并将 rootfs-expanded.img 镜像重命名为 rootfs.img。

因我们的存在，让嵌入式应用更简单

Host# rm rootfs.img

Host# mv rootfs-expanded.img rootfs.img

```
tronlong@tronlong-virtual-machine:~$ rm rootfs.img
tronlong@tronlong-virtual-machine:~$ mv rootfs-expanded.img rootfs.img
tronlong@tronlong-virtual-machine:~$
```

图 71

3.3 基于 rootfs.img 提取 rootfs.tar.gz

rootfs.tar.gz 为根文件系统文件的集合，并进行了压缩，一般比 rootfs.img 更小，便于存储和传输。本小节演示基于 rootfs.img 提取 rootfs.tar.gz 压缩包的方法，如无需提取 rootfs.tar.gz 压缩包，可跳过本小节，不影响正常使用。

在 rootfs.img 文件所在目录下，执行如下命令挂载 rootfs 根文件系统。

Host# sudo mount -o loop rootfs.img ~/mount_dir

```
tronlong@tronlong-virtual-machine:~$ sudo mount -o loop rootfs.img ~/mount_dir
tronlong@tronlong-virtual-machine:~$ ls ~/mount_dir
bin      data    home   lib64   lost+found  oem    rockchip-test  sbin    system  udisk    var
boot     dev     info   libexec media     opt    root           sdcard  test   userdata vendor
busybox.fragment etc     lib    linuxrc mnt       proc   run           sys     tmp    usr
```

图 72

执行如下命令提取 rootfs.tar.gz，提取完成后，将在当前目录生成 rootfs.tar.gz 压缩包。

Host# sudo tar -czf rootfs.tar.gz -C ~/mount_dir/ .

```
tronlong@tronlong-virtual-machine:~$ sudo tar -czf rootfs.tar.gz -C ~/mount_dir/ .
tronlong@tronlong-virtual-machine:~$ ls rootfs.tar.gz
rootfs.tar.gz
tronlong@tronlong-virtual-machine:~$
```

图 73

执行如下命令，卸载 rootfs 根文件系统。

Host# sudo umount -l ~/mount_dir

因我们的存在，让嵌入式应用更简单

```
tronlong@tronlong-virtual-machine:~$ sudo umount -l ~/mount_dir
tronlong@tronlong-virtual-machine:~$
```

图 74

3.4 通过 rootfs.tar.gz 压缩包制作 rootfs.img

如需将提取的 rootfs.tar.gz 压缩包制作成 rootfs.img 根文件系统镜像，请参考如下方法。否则，可跳过本小节。

执行如下命令，创建 rootfs 目录，将 rootfs.tar.gz 压缩包解压至该目录。

```
Host# mkdir -p rootfs
```

```
Host# sudo tar -zxf rootfs.tar.gz -C rootfs/
```

```
tronlong@tronlong-virtual-machine:~$ mkdir -p rootfs
tronlong@tronlong-virtual-machine:~$ ls
Desktop  Downloads  LZ4_old1-dev.zip  mount_expanded  Pictures  RK3576  rootfs.img  snap  Videos
Documents  LZ4_old1-dev  mount_dir  Music  Public  rootfs  rootfs.tar.gz  Templates
```

图 75

执行如下命令，可查看到解压后的 rootfs 大小为 1.1GByte。因此，制作 rootfs.img 根文件系统镜像需大于 1.1GByte。

```
Host# sudo du -sh rootfs
```

```
tronlong@tronlong-virtual-machine:~$ sudo du -sh rootfs
1.1G    rootfs
tronlong@tronlong-virtual-machine:~$
```

图 76

执行如下命令，创建一个内容为空、容量为 2GByte 的 rootfs.img 文件。

```
Host# dd if=/dev/zero of=rootfs.img bs=1G count=2
```

```
tronlong@tronlong-virtual-machine:~$ dd if=/dev/zero of=rootfs.img bs=1G count=2
2+0 records in
2+0 records out
2147483648 bytes (2.1 GB, 2.0 GiB) copied, 1.82196 s, 1.2 GB/s
tronlong@tronlong-virtual-machine:~$
```

图 77

因我们的存在，让嵌入式应用更简单

执行如下命令，将 rootfs.img 文件格式化为 EXT4 格式。

Host# mkfs.ext4 rootfs.img

```
tronlong@tronlong-virtual-machine:~$ mkfs.ext4 rootfs.img
mke2fs 1.46.5 (30-Dec-2021)
Discarding device blocks: done
Creating filesystem with 524288 4k blocks and 131072 inodes
Filesystem UUID: 5de1da10-5fdd-49d9-8688-3032a6e14573
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

tronlong@tronlong-virtual-machine:~$
```

图 78

执行如下命令挂载 rootfs.img，并将 rootfs.tar.gz 解压至 "~/mount_dir" 挂载目录，制作成新的 rootfs.img 根文件系统镜像。

Host# sudo mount -o loop rootfs.img ~/mount_dir

Host# sudo tar -zxf rootfs.tar.gz -C ~/mount_dir

```
tronlong@tronlong-virtual-machine:~$ sudo mount -o loop rootfs.img ~/mount_dir
tronlong@tronlong-virtual-machine:~$ sudo tar -zxf rootfs.tar.gz -C ~/mount_dir
tronlong@tronlong-virtual-machine:~$
```

图 79

执行如下命令卸载 rootfs.img。

Host# sudo umount -l ~/mount_dir

```
tronlong@tronlong-virtual-machine:~$ sudo umount -l ~/mount_dir
tronlong@tronlong-virtual-machine:~$
```

图 80

执行如下命令，再次挂载 rootfs.img，可确认成功通过 rootfs.tar.gz 压缩包制作 rootfs.img 根文件系统镜像。

因我们的存在，让嵌入式应用更简单

```
Host# sudo mount -o loop rootfs.img ~/mount_dir
Host# ls ~/mount_dir
```

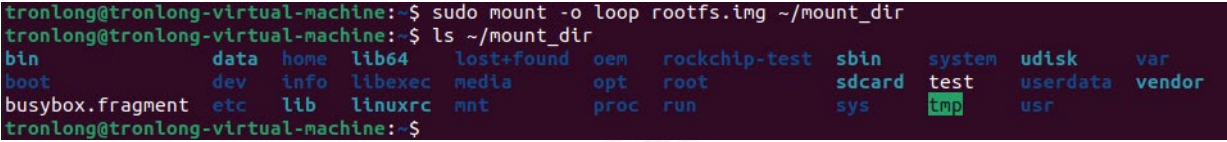


图 81

执行如下命令卸载 rootfs.img。

```
Host# sudo umount -l ~/mount_dir
```

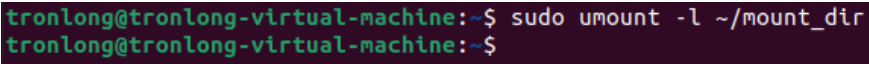


图 82

4. Linux 系统文件替换说明

我司已提供 U-Boot 镜像、Linux 内核镜像、文件系统镜像文件，位于产品资料“4-软件资料\Linux\”目录下（具体如下表），可通过 Linux 命令行或瑞芯微开发工具 RKDevTool 固化至 Linux 系统启动卡或 eMMC。

表 7

U-Boot/image/xxx	uboot.img	U-Boot 镜像文件
Kernel/image/xxx	boot.img	Linux 内核镜像文件
Filesystem/buildroot-2024.02/image/xxx	rootfs.img	Buildroot 文件系统镜像文件

4.1 通过 Linux 命令行固化

如需替换 U-Boot 镜像、内核镜像，请参考如下方法将其替换至 Linux 系统启动卡或 eMMC。

备注：文件系统暂不支持 Linux 命令行方式进行固化，但可通过瑞芯微开发工具 RKDevTool 固化。

4.1.1 替换 U-Boot 镜像

请将 U-Boot 镜像 uboot.img 拷贝至评估板文件系统，执行如下命令将其固化至 Linux 系统启动卡。

备注：如需固化至 eMMC，请将设备节点修改为"/dev/mmcblk0p1"。

Target# dd if=uboot.img of=/dev/mmcblk1p1

Target# sync

Target# reboot

```
root@RK3576-Tronlong:~# ls uboot.img
uboot.img
root@RK3576-Tronlong:~# dd if=uboot.img of=/dev/mmcblk1p1
8192+0 records in
8192+0 records out
4194304 bytes (4.2 MB, 4.0 MiB) copied, 0.718141 s, 5.8 MB/s
root@RK3576-Tronlong:~# sync
root@RK3576-Tronlong:~# reboot
```

图 83

4.1.2 替换内核镜像

请将 Linux 内核镜像拷贝至评估板文件系统，执行如下命令将其固化至 Linux 系统启动卡。

备注：如需固化至 eMMC，请将设备节点修改为"/dev/mmcblk0p3"。

Target# dd if=boot.img of=/dev/mmcblk1p3

Target# sync

Target# reboot

```
root@RK3576-Tronlong:~# ls boot.img
boot.img
root@RK3576-Tronlong:~# dd if=boot.img of=/dev/mmcblk1p3
78789+0 records in
78789+0 records out
40339968 bytes (40 MB, 38 MiB) copied, 7.84177 s, 5.1 MB/s
root@RK3576-Tronlong:~# sync
root@RK3576-Tronlong:~# reboot
```

图 84

因我们的存在，让嵌入式应用更简单

4.2 通过瑞芯微开发工具 RKDevTool 固化

请确保评估板 Micro SD 卡槽未插入 Micro SD 卡，并使用 Type-C 线将评估板 USB3.2 OTG 接口连接至 PC 机 USB 接口。

备注：本小节操作方法仅支持固化 Linux 系统文件至 eMMC，不支持固化至 Linux 系统启动卡。

请参考《Linux 系统启动卡制作及系统固化》文档安装瑞芯微开发工具 RKDevTool。

将评估板断电，长按 USER1(KEY4)按键，将评估板上电启动，此时瑞芯微开发工具界面将会出现提示信息“发现一个 LOADER 设备”，然后松开 USER1 按键。

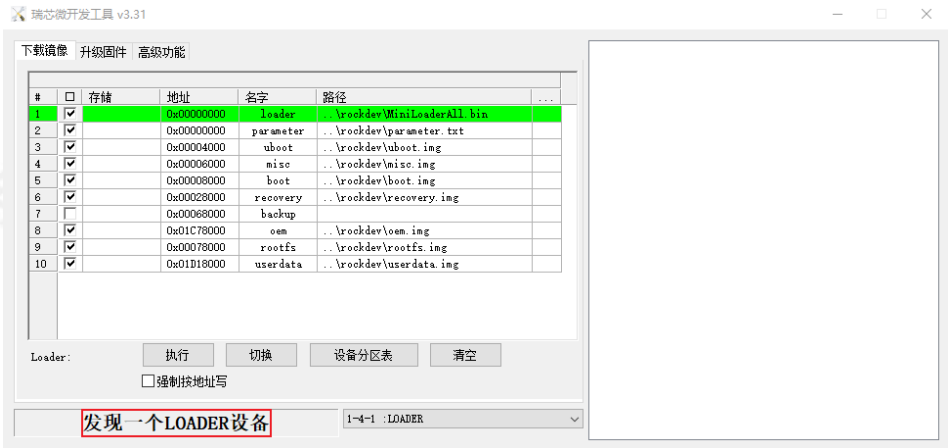


图 85

点击“设备分区表”，获取分区表信息。

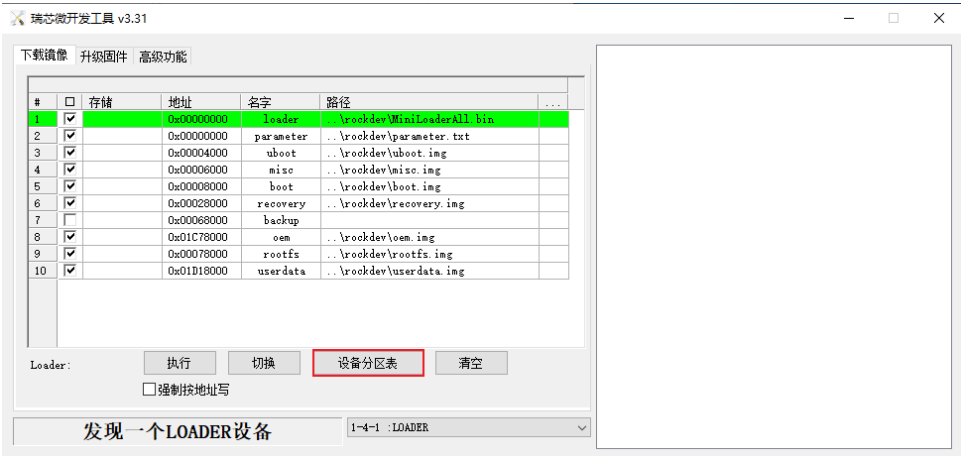


图 86

因我们的存在，让嵌入式应用更简单

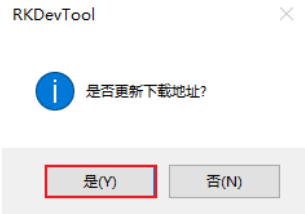


图 87

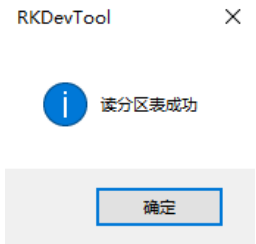


图 88

将待替换的 U-Boot 镜像、Linux 内核镜像或文件系统镜像文件拷贝至 Windows 非中文工作目录下。

- (1) 打开瑞芯微开发工具，选择待替换的 U-Boot 镜像、Linux 内核镜像或文件系统镜像文件存放路径，并勾选对应选项，具体如下图所示。

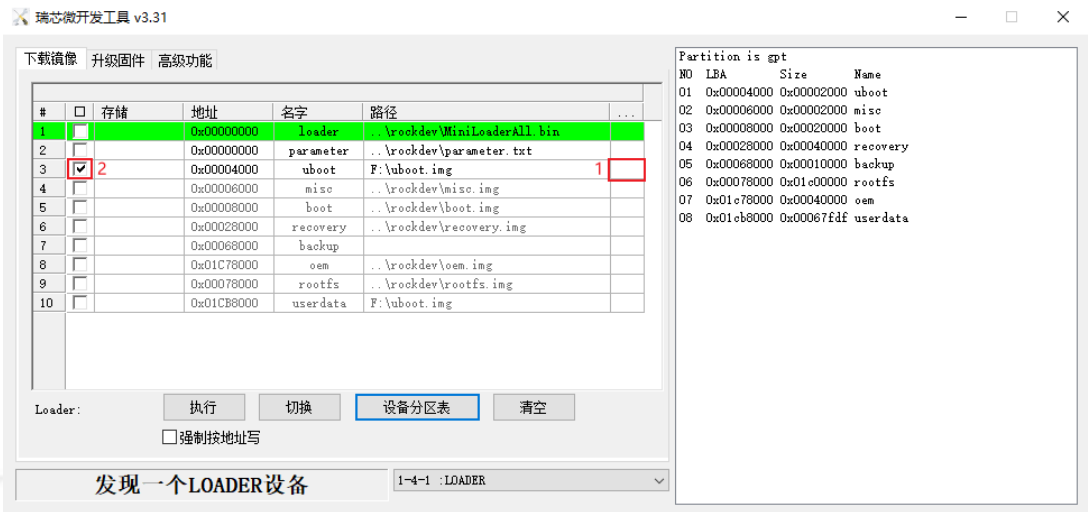


图 89 替换 U-Boot 镜像选项

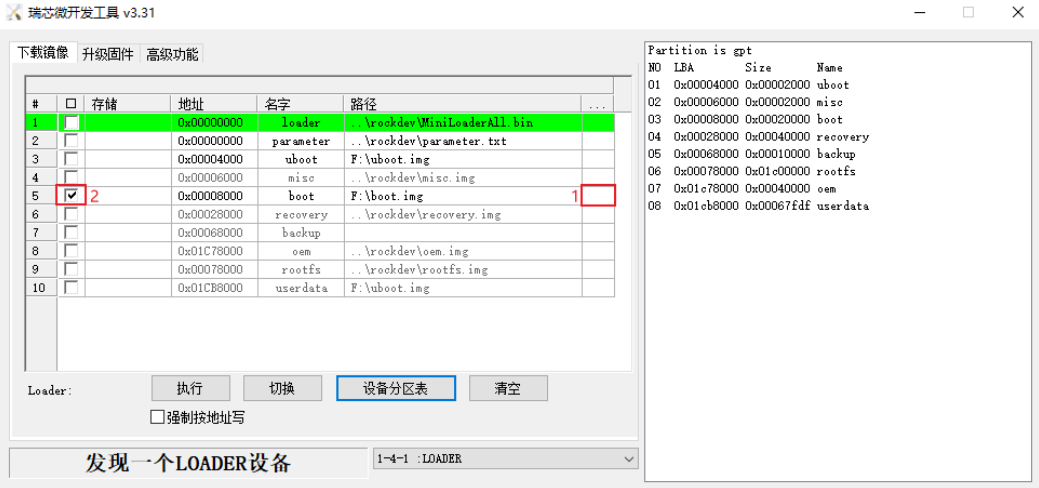


图 90 替换 Linux 内核镜像选项

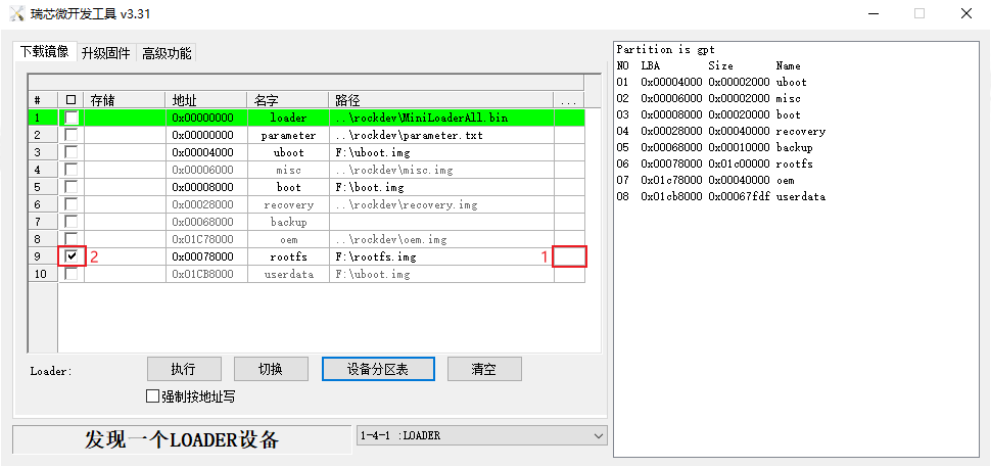


图 91 替换文件系统镜像选项

(2) 本次测试以替换 Linux 内核镜像为例进行演示。

打开瑞芯微开发工具，boot 选择对应 Linux 内核镜像 boot.img 存放路径，并勾选 boot 选项，如下图所示。

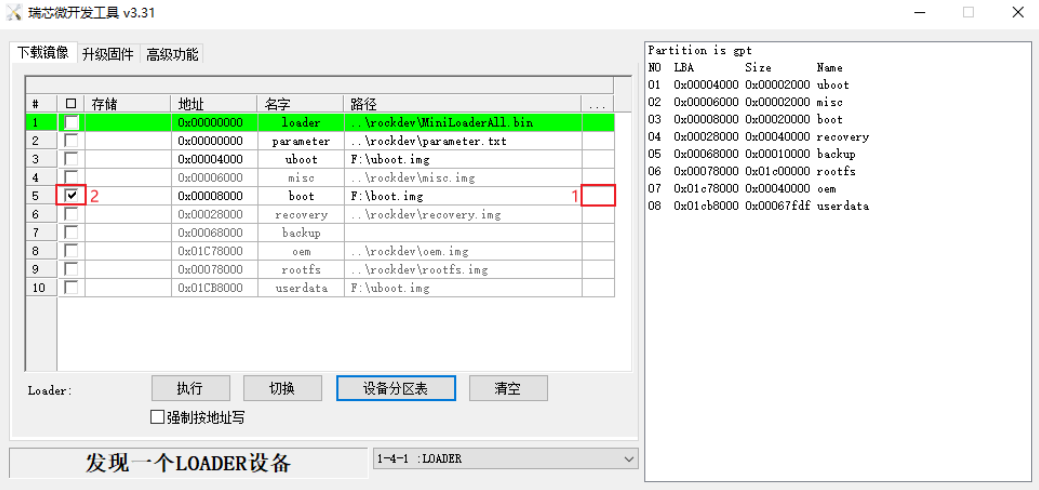


图 92

(3) 点击“执行”选项，将 Linux 内核镜像固化至 eMMC。

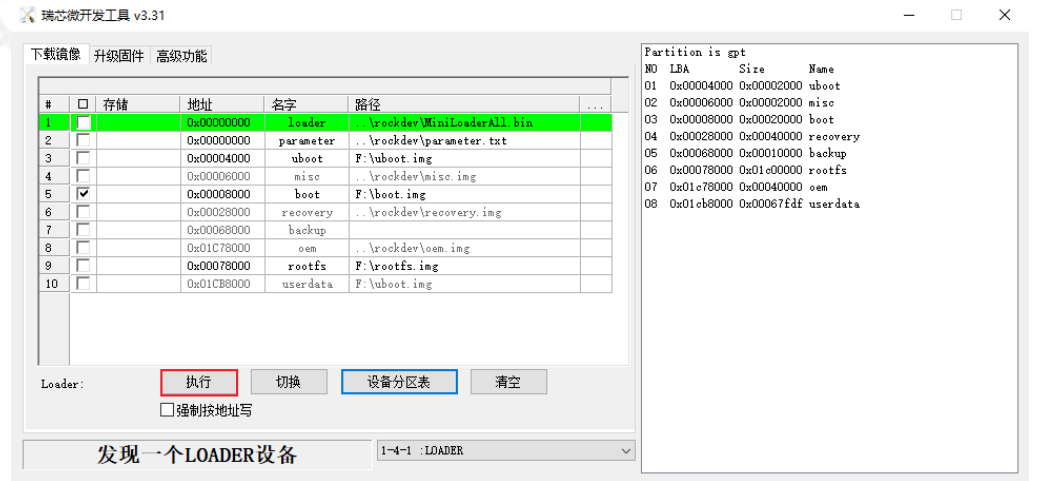


图 93

(4) 直至出现如下界面，提示“下载完成”的信息，表示将 Linux 内核镜像固化至 eMMC 成功，此时评估板将自动重启。



图 94

5. U-Boot 命令和环境变量说明

评估板上电启动后，在 U-Boot 倒计时结束之前按下"Ctrl + C"进入 U-Boot 命令行模式，执行 help 或者?命令，可查看当前 U-Boot 所支持的命令。

U-Boot# help

```
Net: eth0: ethernet@2a220000, eth1: ethernet@2a230000
Hit key to stop autoboot('CTRL+C'): 0
=> <INTERRUPT>
=> <INTERRUPT>
=> <INTERRUPT>
=> help
? - alias for 'help'
android_print_hdr- print android image header
atags - Dump all atags
base - print or set address offset
bdfinfo - print Board Info structure
bidram_dump- Dump bidram layout
blk - Block device sub-system
boot - boot default, i.e., run 'bootcmd'
boot_android- Execute the Android Bootloader flow.
boot_fit- Boot FIT Image from memory or boot/recovery partition
bootavb - Execute the Android avb a/b boot flow.
bootd - boot default, i.e., run 'bootcmd'
booti - boot arm64 Linux Image image from memory
bootm - boot application image from memory
bootp - boot image via network using BOOTP/TFTP protocol
bootz - boot linux zImage image from memory
charge - charge display
cmp - memory compare
coninfo - print console devices and information
cp - memory copy
crc32 - checksum calculation
crypto_sum- crypto checksum engine
dhcp - boot image via network using DHCP/TFTP protocol
dm - Driver model low level access
download- enter rockusb/bootrom download mode
dtimg - manipulate dtb/dtbo Android image
dump_irqs- Dump IRQs
dump_resource- dump resource files
echo - echo args to console
editenv - edit environment variable
env - environment handling commands
exit - exit script
ext2load- load binary file from a Ext2 filesystem
ext2ls - list files in a directory (default /)
ext4load- load binary file from a Ext4 filesystem
ext4ls - list files in a directory (default /)
ext4size- determine a file's size
false - do nothing, unsuccessfully
fastboot- use USB or UDP Fastboot protocol
fatinfo - print information about filesystem
fatload - load binary file from a dos filesystem
fatls - list files in a directory (default /)
fatsize - determine a file's size
fatwrite- write file into a dos filesystem
fdt - flattened device tree utility commands
fstype - Look up a filesystem type
go - start application at address 'addr'
```

图 95

因我们的存在，让嵌入式应用更简单

```
gpio - query and control gpio pins
gpt - GUID Partition Table
help - print command description/usage
iomem - Show iomem data by device compatible(high priority) or node name
lcdputs - print string on video framebuffer
load - load binary file from a filesystem
loop - infinite loop on address range
ls - list files in a directory (default /)
md - memory display
mdio - MDIO utility commands
mii - MII utility commands
mm - memory modify (auto-incrementing address)
mmc - MMC sub-system
mmcinfo - display MMC info
mtd_blk - MTD Block device sub-system
mw - memory write (fill)
nfs - boot image via network using NFS protocol
nm - memory modify (constant address)
part - disk partition related commands
pd - pd default, select power from dtb
ping - send ICMP ECHO_REQUEST to network host
pinmux - show pin-controller muxing
printenv - print environment variables
pxe - commands to get and boot from pxe files
rbrom - Perform RESET of the CPU
reboot - Perform RESET of the CPU, alias of 'reset'
reset - Perform RESET of the CPU
rkimgtest - Test if storage media have rockchip image
rockchip_show_bmp - load and display bmp from resource partition
rockchip_show_logo - load and display log from resource partition
rockusb - use the rockusb Protocol
run - run commands in an environment variable
save - save file to a filesystem
saveenv - save environment variables to persistent storage
scsi - SCSI sub-system
scsiboot - boot from SCSI device
setcurs - set cursor position within screen
setenv - set environment variables
setexpr - set environment variable as the result of eval expression
sf - SPI flash sub-system
showvar - print local hushshell variables
size - determine a file's size
source - run script from memory
sspi - SPI utility command
sysboot - command to get and boot from syslinux files
system_dump - Dump system layout
system_search - Search a available system region
test - minimal test like /bin/sh
tftp - download image via network using TFTP protocol
tftpbootm - tftpbootm aosp/uimage/FIT image via network using TFTP protocol
tftpflash - flash image via network using TFTP protocol
tftpput - TFTP put command, for uploading files to a server
true - do nothing, successfully
ufs - UFS sub-system
ums - Use the UMS [USB Mass Storage]
usb - USB sub-system
usbboot - boot from USB device
version - print monitor, compiler and linker version
vop_dump - dump vop regs
=>
```

图 96

表 8

命令	解析
setenv	设置或者修改环境变量的值
saveenv	用于保存修改后的环境变量，环境变量保存至 eMMC 的 BOOT0 分区
env default -f -a	恢复默认环境变量
printenv	输出当前 U-Boot 环境变量信息
boot	读取环境变量 bootcmd（U-Boot 启动的命令集合）来启动 Linux 系统
help 或?	查看当前 U-Boot 支持的命令

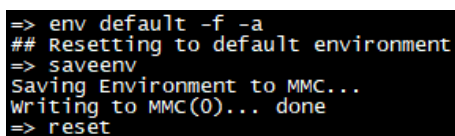
因我们的存在，让嵌入式应用更简单

执行命令 `setenv` 或 `"env default -f -a"` 修改的是运行空间中的环境变量值，须使用 `saveenv` 命令将修改后的环境变量保存起来。否则 U-Boot 重启后，将会使用修改前的环境变量值。环境变量修改完成后，执行 `reset` 命令，即可使用修改后的 U-Boot 环境变量启动。

U-Boot# `env default -f -a`

U-Boot# `saveenv`

U-Boot# `reset`



```
=> env default -f -a
## Resetting to default environment
=> saveenv
Saving Environment to MMC...
Writing to MMC(0)... done
=> reset
```

图 97

在 U-Boot 命令行执行 `printenv` 命令可查看环境变量，如下提供主要 U-Boot 环境变量的解析说明。不同版本的 U-Boot，环境变量可能会有所不同，内容仅供参考。

`/*调试串口波特率为 115200*/`

`baudrate=115200`

`/*系统启动时，将会先设置启动方式*/`

`bootcmd=boot_android ${devtype} ${devnum};boot_fit;bootrkp;run distro_bootcmd;`

`/*设置启动方式为 Llinux 启动*/`

`distro_bootcmd=setenv scsi_need_init; for target in ${boot_targets}; do run bootcmd_${target}; done`

`/*启动延时为 0 秒*/`

`bootdelay=0`

`/*设备树文件读取至 DDR 的加载地址 0x48300000*/`

`fdt_addr_r=0x48300000`

`/*内核镜像读取至 DDR 的加载地址 0x40400000*/`

`kernel_addr_r=0x40400000`

`/*mmc 启动命令。配置 mmc 启动参数*/`

因我们的存在，让嵌入式应用更简单

```
mmc_boot=if mmc dev ${devnum}; then setenv devtype mmc; run scan_dev_for_boot_part; fi
```

6. 文件系统中文支持说明

我司提供的文件系统已加入中文语言库，支持在文件系统输入并显示中文。执行如下命令可创建中文名目录或中文名文件。

Target# touch 创龙科技

Target# mkdir 测试目录

```
root@RK3576-Tronlong:~# touch 创龙科技
root@RK3576-Tronlong:~# ls
创龙科技
root@RK3576-Tronlong:~# mkdir 测试目录
root@RK3576-Tronlong:~# ls
创龙科技 测试目录
root@RK3576-Tronlong:~#
```

图 98

7. 系统信息查询

评估板系统启动后，会自动登录 root 用户，可参照如下方法查询系统相关信息。

(1) 执行如下命令查看操作系统登录欢迎信息。

Target# cat /etc/issue

```
root@RK3576-Tronlong:~# cat /etc/issue
welcome to RK3576 Buildroot
root@RK3576-Tronlong:~#
```

图 99

操作系统登录欢迎信息记录在"/etc/issue"文件，直接修改此文件内容改变系统登录信息。

(2) 执行如下命令查看计算机名字。

Target# hostname

因我们的存在，让嵌入式应用更简单

```
root@RK3576-Tronlong:~# hostname
RK3576-Tronlong
root@RK3576-Tronlong:~#
```

图 100

主机名字记录在"/etc/hostname"里面，可修改此文件内容改变主机名字。

(3) 执行如下命令可查看当前 Linux 内核版本，信息解析示例如下表所示。

Target# cat /proc/version

```
root@RK3576-Tronlong:~# cat /proc/version
Linux version 6.1.115-rt16-03117-ge2d5ae01404e (tronlong@tronlong-virtual-machine) (aarch64-
none-linux-gnu-gcc (GNU Toolchain for the A-profile Architecture 10.3-2021.07 (arm-10.29)) 1
0.3.1 20210621, GNU ld (GNU Toolchain for the A-profile Architecture 10.3-2021.07 (arm-10.29
)) 2.36.1.20210621) #1 SMP Thu Mar 6 11:30:05 CST 2025
root@RK3576-Tronlong:~#
```

图 101 Linux 内核示例

```
root@RK3576-Tronlong:~# cat /proc/version
Linux version 6.1.115-rt16-03117-ge2d5ae01404e-dirty (tronlong@tronlong-virtual-machine) (aa
rch64-none-linux-gnu-gcc (GNU Toolchain for the A-profile Architecture 10.3-2021.07 (arm-10.
29)) 10.3.1 20210621, GNU ld (GNU Toolchain for the A-profile Architecture 10.3-2021.07 (arm
-10.29)) 2.36.1.20210621) #132 SMP PREEMPT_RT Thu Mar 6 10:46:10 CST 2025
root@RK3576-Tronlong:~#
```

图 102 Linux-RT 内核示例

表 9

内容	说明
6.1.115-rt16	内核版本
rt16	实时内核补丁版本
ge2d5ae01404e	Git 序列号
aarch64-none-linux-gnu-gcc	交叉编译工具链
GNU Toolchain for the A-profile Arch itecture 10.3-2021.07(arm-10.29)	交叉编译工具链版本
#1	清理内核后编译次数
SMP/SMP PREEMPT_RT	SMP: Linux 内核 SMP PREEMPT_RT: Linux-RT 内核
Thu Mar 6 10:46:10 CST 2025	内核镜像编译时间

(4) 执行如下命令查看当前 CPU 使用率。

因我们的存在，让嵌入式应用更简单

Target# top

```
top - 10:35:15 up 9 min, 1 user, load average: 0.03, 0.07, 0.06
Tasks: 295 total, 1 running, 294 sleeping, 0 stopped, 0 zombie
%Cpu0 :  0.0/0.0  0[   ]   %Cpu1 :  0.0/0.0  0[   ]   ]
%Cpu2 :  0.0/0.0  0[   ]   %Cpu3 :  0.0/0.0  0[   ]   ]
%Cpu4 :  0.0/0.7  1[   ]   %Cpu5 :  0.0/0.0  0[   ]   ]
%Cpu6 :  0.0/0.7  1[   ]   %Cpu7 :  0.0/0.0  0[   ]   ]
GiB Mem :  8.5/1.9
GiB Swap :  0.0/0.0
```

PID	USER	PR	NI	VIRT	RES	%CPU	%MEM	TIME+	S	COMMAND
1	root	20	0	3.2m	1.2m	0.0	0.1	0:00.42	S	init
545	root	20	0	3.2m	1.2m	0.0	0.1	0:00.01	S	- /sbin/syslogd -n
548	root	20	0	3.8m	1.3m	0.0	0.1	0:00.00	S	- /bin/sh -e /bin/log-guar+
549	root	20	0	8.5m	2.4m	0.0	0.1	0:00.00	S	- sleep 10m
552	root	20	0	3.2m	1.1m	0.0	0.1	0:00.01	S	- /sbin/klogd -n
576	root	20	0	18.3m	3.4m	0.0	0.2	0:00.38	S	- /sbin/udevd -d
935	dbus	20	0	3.3m	1.4m	0.0	0.1	0:00.00	S	- dbus-daemon --system
939	root	20	0	2.2m	1.3m	0.0	0.1	0:00.00	S	- /usr/sbin/rpcbind
951	root	20	0	5.5m	3.2m	0.0	0.2	0:00.00	S	- /usr/libexec/bluetooth/b+
990	dhcpcd	20	0	3.0m	2.0m	0.0	0.1	0:00.00	S	- dhcpcd: [manager] [ip4] +
991	root	20	0	2.8m	2.1m	0.0	0.1	0:00.02	S	- dhcpcd: [privileged +
992	dhcpcd	20	0	2.4m	0.2m	0.0	0.0	0:00.00	S	- dhcpcd: [network pro+
993	dhcpcd	20	0	2.4m	0.2m	0.0	0.0	0:00.00	S	- dhcpcd: [control pro+
1033	chrony	20	0	75.6m	1.8m	0.0	0.1	0:00.00	S	- chronyd
1038	ntp	20	0	74.3m	5.0m	0.0	0.3	0:00.02	S	- /usr/sbin/ntpd -u ntp:ntp
1047	root	20	0	310.5m	57.8m	0.0	2.9	0:00.44	S	- /usr/bin/weston
1351	root	20	0	81.1m	25.1m	0.0	1.3	0:00.15	S	- /usr/libexec/weston+
1352	root	20	0	94.6m	39.2m	0.0	2.0	0:00.46	S	- /usr/libexec/weston+
1048	root	20	0	11.8m	2.3m	0.0	0.1	0:00.00	S	- tee /var/log/weston.log
1056	mosquit+	20	0	8.5m	4.8m	0.0	0.2	0:00.05	S	- /usr/sbin/mosquitto -c /+
1058	root	9	-11	219.4m	7.3m	0.0	0.4	0:00.21	S	- /usr/bin/pulseaudio
1065	root	20	0	9.4m	2.6m	0.0	0.1	0:00.00	S	- sshd: /usr/sbin/sshd [li+
1070	root	20	0	3.2m	1.2m	0.0	0.1	0:00.00	S	- /usr/sbin/telnetd -f
1097	root	20	0	2.4m	1.4m	0.0	0.1	0:00.00	S	- rpc.statd
1318	root	20	0	7.2m	2.1m	0.0	0.1	0:00.00	S	- /bin/sh /usr/bin/usbdevi+
1320	root	20	0	162.3m	3.1m	0.0	0.2	0:00.01	S	- /usr/bin/adbd
1380	root	20	0	2.9m	1.4m	0.0	0.1	0:00.00	S	- rpc.mountd
1389	root	20	0	2.7m	0.8m	0.0	0.0	0:00.06	S	- /usr/sbin/ptp4l -f /etc/+
1396	root	20	0	2.7m	0.9m	0.0	0.0	0:00.01	S	- /usr/sbin/phc2sys -a -r +
1415	root	20	0	8.3m	2.9m	0.0	0.1	0:00.00	S	- /usr/sbin/vsftpd
1448	root	20	0	2.8m	0.2m	0.0	0.0	0:00.00	S	- /usr/sbin/dnsmasq --pid+
1460	root	20	0	10.2m	4.6m	0.0	0.2	0:00.00	S	- /usr/sbin/cupsd -C /etc/+
1491	root	20	0	2.1m	0.1m	0.0	0.0	0:00.01	S	- input-event-daemon
1492	root	20	0	7.1m	3.8m	0.0	0.2	0:00.01	S	- bash
1823	root	20	0	6.9m	2.7m	1.3	0.1	0:00.12	R	- top
2	root	20	0	0.0m	0.0m	0.0	0.0	0:00.01	S	[kthreadd]
3	root	0	-20	0.0m	0.0m	0.0	0.0	0:00.00	I	[rcu_gp]
4	root	0	-20	0.0m	0.0m	0.0	0.0	0:00.00	I	[rcu_par_gp]
5	root	0	-20	0.0m	0.0m	0.0	0.0	0:00.00	I	[slub_flushwq]
6	root	0	-20	0.0m	0.0m	0.0	0.0	0:00.00	I	[netns]
9	root	20	0	0.0m	0.0m	0.0	0.0	0:00.02	I	[kworker/u16:0-events_fr+
10	root	0	-20	0.0m	0.0m	0.0	0.0	0:00.00	I	[mm_percpu_wq]
11	root	20	0	0.0m	0.0m	0.0	0.0	0:00.25	S	[ksoftirqd/0]
12	root	-2	0	0.0m	0.0m	0.0	0.0	0:00.02	S	[ktimers/0]
14	root	-2	0	0.0m	0.0m	0.0	0.0	0:00.11	I	[rcu_preempt]
15	root	-2	0	0.0m	0.0m	0.0	0.0	0:00.05	S	[rcuog/0]
16	root	-2	0	0.0m	0.0m	0.0	0.0	0:00.02	S	[rcuop/0]
17	root	-2	0	0.0m	0.0m	0.0	0.0	0:00.00	S	[rcub/0]
18	root	-2	0	0.0m	0.0m	0.0	0.0	0:00.14	S	[rcuc/0]
19	root	rt	0	0.0m	0.0m	0.0	0.0	0:00.00	S	[migration/0]
20	root	-2	0	0.0m	0.0m	0.0	0.0	0:00.00	S	[irq_work/0]
21	root	20	0	0.0m	0.0m	0.0	0.0	0:00.00	S	[cpuhp/0]

图 103

(5) 执行如下命令查看当前内存使用情况。

Target# cat /proc/meminfo

```
root@RK3576-Tronlong:~# cat /proc/meminfo
MemTotal: 2008880 kB
MemFree: 1653068 kB
MemAvailable: 1838276 kB
Buffers: 7348 kB
Cached: 171684 kB
SwapCached: 0 kB
Active: 32812 kB
Inactive: 200224 kB
Active(anon): 596 kB
Inactive(anon): 59820 kB
Active(file): 32216 kB
Inactive(file): 140404 kB
Unevictable: 4000 kB
Mlocked: 0 kB
SwapTotal: 0 kB
SwapFree: 0 kB
Dirty: 0 kB
Writeback: 0 kB
AnonPages: 58188 kB
Mapped: 57188 kB
Shmem: 6404 kB
KReclaimable: 37584 kB
Slab: 74468 kB
SReclaimable: 37584 kB
SUnreclaim: 36884 kB
KernelStack: 5520 kB
PageTables: 1904 kB
SecPageTables: 0 kB
NFS_Unstable: 0 kB
Bounce: 0 kB
WritebackTmp: 0 kB
CommitLimit: 1004440 kB
Committed_AS: 216832 kB
VmallocTotal: 259653632 kB
VmallocUsed: 10260 kB
VmallocChunk: 0 kB
Percpu: 832 kB
CmaTotal: 16384 kB
CmaAllocated: 3440 kB
CmaReleased: 12944 kB
CmaFree: 12944 kB
root@RK3576-Tronlong:~#
```

图 104

(6) 执行如下命令查看系统环境变量信息。

Target# env

```
root@RK3576-Tronlong:~# env
SHELL=/bin/bash
GST_V4L2_PREFERRED_FOURCC=NV12:YU12:NV16:YUY2
GST_VIDEO_CONVERT_PREFERRED_FORMAT=NV12:NV16:I420:YUY2
UMS_RO=0
QT_TRONLONG_DISPLAY_CONFIG=HDMI
CHROMIUM_FLAGS=--enable-wayland-ime
GST_V4L2_USE_LIBV4L2=1
WESTON_DRM_MIN_BUFFERS=2
WL_OUTPUT_VERSION=3
UMS_MOUNTPOINT=/mnt/ums
GST_INSPECT_NO_COLORS=1
UMS_MOUNT=0
PULSE_HOME=/userdata/.pulse
EDITOR=/bin/vi
WESTON_DRM_KEEP_RATIO=1
GST_DEBUG_NO_COLOR=1
PWD=/root
LOGNAME=root
WESTON_VNC_MIN_BUFFERS=4
UMS_SIZE=256M
HOME=/root
LANG=en_US.UTF-8
WESTON_DRM_PRIMARY=HDMI-A-1
WESTON_FREEZE_DISPLAY=/tmp/.freeze_weston
WAYLANDSINK_FORCE_DMABUF=1
GST_V4L2SRC_DEFAULT_DEVICE=/dev/video-camera0
TERM=vt100
USER=root
UMS_FILE=/userdata/ums_shared.img
AUTOAUDIOSINK_PREFERRED=pulsesink
ADB_SHELL=/bin/bash
GST_V4L2SRC_RK_DEVICES=_mainpath:_selfpath:_bypass:_scale
WESTON_DRM_MIRROR=1
UMS_FSTYPE=vfat
SHLVL=1
USB_FUNCS=adb
WESTON_DISABLE_ATOMIC=1
LC_CTYPE=zh_CN.UTF-8
XDG_RUNTIME_DIR=/var/run
PLAYBIN2_PREFERRED_AUDIOSINK=pulsesink
PATH=/usr/bin:/usr/sbin
GST_V4L2SRC_MAX_RESOLUTION=3840x2160
GST_VIDEO_DECODER_QOS=0
_=/usr/bin/env
root@RK3576-Tronlong:~#
```

图 105

(7) 执行如下命令可查看文件系统支持库的存放目录。

Target# ls /usr/lib/

```
root@RK3576-Tronlong:~# ls /usr/lib/
alsa-lib                               libicalvcal.so
chromium                               libicalvcal.so.1
dhcpcd                                libicalvcal.so.1.0.1
directfb-1.7-7                         libidn2.so
engines-3                              libidn2.so.0
firmware                              libidn2.so.0.3.8
gdk-pixbuf-2.0                         libinput.so
gentlproducer                          libinput.so.10
gio                                    libinput.so.10.13.0
gnome-settings-daemon-3.0              libip4tc.so
gststreamer-1.0                        libip4tc.so.2
gtk-2.0                                libip4tc.so.2.0.0
ld-linux-aarch64.so.1                  libip6tc.so
lib++dfb-1.7.so.7                      libip6tc.so.2
lib++dfb-1.7.so.7.0.0                  libip6tc.so.2.0.0
lib++dfb.so                             libiperf.so
libEGL.so                              libiperf.so.0
libEGL.so.1                            libiperf.so.0.0.0
libFLAC++.so                           libjpeg.so
libFLAC++.so.10                        libjpeg.so.62
libFLAC++.so.10.0.1                    libjpeg.so.62.3.0
libFLAC.so                             libkmod.so
libFLAC.so.12                          libkmod.so.2
libFLAC.so.12.1.0                      libkmod.so.2.4.1
libFirmwareupdate_gcc_v3_1_Basler_pylon_v5_1.so liblavfile-2.2.so.0
libGCBase_gcc_v3_1_Basler_pylon_v5_1.so liblavfile-2.2.so.0.0.0
libGLESv1_CM.so                        liblavfile.so
libGLESv1_CM.so.1                      liblavjpeg-2.2.so.0
libGLESv2.so                           liblavjpeg-2.2.so.0.0.0
libGLESv2.so.2                         liblavjpeg.so
libGenApi_gcc_v3_1_Basler_pylon_v5_1.so liblockfile.so
```

图 106

因我们的存在，让嵌入式应用更简单

8. 文件系统组件安装说明

我司提供的文件系统部分组件需手动安装，详情请查看产品资料“4-软件资料\Ubuntu\Filesystem\buildroot-2024.02\”目录下的《rootfs-feature-support》。本章节主要介绍通过 Buildroot 配置界面安装系统组件。

本小节以安装 nodejs 工具为例，请参考本文档正确安装 LinuxSDK，然后进入 LinuxSDK 源码"buildroot/package/nodejs/"目录，执行如下命令打开 Config.in 文件。

Host# vim Config.in

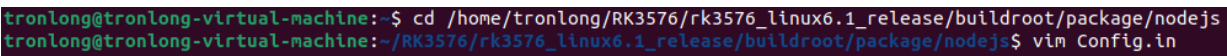


图 107

可查看到 BR2_PACKAGE_NODEJS，该选项为 Buildroot 配置中引导 nodejs 工具安装的关键字。

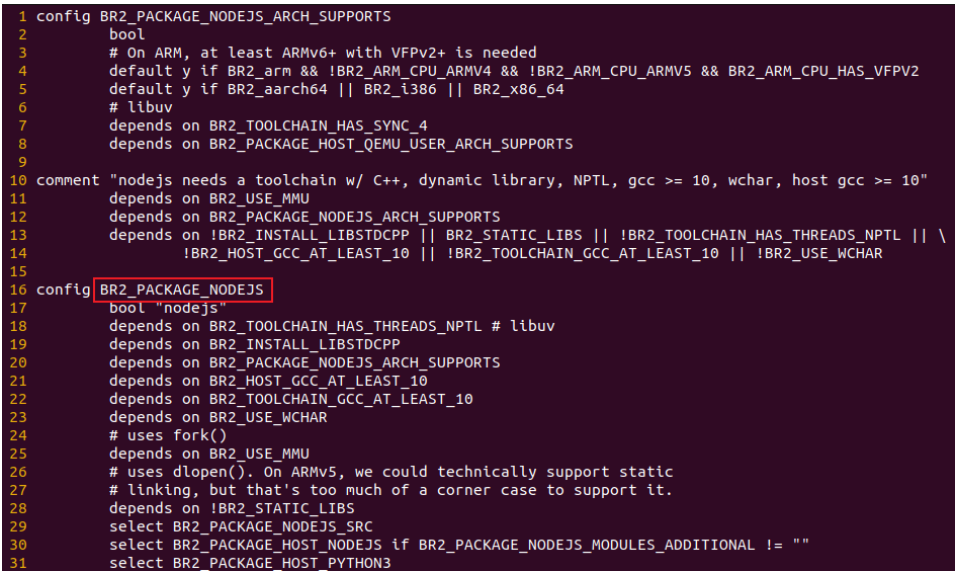


图 108

在 LinuxSDK 源码 buildroot 目录下执行如下命令，打开 Buildroot 配置界面。

Host# make menuconfig

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release/buildroot$ make menuconfig
mkdir -p /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config/lxdialog
PKG_CONFIG_PATH="" make CC="/usr/bin/gcc" HOSTCC="/usr/bin/gcc" \
  -obj=/home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config -C support/kconfig -f
Makefile.br mconf
/usr/bin/gcc -D_DEFAULT_SOURCE -D_XOPEN_SOURCE=600 -DCURSES_LOCALE="ncurses.h" -DNCURSES_WIDECHAR=1 -DLOCALE -I/home/tronlong/RK3576
/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config -DCONFIG_="" -c lxdialog/checklist.c -o /home/t
ronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config/lxdialog/checklist.o
/usr/bin/gcc -D_DEFAULT_SOURCE -D_XOPEN_SOURCE=600 -DCURSES_LOCALE="ncurses.h" -DNCURSES_WIDECHAR=1 -DLOCALE -I/home/tronlong/RK3576
/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config -DCONFIG_="" -c lxdialog/inputbox.c -o /home/tr
onlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config/lxdialog/inputbox.o
/usr/bin/gcc -D_DEFAULT_SOURCE -D_XOPEN_SOURCE=600 -DCURSES_LOCALE="ncurses.h" -DNCURSES_WIDECHAR=1 -DLOCALE -I/home/tronlong/RK3576
/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config -DCONFIG_="" -c lxdialog/menubox.c -o /home/tro
nlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config/lxdialog/menubox.o
/usr/bin/gcc -D_DEFAULT_SOURCE -D_XOPEN_SOURCE=600 -DCURSES_LOCALE="ncurses.h" -DNCURSES_WIDECHAR=1 -DLOCALE -I/home/tronlong/RK3576
/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config -DCONFIG_="" -c lxdialog/textbox.c -o /home/tro
nlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config/lxdialog/textbox.o
/usr/bin/gcc -D_DEFAULT_SOURCE -D_XOPEN_SOURCE=600 -DCURSES_LOCALE="ncurses.h" -DNCURSES_WIDECHAR=1 -DLOCALE -I/home/tronlong/RK3576
/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config -DCONFIG_="" -c lxdialog/utill.c -o /home/tronlo
ng/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config/lxdialog/utill.o
/usr/bin/gcc -D_DEFAULT_SOURCE -D_XOPEN_SOURCE=600 -DCURSES_LOCALE="ncurses.h" -DNCURSES_WIDECHAR=1 -DLOCALE -I/home/tronlong/RK3576
/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config -DCONFIG_="" -c lxdialog/yesno.c -o /home/tronl
ong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config/lxdialog/yesno.o
/usr/bin/gcc -D_DEFAULT_SOURCE -D_XOPEN_SOURCE=600 -DCURSES_LOCALE="ncurses.h" -DNCURSES_WIDECHAR=1 -DLOCALE -I/home/tronlong/RK3576
/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config -DCONFIG_="" -c mconf.c -o /home/tronlong/RK3576
/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config/mconf.o
/usr/bin/gcc -D_DEFAULT_SOURCE -D_XOPEN_SOURCE=600 -DCURSES_LOCALE="ncurses.h" -DNCURSES_WIDECHAR=1 -DLOCALE -I/home/tronlong/RK3576
/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config -DCONFIG_="" -c /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/
output/rockchip_rk3576/build/buildroot-config/zconf.tab.o /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/buil
d/buildroot-config/lxdialog/checklist.o /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/buil
d/buildroot-config/lxdialog/utill.o /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-co
nfig/lxdialog/inputbox.o /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config/lxdia
log/textbox.o /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config/lxdialog/yesno.o
/home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config/lxdialog/menubox.o -lncursesw
-ltinfo -o /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/build/buildroot-config/mconf
GEN /home/tronlong/RK3576/rk3576_linux6.1_release/buildroot/output/rockchip_rk3576/Makefile
```

图 109

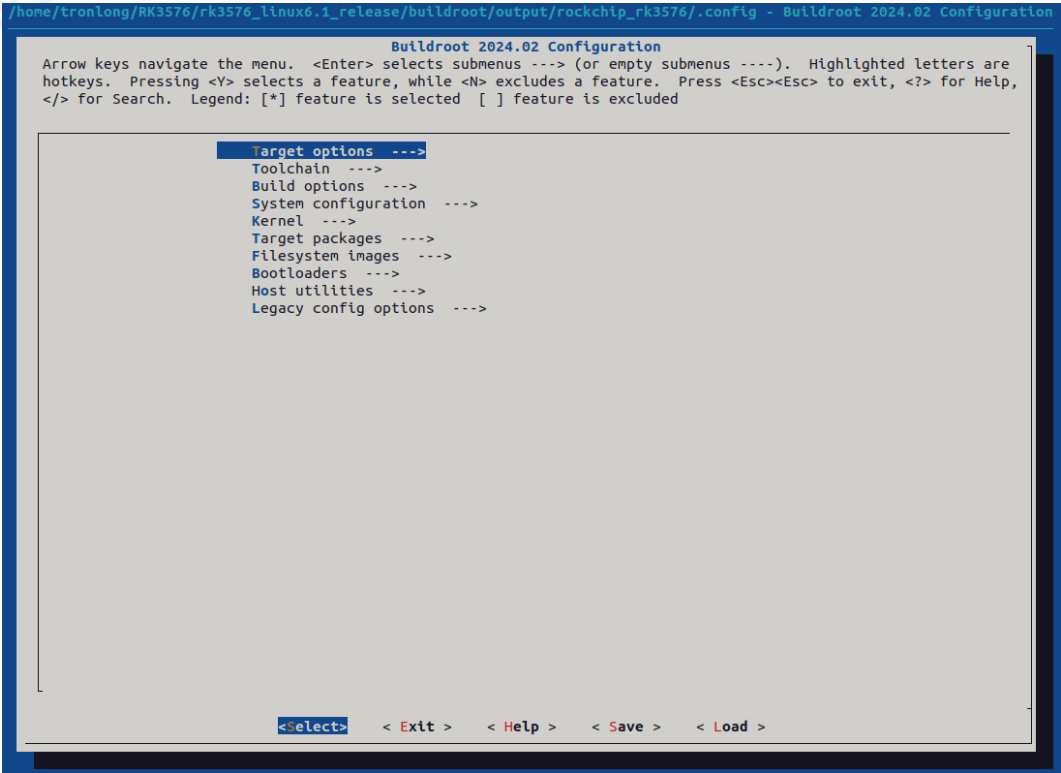


图 110

输入"/"进行搜索，查找 BR2_PACKAGE_NODEJS 选项。

因我们的存在，让嵌入式应用更简单

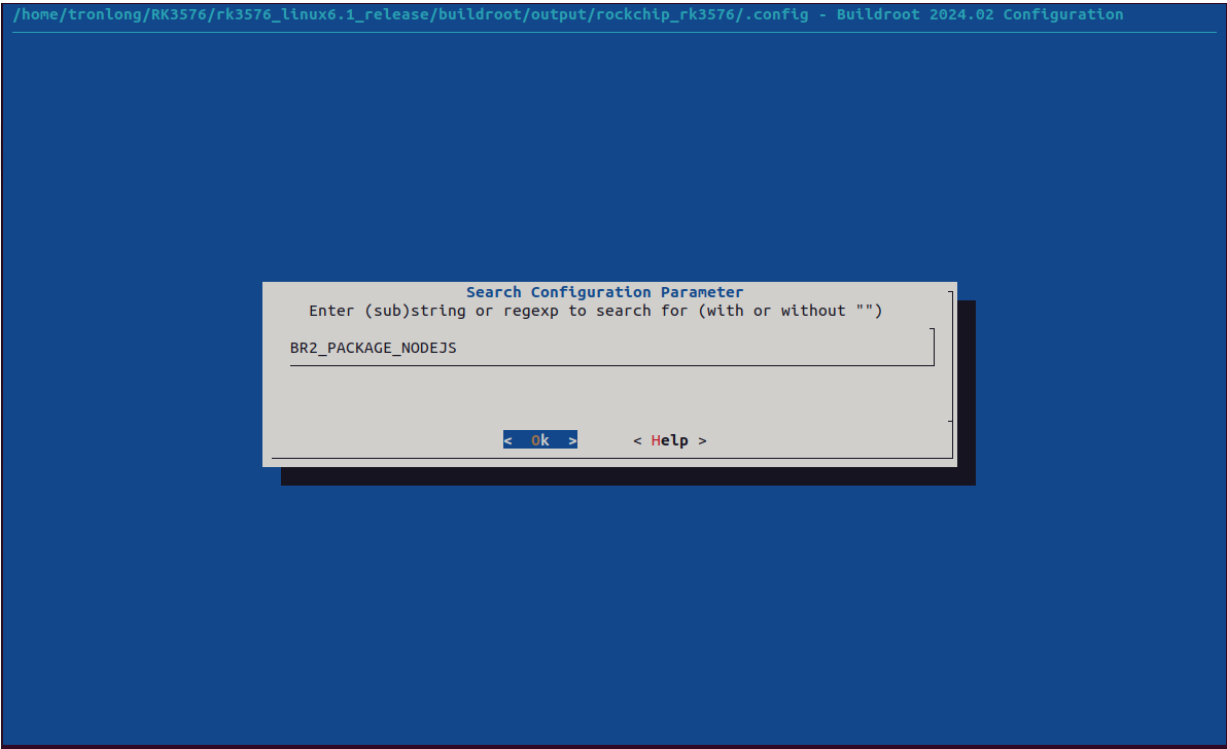


图 111

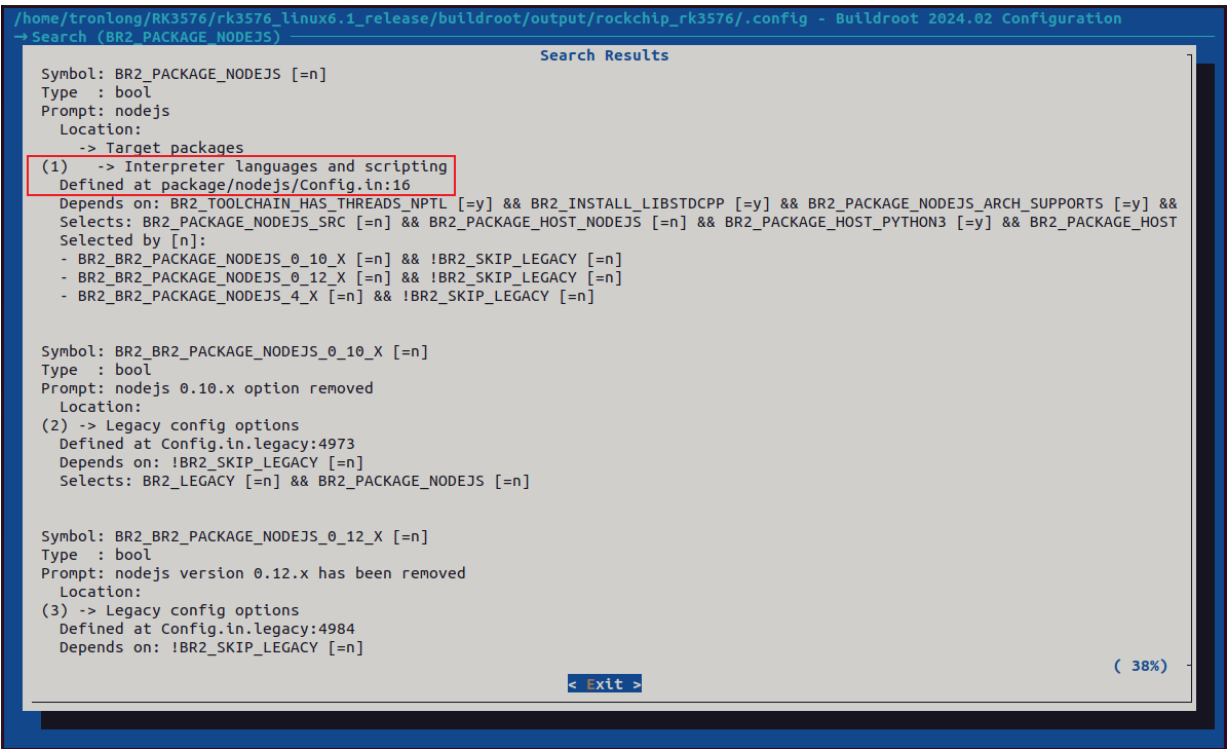


图 112

输入 **1** 选择第一个选项，并按下 **Y** 键配置该选项。

因我们的存在，让嵌入式应用更简单

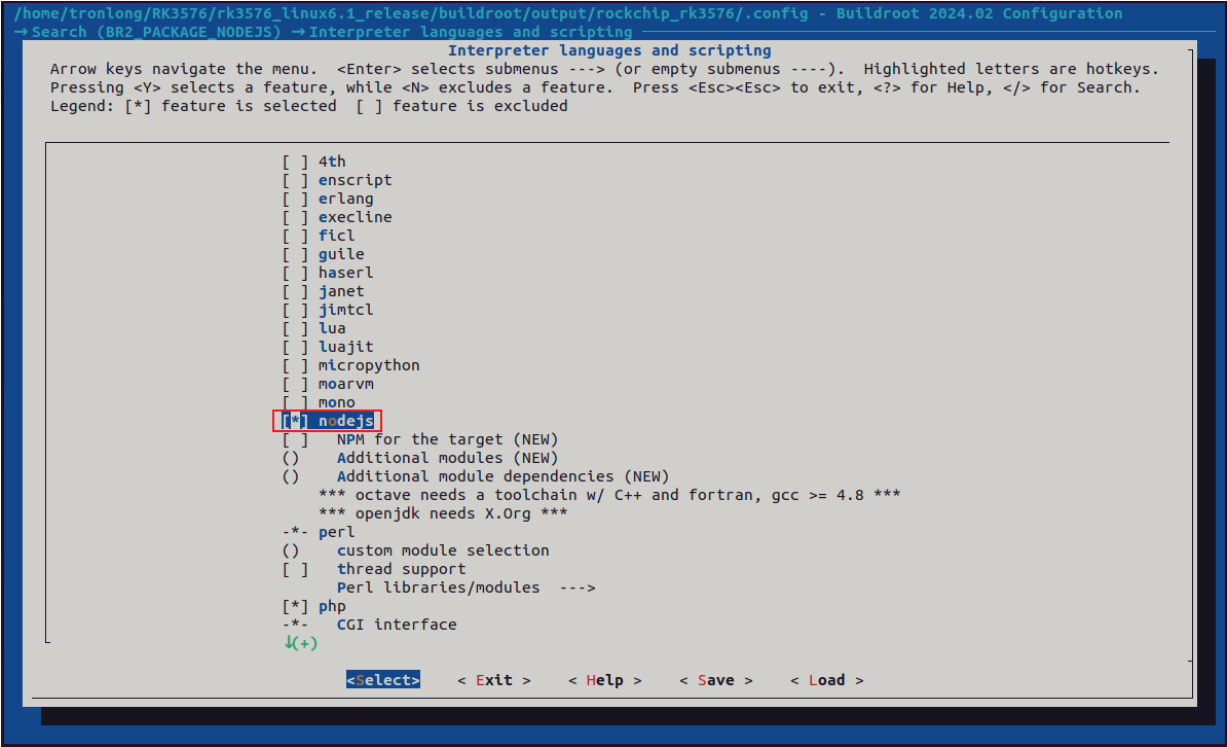


图 113

配置完成后，请参考“Linux 系统镜像编译、生成”章节保存 Buildroot 配置，然后编译生成系统镜像，在生成系统镜像的同时将 nodejs 工具安装在系统镜像内。

9. 内存分配说明

我司提供的 Linux 系统已对核心板 DDR 内存进行划分，具体配置说明如下表。

表 10

DDR 容量	用途	地址范围	大小	说明
2GByte	Linux 系统	0x40200000~0x483fffff	129MByte	属于内核管理的内存
		0x49400000~0xbfffffff	1899MByte	
4GByte	Linux 系统	0x40200000~0x483fffff	129MByte	属于内核管理的内存
		0x49400000~0x13ffffff	3947MByte	
8GByte	Linux 系统	0x40200000~0x483fffff	129MByte	属于内核管理的内存

因我们的存在，让嵌入式应用更简单

		0x49400000~0x23ffffff	8043MByte	
--	--	-----------------------	-----------	--

执行如下命令，可查看内核管理的内存空间。下图为 2GByte 容量 DDR 配置核心板的查询结果。

(1) 执行如下命令查看内核管理的内存空间。

Target# cat /sys/kernel/debug/memblock/memory

```
root@RK3576-Tronlong:~# cat /sys/kernel/debug/memblock/memory
0: 0x0000000040200000..0x00000000483ffffff
1: 0x0000000049400000..0x00000000bfffffff
root@RK3576-Tronlong:~#
```

图 114

(2) 执行如下命令查看内核已经分配的内存。

Target# cat /sys/kernel/debug/memblock/reserved

```
root@RK3576-Tronlong:~# cat /sys/kernel/debug/memblock/reserved
0: 0x0000000040110000..0x00000000401efffff
1: 0x0000000040410000..0x000000004230ffff
2: 0x000000004230f000..0x000000004230ffff
3: 0x0000000048300000..0x0000000048346ffff
4: 0x00000000bbe00000..0x00000000bddffffff
5: 0x00000000be700000..0x00000000bf7ffffff
6: 0x00000000bfa0f200..0x00000000bfa0f707
7: 0x00000000bfa0f740..0x00000000bfa0fc1f
8: 0x00000000bfa0fc40..0x00000000bfa0fdc7
9: 0x00000000bfa0fe00..0x00000000bfa97fff
10: 0x00000000bfa9a000..0x00000000bfa9b11f
11: 0x00000000bfa9b140..0x00000000bfa9b19f
12: 0x00000000bfa9b1c0..0x00000000bfa9b1c7
13: 0x00000000bfa9b200..0x00000000bfa9b207
14: 0x00000000bfa9b240..0x00000000bfa9b385
15: 0x00000000bfa9b3c0..0x00000000bfa9b505
16: 0x00000000bfa9b540..0x00000000bfaac86b
17: 0x00000000bfaac870..0x00000000bfaac89b
18: 0x00000000bfaac8a0..0x00000000bfaac8cb
19: 0x00000000bfaac8d0..0x00000000bfaac8fb
20: 0x00000000bfaac900..0x00000000bfaac92b
```

图 115

```
51: 0x00000000bfaacfe8..0x00000000bfaad013
52: 0x00000000bfaad018..0x00000000bfaad044
53: 0x00000000bfaad048..0x00000000bfaad074
54: 0x00000000bfaad078..0x00000000bfaad0a4
55: 0x00000000bfaad0a8..0x00000000bfaad0d4
56: 0x00000000bfaad0d8..0x00000000bfaad104
57: 0x00000000bfaad108..0x00000000bfaad134
58: 0x00000000bfaad138..0x00000000bfaad164
59: 0x00000000bfaad168..0x00000000bfaad198
60: 0x00000000bfaad1a0..0x00000000bfaad1d0
61: 0x00000000bfaad1d8..0x00000000bfaad208
62: 0x00000000bfaad210..0x00000000bfaad240
63: 0x00000000bfaad248..0x00000000bfaad278
64: 0x00000000bfaad280..0x00000000bfaad2b0
65: 0x00000000bfaad2b8..0x00000000bfaad2e8
66: 0x00000000bfaad2f0..0x00000000bfaad320
67: 0x00000000bfaad328..0x00000000bfaad359
68: 0x00000000bfaad360..0x00000000bfffffff
root@RK3576-Tronlong:~#
```

图 116

因我们的存在，让嵌入式应用更简单

从 U-Boot 启动打印信息可以看出，编号 0 用于存放 kernel panel 信息；编号 1 和编号 2 用于存放内核镜像文件；编号 3 用于存放设备树文件。

```
root@RK3576-Tronlong:~# cat /sys/kernel/debug/memblock/reserved
0: 0x0000000040110000..0x00000000401effff
1: 0x0000000040410000..0x000000004230cfff
2: 0x000000004230f000..0x000000004230ffff
3: 0x0000000048300000..0x0000000048346fff
4: 0x00000000bbe00000..0x00000000bddfffff
5: 0x00000000be700000..0x00000000bf7fffff
6: 0x00000000bfa0f200..0x00000000bfa0f70f
7: 0x00000000bfa0f740..0x00000000bfa0fc1f
8: 0x00000000bfa0fc40..0x00000000bfa0fdc7
9: 0x00000000bfa0fe00..0x00000000bfa97fff
10: 0x00000000bfa9a000..0x00000000bfa9b11f
11: 0x00000000bfa9b140..0x00000000bfa9b19f
12: 0x00000000bfa9b1c0..0x00000000bfa9b1c7
```

图 117

```
vp0, plane_mask:0x4, primary-id:2, curser-id:-1
vp1, plane_mask:0x8, primary-id:3, curser-id:-1
vp2, plane_mask:0x0, primary-id:0, curser-id:-1
## reserved-memory:
  drm-logo@0: addr=bdf00000 size=b8000
  ramoops@40110000: addr=40110000 size=e0000
Adding bank: 0x40200000 - 0x48400000 (size: 0x08200000)
Adding bank: 0x49400000 - 0xc0000000 (size: 0x76c00000)
Total: 5692.268/6332.528 ms

Starting kernel ...
```

图 118

```
root@RK3576-Tronlong:~# cat /sys/kernel/debug/memblock/reserved
0: 0x0000000040110000..0x00000000401effff
1: 0x0000000040410000..0x000000004230cfff
2: 0x000000004230f000..0x000000004230ffff
3: 0x0000000048300000..0x0000000048346fff
4: 0x00000000bbe00000..0x00000000bddfffff
5: 0x00000000be700000..0x00000000bf7fffff
6: 0x00000000bfa0f200..0x00000000bfa0f70f
7: 0x00000000bfa0f740..0x00000000bfa0fc1f
8: 0x00000000bfa0fc40..0x00000000bfa0fdc7
9: 0x00000000bfa0fe00..0x00000000bfa97fff
10: 0x00000000bfa9a000..0x00000000bfa9b11f
11: 0x00000000bfa9b140..0x00000000bfa9b19f
12: 0x00000000bfa9b1c0..0x00000000bfa9b1c7
```

图 119

因我们的存在，让嵌入式应用更简单

```
## Loading fdt from FIT Image at b974fa80 ...
Using 'conf' configuration
Trying 'fdt' fdt subimage
Description: unavailable
Type: Flat Device Tree
Compression: uncompressed
Data Start: 0xb9750280
Data Size: 286679 Bytes = 280 KiB
Architecture: AArch64
Load Address: 0x48300000
Hash algo: sha256
Hash value: d481782a47dd1c556abaacbfd188d04787658368ab4ba4e5dc1a6554c08bb842
Verifying Hash Integrity ... sha256+ OK
Loading fdt from 0x48300000 to 0x48300000
Booting using the fdt blob at 0x48300000
Loading kernel Image from 0xb9796280 to 0x40400000 ... OK
kernel loaded at 0x40400000, end = 0x42263200
Using Device Tree in place at 0000000048300000, end 0000000048348fd6
scanning bus for devices...
ufshcd_link_startup: Device not present
link startup failed -6
ufshcd_pltfrm_init() failed -6
FDT: UFS was not detected, disabling UFS.
vp0, plane_mask:0x4, primary-id:2, curser-id:-1
vp1, plane_mask:0x8, primary-id:3, curser-id:-1
vp2, plane_mask:0x0, primary-id:0, curser-id:-1
## reserved-memory:
```

图 120

```
root@RK3576-Tronlong:~# cat /sys/kernel/debug/memblock/reserved
0: 0x0000000040110000..0x00000000401effff
1: 0x0000000040410000..0x000000004230cfff
2: 0x000000004230f000..0x000000004230ffff
3: 0x0000000048300000..0x0000000048346fff
4: 0x00000000b0000000..0x00000000b000ffff
5: 0x00000000b0700000..0x00000000b07fffff
6: 0x00000000b0f80f200..0x00000000b0fa0f707
7: 0x00000000b0fa0f740..0x00000000b0fa0fc1f
8: 0x00000000b0fa0fc40..0x00000000b0fa0fdc7
9: 0x00000000b0fa0fe00..0x00000000b0fa97fff
10: 0x00000000b0fa9a000..0x00000000b0fa9b11f
11: 0x00000000b0fa9b140..0x00000000b0fa9b19f
12: 0x00000000b0fa9b1c0..0x00000000b0fa9b1c7
```

图 121

```
## Loading fdt from FIT Image at b974fa80 ...
Using 'conf' configuration
Trying 'fdt' fdt subimage
Description: unavailable
Type: Flat Device Tree
Compression: uncompressed
Data Start: 0xb9750280
Data Size: 286679 Bytes = 280 KiB
Architecture: AArch64
Load Address: 0x48300000
Hash algo: sha256
Hash value: d481782a47dd1c556abaacbfd188d04787658368ab4ba4e5dc1a6554c08bb842
Verifying Hash Integrity ... sha256+ OK
Loading fdt from 0x48300000 to 0x48300000
Booting using the fdt blob at 0x48300000
Loading kernel Image from 0xb9796280 to 0x40400000 ... OK
kernel loaded at 0x40400000, end = 0x42263200
Using Device Tree in place at 0000000048300000, end 0000000048348fd6
scanning bus for devices...
ufshcd_link_startup: Device not present
link startup failed -6
ufshcd_pltfrm_init() failed -6
FDT: UFS was not detected, disabling UFS.
vp0, plane_mask:0x4, primary-id:2, curser-id:-1
vp1, plane_mask:0x8, primary-id:3, curser-id:-1
vp2, plane_mask:0x0, primary-id:0, curser-id:-1
```

图 122

从 Kernel 启动打印信息可以看出，编号 4 为 CMA（连续内存区管理）空间，其余为内核管理空间。

因我们的存在，让嵌入式应用更简单

```
root@RK3576-Tronlong:~# cat /sys/kernel/debug/memblock/reserved
0: 0x0000000040110000..0x00000000401effff
1: 0x0000000040410000..0x000000004230cfff
2: 0x000000004230f000..0x000000004230ffff
3: 0x0000000048300000..0x0000000048346fff
4: 0x00000000bbe00000..0x00000000bddfffff
5: 0x00000000be000000..0x00000000bfffff
6: 0x00000000bf80f200..0x00000000bfa0f707
7: 0x00000000bfa0f740..0x00000000bfa0fc1f
8: 0x00000000bfa0fc40..0x00000000bfa0fdc7
9: 0x00000000bfa0fe00..0x00000000bfa97fff
10: 0x00000000bfa9a000..0x00000000bfa9b11f
11: 0x00000000bfa9b140..0x00000000bfa9b19f
12: 0x00000000bfa9b1c0..0x00000000bfa9b1c7
```

图 123

```
Starting kernel ...
[ 6.341917] Booting Linux on physical CPU 0x000000000 [0x410fd034]
[ 6.341935] Linux version 6.1.115-rt16-03117-ge2d5ae01404e-dirty (tronlong@tronlong-virtu
al-machine) (aarch64-none-linux-gnu-gcc (GNU Toolchain for the A-profile Architecture 10.3-2
021.07 (arm-10.29)) 10.3.1 20210621, GNU ld (GNU Toolchain for the A-profile Architecture 10
.3-2021.07 (arm-10.29)) 2.36.1.20210621) #132 SMP PREEMPT_RT Thu Mar 6 10:46:10 CST 2025
[ 6.344966] random: crng init done
[ 6.349841] Machine model: TL3576-EVM
[ 6.355037] earlycon: uart8250 at MMIO32 0x000000002ad40000 (options '')
[ 6.411981] printk: bootconsole [uart8250] enabled
[ 6.414466] efi: UEFI not found.
[ 6.416919] OF: fdt: Reserved memory: failed to reserve memory for node 'drm-cubic-lut@0'
base 0x0000000000000000, size 0 MiB
Zone ranges:
[ 6.468812] DMA [mem 0x0000000040200000-0x00000000bffffff]
[ 6.468820] DMA32 empty
[ 6.468829] Normal empty
[ 6.468832] Movable zone start for each node
[ 6.468834] Early memory node ranges
[ 6.468837] node 0: [mem 0x0000000040200000-0x00000000483fffff]
[ 6.468842] node 0: [mem 0x0000000049400000-0x00000000bffffff]
[ 6.468847] Initmem setup node 0 [mem 0x0000000040200000-0x00000000bffffff]
[ 6.469424] On node 0, zone DMA: 512 pages in unavailable ranges
[ 6.477410] On node 0, zone DMA: 4096 pages in unavailable ranges
[ 6.477425] cma: Reserved 16 MiB at 0x00000000be800000
[ 6.477614] psci: probing for conduit method from DT.
[ 6.477621] psci: PSCIv1.1 detected in firmware.
[ 6.477624] psci: Using standard PSCI v0.2 function IDs
[ 6.477629] psci: Trusted OS migration not required
[ 6.477639] psci: SMC Calling Convention v1.2
[ 6.477855] percpu: Embedded 17 pages/cpu s29824 r8192 d31616 u69632
```

图 124

10. 程序开机自启动说明

在 Linux 系统下，可通过 init 进程方式和 systemd 服务方式实现程序开机自启动。本章以 led_flash 案例为例，演示通过 init 进程方式，实现程序开机自启动。

请将产品资料“4-软件资料\Demo\base-demos\led_flash\bin\”目录下的可执行文件拷贝至评估板文件系统“/etc/init.d/”目录下。

执行如下命令，改变 led_flash 可执行文件权限。

Target# `chmod a+x /etc/init.d/led_flash`

```
root@RK3576-Tronlong:~# ls /etc/init.d/led_flash
/etc/init.d/led_flash
root@RK3576-Tronlong:~# chmod a+x /etc/init.d/led_flash
root@RK3576-Tronlong:~#
```

图 125

因我们的存在，让嵌入式应用更简单

在评估板文件系统可执行文件目录下，执行如下命令修改"/etc/init.d/"目录下的 rcS 配置文件。

Target# vi /etc/init.d/rcS

```
root@RK3576-Tronlong:~# vi /etc/init.d/rcs
root@RK3576-Tronlong:~#
```

图 126

在配置文件文末添加如下内容：

```
/etc/init.d/led_flash -n 2 & //设置自启动的程序路径
```

```
#!/bin/sh

# Start all init scripts in /etc/init.d
# executing them in numerical order.
#
for i in /etc/init.d/s??* ;do
    # Ignore dangling symlinks (if any).
    [ ! -f "$i" ] && continue

    case "$i" in
        *.sh)
            # Source shell script for speed.
            (
                trap - INT QUIT TSTP
                set start
                . $i
            )
            ;;
        *)
            # No sh extension, so fork subprocess.
            $i start
            ;;
    esac
done
/etc/init.d/led_flash -n 2 &
```

图 127

修改完成后，保存并退出。请将评估板断电重启，Linux 系统将会自动运行程序，调试串口终端将打印如下类似信息，同时可观察到评估底板 LED 进行闪烁。

```
Starting input-event-daemon: done

system leds :
/sys/class/leds/disk
/sys/class/leds/heartbeat
/sys/class/leds/mmc0::
/sys/class/leds/user-led0
/sys/class/leds/user-led1

Flashing leds :
/sys/class/leds/user-led0
/sys/class/leds/user-led1

Welcome to RK3576 Buildroot
RK3576-Tronlong login: root (automatic login)
```

图 128

如需取消程序开机自启动，请修改 rcS 配置文件，使用"#"字符将相应命令进行注释，如下图所示。

```
#!/bin/sh

# Start all init scripts in /etc/init.d
# executing them in numerical order.
#
for i in /etc/init.d/S??* ;do
    # Ignore dangling symlinks (if any).
    [ ! -f "$i" ] && continue
    case "$i" in
        *.sh)
            # Source shell script for speed.
            (
                trap - INT QUIT TSTP
                set start
                . $i
            )
            ;;
        *)
            # No sh extension, so fork subprocess.
            $i start
            ;;
    esac
done

# /etc/init.d/led_flash -n 2 &
```

图 129

11. CPU 主频调节说明

根据官方手册《Rockchip RK3576J Datasheet V1.0-20241028》文档说明，RK3576J 处理器长时间在 overdrive mode 运行环境下，特别是在高温条件下，处理器使用寿命可能会缩短。

Note (1):For all power supply inputs, the min and max voltage requirement includes dynamic voltage events such as AC ripple, voltage transients, voltage dips, and so forth.
Note (2):When LOGIC_DVDD works at under drive voltage, the frequency of modules in VD_LOGIC should lower down.
Note (3): **Normal mode:** For the industrial operating environment, the lifetime is reasonably determined. **Overdrive mode:** The running frequency is higher, and the voltage will increase accordingly. If the chip is in overdrive mode for a long time, especially in high temperature condition, the lifetime may be shortened. If you need high frequency, we recommend that you dynamically adjust frequency&voltage and keep heat dissipation.

图 130

为保障处理器使用寿命，满足更多工业应用场景要求，我司已将 RK3576J/RK3576 处理器 Cortex-A72 核心最高主频默认配置为 1.6GHz，Cortex-A53 核心最高主频默认配置为 1.4GHz。如需调整为更高主频，请参考下文“通过设备树配置方法”章节进行操作。

11.1 CPU 主频说明

评估板支持 normal mode 和 overdrive mode，系统默认配置为 normal mode，如下为 normal mode 可配置的主频频率说明。

表 11

处理器	RK3576		RK3576J	
核心	Cortex-A53	Cortex-A72	Cortex-A53	Cortex-A72
频率	0.408GHz	0.408GHz	0.408GHz	0.408GHz
	0.600GHz	0.600GHz	0.600GHz	0.600GHz
	0.816GHz	0.816GHz	0.816GHz	0.816GHz
	1.008GHz	1.008GHz	1.008GHz	1.008GHz
	1.200GHz	1.200GHz	1.200GHz	1.200GHz
	1.416GHz	1.416GHz	1.416GHz	1.416GHz
	-	1.608GHz	-	1.608GHz

系统支持 7 种 CPU 主频模式，默认配置为 performance 主频模式，主频频率及相关说明如下表所示。可通过内核配置或命令行配置方式修改 CPU 主频模式，可通过内核配置、命令行配置或设备树配置方式修改 CPU 频率。

因我们的存在，让嵌入式应用更简单

表 12

主频模式	CPU 默认初始时钟频率		模式说明
	RK3576、RK3576J		
	Cortex-A53	Cortex-A72	
performance	1.416GHz	1.608GHz	性能优先，始终将频率设置为最高值
userspace	1.200GHz	1.200GHz	用户自定义设置电压和频率，系统不自动调整
powersave	0.408GHz	0.408GHz	功耗优先，始终将频率设置为最低值
interactive	动态调节	动态调节	快升慢降，当有高负载时，迅速提高至高频率，当低负载时，逐渐降低频率
ondemand	动态调节	动态调节	按需调节，调频幅度大，当工作时，会迅速提高至最高频率，当空闲时，迅速降低频率
conservative	动态调节	动态调节	慢升快降，当有高负载时，逐渐提高频率，当低负载时，迅速降低频率
schedutil	动态调节	动态调节	Android 中引入新的调频策略，通过 PELT 来统计各个 Task 的负载，进行负载均衡，特点是调节延时低

11.2 查看 CPU 主频

Linux 系统默认将 RK3576J/RK3576 处理器核心分为 2 组，4 个 Cortex-A53 核心为 policy0 组，4 个 Cortex-A72 核心为 policy4 组。

11.2.1 通过 policy 方式查看

进入评估板文件系统，执行如下命令，可查看当前 CPU 主频模式。本次操作以查看 policy0 组为例进行演示。

备注：如需查看其他组的 CPU 主频模式，可将 policy0 修改为 policy4。

Target# cat /sys/devices/system/cpu/cpufreq/policy0/scaling_governor

```
root@RK3576-Tronlong:~# cat /sys/devices/system/cpu/cpufreq/policy0/scaling_governor
performance
root@RK3576-Tronlong:~#
```

图 131

执行如下命令，查看当前 CPU 主频。本次操作以查看 policy0 组为例进行演示。

Target# `cat /sys/devices/system/cpu/cpufreq/policy0/cpuinfo_cur_freq`

```
root@RK3576-Tronlong:~# cat /sys/devices/system/cpu/cpufreq/policy0/cpuinfo_cur_freq
1416000
root@RK3576-Tronlong:~#
```

图 132

执行如下命令，可查看 CPU 可配置的主频。本次操作以 policy0 组为例进行演示。

Target# `cat /sys/devices/system/cpu/cpufreq/policy0/scaling_available_frequencies`

```
root@RK3576-Tronlong:~# cat /sys/devices/system/cpu/cpufreq/policy0/scaling_available_frequencies
408000 600000 816000 1008000 1200000 1416000
root@RK3576-Tronlong:~#
```

图 133

执行如下命令，可查看当前内核支持的主频模式。本次操作以 policy0 组为例进行演示。

Target# `cat /sys/devices/system/cpu/cpufreq/policy0/scaling_available_governors`

```
root@RK3576-Tronlong:~# cat /sys/devices/system/cpu/cpufreq/policy0/scaling_available_governors
interactive conservative ondemand userspace powersave performance schedutil
root@RK3576-Tronlong:~#
```

图 134

11.2.2 通过单个 CPU 核心方式查看

执行如下命令，可查看当前 CPU 主频模式。本次操作以查看单个 Cortex-A72(CPU7)核心为例进行演示。

备注：如需查看其他核心 CPU 主频，根据实际需求修改为 `cpu $x$` 。

Target# `cat /sys/devices/system/cpu/cpu7/cpufreq/scaling_governor`

```
root@RK3576-Tronlong:~# cat /sys/devices/system/cpu/cpu7/cpufreq/scaling_governor
performance
root@RK3576-Tronlong:~#
```

图 135

执行如下命令，可查看当前 CPU 主频。本次操作以单个 Cortex-A72(CPU7)核心为例进行演示。

Target# `cat /sys/devices/system/cpu/cpu7/cpufreq/cpuinfo_cur_freq`

```
root@RK3576-Tronlong:~# cat /sys/devices/system/cpu/cpu7/cpufreq/cpuinfo_cur_freq
1608000
root@RK3576-Tronlong:~#
```

图 136

执行如下命令，可查看 CPU 可配置的主频。本次操作以单个 Cortex-A53(CPU0)核心为例进行演示。

Target# `cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_available_frequencies`

```
root@RK3576-Tronlong:~# cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_available_frequencies
408000 600000 816000 1008000 1200000 1416000
root@RK3576-Tronlong:~#
```

图 137

执行如下命令，可查看当前内核支持的 CPU 主频模式。本次操作以单个 Cortex-A53(CPU2)核心为例进行演示。

Target# `cat /sys/devices/system/cpu/cpu2/cpufreq/scaling_available_governors`

```
root@RK3576-Tronlong:~# cat /sys/devices/system/cpu/cpu2/cpufreq/scaling_available_governors
interactive conservative ondemand userspace powersave performance schedutil
root@RK3576-Tronlong:~#
```

图 138

11.3 CPU 主频配置

我司默认对 RK3576/RK3576J 处理器主频频率做了限制，即 Cortex-A72 核心最高主频默认配置为 1.6GHz，Cortex-A53 核心最高主频默认配置为 1.4GHz。如需调整为更高主频，

因我们的存在，让嵌入式应用更简单

请参考“通过设备树配置方法”章节。

备注：通过修改内核或命令行方式仅支持配置 CPU 主频模式，无法调整为更高主频。

11.3.1 通过内核配置方法

进入 LinuxSDK 源码目录下，执行如下命令打开内核图形配置界面。

Host# ./build.sh kernel-config



图 139

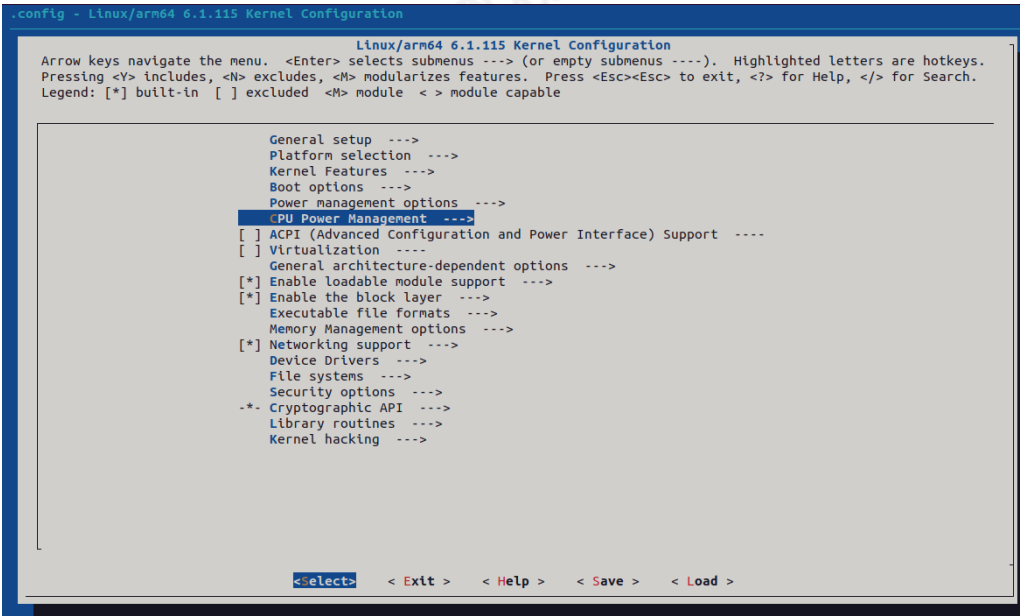


图 140

在图形配置界面中，进入"CPU Power Management -> CPU Frequency scaling"路径，

因我们的存在，让嵌入式应用更简单

选中"Default CPUFreq governor"选项，该选项的默认配置为 performance 主频模式。

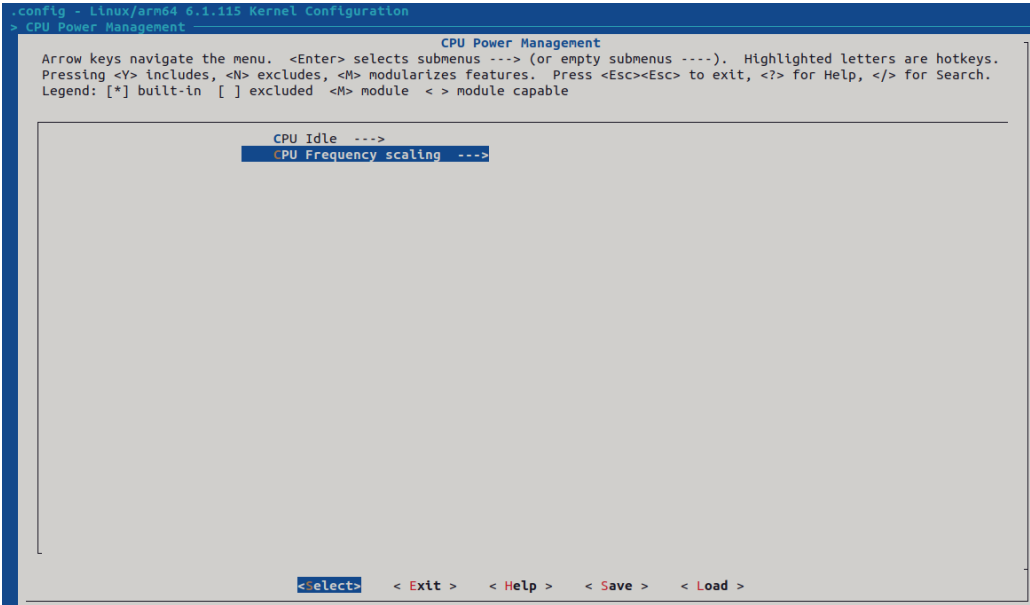


图 141

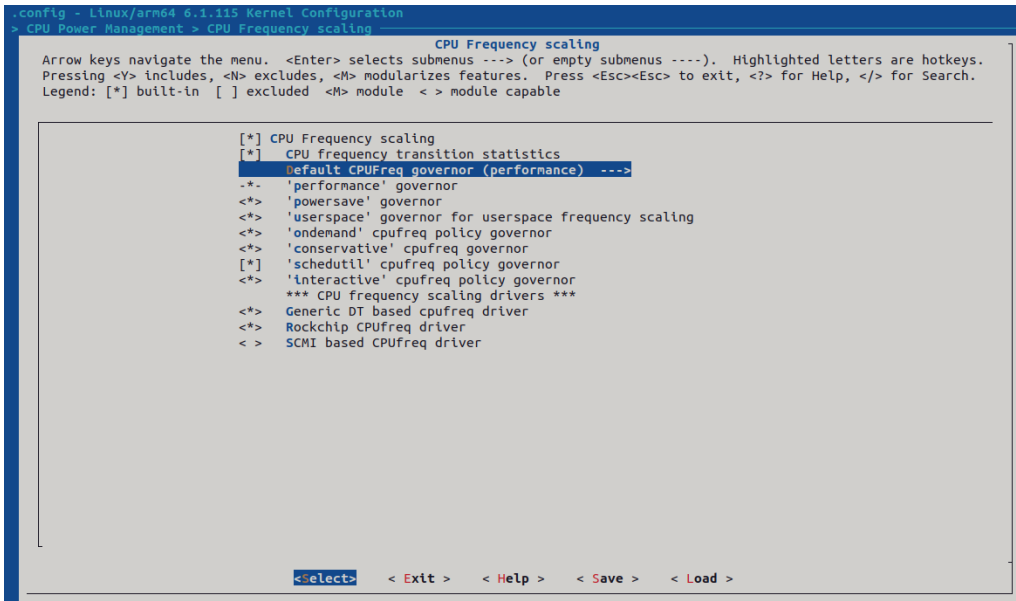


图 142

可通过修改"Default CPUFreq governor"选项，配置 CPU 主频模式，本次操作修改为 powersave 模式。

因我们的存在，让嵌入式应用更简单

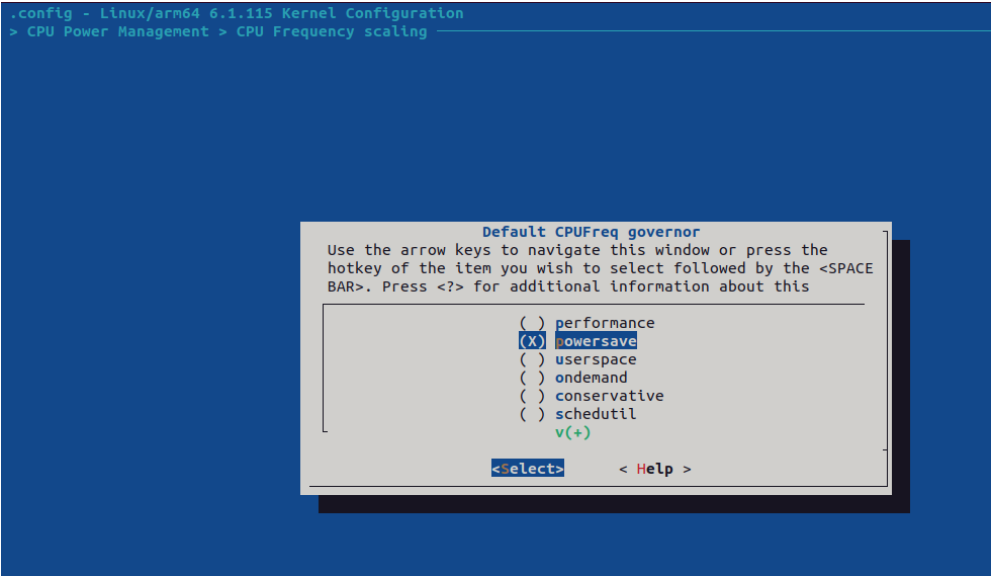


图 143

配置完毕后，选中<Save>，按 **Enter** 键保存配置选项。然后选中<Exit>，按 **Enter** 键退出。

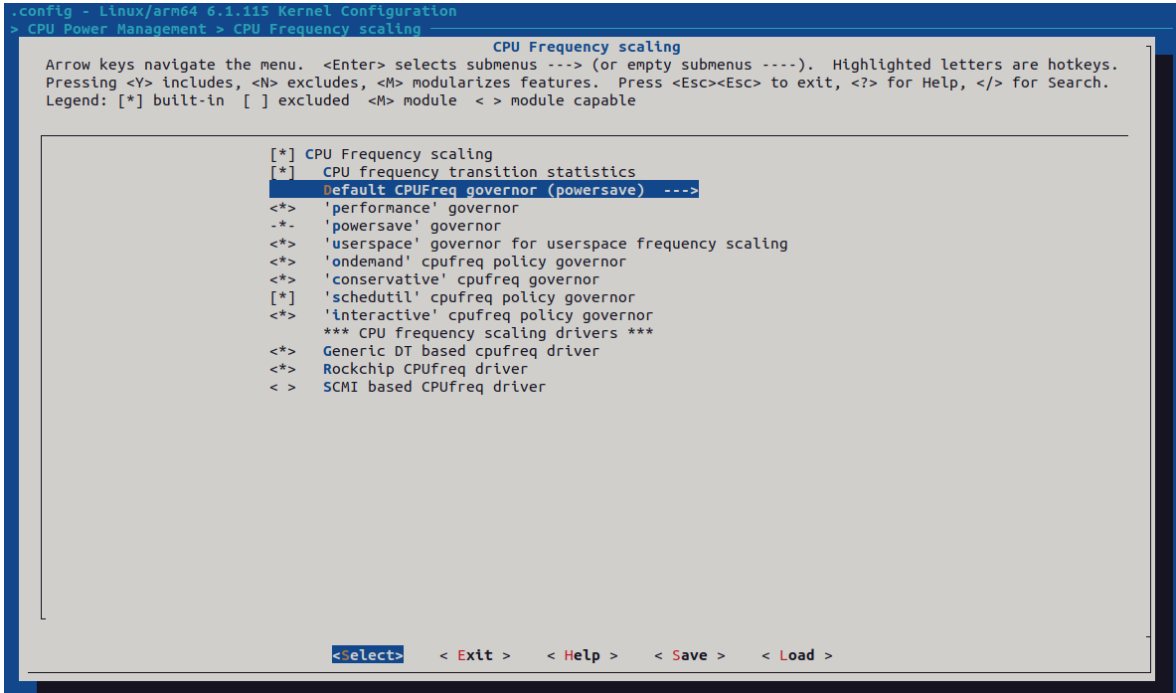


图 144

主频模式配置完成后，请参考“内核编译”章节重新编译生成内核镜像，再参考“Linux 系统文件替换说明”替换内核镜像，断电重启即可生效。进入评估板文件系统，执行如下

因我们的存在，让嵌入式应用更简单

命令查看当前 CPU 主频模式如下。

Target# cat /sys/devices/system/cpu/cpufreq/**policy0**/scaling_governor

Target# cat /sys/devices/system/cpu/cpufreq/**policy4**/scaling_governor

```
root@RK3576-Tronlong:~# cat /sys/devices/system/cpu/cpufreq/policy0/scaling_governor
powersave
root@RK3576-Tronlong:~# cat /sys/devices/system/cpu/cpufreq/policy4/scaling_governor
powersave
root@RK3576-Tronlong:~#
```

图 145

11.3.2 通过命令行配置方法

本小节以 normal 模式为例演示配置方法。

进入评估板文件系统，执行如下命令，修改 CPU 主频和主频模式。本次操作以 policy0 组为例进行演示，配置 CPU 为 userspace 主频模式，主频为 1200000kHz。

备注：命令行配置 CPU 主频的方式临时生效，系统重启后需重新配置。

Target# echo userspace > /sys/devices/system/cpu/cpufreq/**policy0**/scaling_governor

Target# echo 1200000 > /sys/devices/system/cpu/cpufreq/**policy0**/scaling_setspeed

Target# cat /sys/devices/system/cpu/cpufreq/**policy0**/scaling_cur_freq

```
root@RK3576-Tronlong:~# echo userspace > /sys/devices/system/cpu/cpufreq/policy0/scaling_governor
root@RK3576-Tronlong:~# echo 1200000 > /sys/devices/system/cpu/cpufreq/policy0/scaling_setspeed
root@RK3576-Tronlong:~# cat /sys/devices/system/cpu/cpufreq/policy0/scaling_cur_freq
1200000
root@RK3576-Tronlong:~#
```

图 146

执行如下命令，修改 CPU 主频和主频模式。本次操作以单个 Cortex-A72(CPU4)核心为例进行演示，配置 CPU 为 userspace 主频模式，主频为 1200000kHz。

Target# echo userspace > /sys/devices/system/cpu/**cpu4**/cpufreq/scaling_governor

Target# echo 1200000 > /sys/devices/system/cpu/**cpu4**/cpufreq/scaling_setspeed

Target# cat /sys/devices/system/cpu/**cpu4**/cpufreq/scaling_cur_freq

```
root@RK3576-Tronlong:~# echo userspace > /sys/devices/system/cpu/cpu4/cpufreq/scaling_governor
root@RK3576-Tronlong:~# echo 1200000 > /sys/devices/system/cpu/cpu4/cpufreq/scaling_setspeed
root@RK3576-Tronlong:~# cat /sys/devices/system/cpu/cpu4/cpufreq/scaling_cur_freq
1200000
root@RK3576-Tronlong:~#
```

图 147

11.3.3 通过设备树配置方法

请修改 Linux 内核源码"arch/arm64/boot/dts/rockchip/"目录下的内核设备树文件 tl3576-evm.dts。其中 CPU 为 RK3576J 请修改 opp-j-m-xxxxxxx，CPU 为 RK3576（商业级）请修改 opp-xxxxxxx。本次以 RK3576（商业级）的 policy4 组作为例进行演示，将 CPU 主频调整为支持 1.800GHz、2.016GHz 和 2.208GHz。

进入 Kernel 内核源码目录下，执行如下命令修改 tl3576-evm.dts 设备树文件。

```
Host# cd /home/tronlong/RK3576/rk3576_linux6.1_release/kernel
```

```
Host# vim arch/arm64/boot/dts/rockchip/tl3576-evm.dts
```

```
tronlong@tronlong-virtual-machine:~$ cd /home/tronlong/RK3576/rk3576_linux6.1_release/kernel
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release/kernel$ vim arch/arm64/boot/dts/rockchip/tl3576-evm.dts
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release/kernel$
```

图 148

修改内容如下，添加"//"注释符，将 CPU 主频调整为支持 1.800GHz、2.016GHz 和 2.208GHz。

```
// /delete-node/ opp-1800000000; //添加"//"注释符
// /delete-node/ opp-2016000000; //添加"//"注释符
// /delete-node/ opp-2208000000; //添加"//"注释符
```

```

170 &cluster0_opp_table{
171     /* RK3576 A53 core limit at 1.4GHz */
172     /delete-node/ opp-1608000000;
173     /delete-node/ opp-1800000000;
174     /delete-node/ opp-2016000000;
175
176     /* RK3576J A53 core limit at 1.4GHz */
177     /delete-node/ opp-j-m-1608000000;
178     /delete-node/ opp-j-m-1800000000;
179     /delete-node/ opp-j-m-1920000000;
180 };
181
182 &cluster1_opp_table{
183     /* RK3576 A72 core limit at 1.6GHz */
184     // /delete-node/ opp-1800000000;
185     // /delete-node/ opp-2016000000;
186     // /delete-node/ opp-2208000000;
187
188     /* RK3576J A72 core limit at 1.6GHz */
189     /delete-node/ opp-j-m-1800000000;
190     /delete-node/ opp-j-m-2016000000;
191     /delete-node/ opp-j-m-2112000000;
192 };

```

图 149

修改完成后，请参考“内核编译”章节重新编译生成内核镜像，再参考“Linux 系统文件替换说明”替换内核镜像，断电重启即可生效。系统启动后，执行如下命令查看当前 CPU 主频和主频模式，本次操作以 policy4 组为例进行演示。

```

Target# cat /sys/devices/system/cpu/cpufreq/policy4/scaling_available_frequencies
Target# cat /sys/devices/system/cpu/cpufreq/policy4/scaling_governor
Target# cat /sys/devices/system/cpu/cpufreq/policy4/scaling_cur_freq

```

```

root@RK3576-Tronlong:~# cat /sys/devices/system/cpu/cpufreq/policy4/scaling_available_frequencies
408000 600000 816000 1008000 1200000 1416000 1608000 1800000 2016000 2208000
root@RK3576-Tronlong:~# cat /sys/devices/system/cpu/cpufreq/policy4/scaling_governor
performance
root@RK3576-Tronlong:~# cat /sys/devices/system/cpu/cpufreq/policy4/scaling_cur_freq
2208000
root@RK3576-Tronlong:~#

```

图 150

12. TFTP 使用说明

TFTP(Trivial File Transfer Protocol)是用来下载远程文件的简单网络协议，它是基于 UDP 协议的应用。嵌入式 Linux 的 TFTP 包括服务器和客户端，常用来完成评估板（客户端）和 PC 机（服务器）之间的文件传输功能，可避免频繁的 U 盘拷贝的过程。

评估板支持 TFTP 服务器和客户端程序（如下图所示）。本章节主要演示评估板作为客户端使用，同时在 PC 机 Linux 系统中搭建 TFTP 服务器，最终实现 PC 机与评估板之间

因我们的存在，让嵌入式应用更简单

的 TFTP 文件传输方法。

Target# find /usr/ -name "tftp"

```
root@RK3576-Tronlong:~# find /usr/ -name "tftp"
/usr/bin/tftp
root@RK3576-Tronlong:~#
```

图 151

12.1 TFTP 服务器搭建

进入 PC 机 Ubuntu 系统，执行如下命令从网络安装 TFTP 服务器相关软件 tftp-hpa（客户端）、tftpd-hpa（服务器）和 xinetd（配置文件）。

Host# sudo apt-get install tftp-hpa tftpd-hpa xinetd -y

```
tronlong@tronlong-virtual-machine:~$ sudo apt-get install tftp-hpa tftpd-hpa xinetd -y
[sudo] password for tronlong:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Suggested packages:
  pxelinux
The following NEW packages will be installed:
  tftp-hpa tftpd-hpa xinetd
0 upgraded, 3 newly installed, 0 to remove and 41 not upgraded.
Need to get 168 kB of archives.
After this operation, 494 kB of additional disk space will be used.
Get:1 http://cn.archive.ubuntu.com/ubuntu jammy/main amd64 tftp-hpa amd64 5.2+20150808-1.2build2 [19.9 kB]
Get:2 http://cn.archive.ubuntu.com/ubuntu jammy/main amd64 tftpd-hpa amd64 5.2+20150808-1.2build2 [40.7 kB]
Get:3 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy/universe amd64 xinetd amd64 1:2.3.15.3-1 [108 kB]
Fetched 168 kB in 2s (75.2 kB/s)
Preconfiguring packages ...
Selecting previously unselected package tftp-hpa.
(Reading database ... 202566 files and directories currently installed.)
Preparing to unpack .../tftp-hpa_5.2+20150808-1.2build2_amd64.deb ...
Unpacking tftp-hpa (5.2+20150808-1.2build2) ...
Selecting previously unselected package tftpd-hpa.
Preparing to unpack .../tftpd-hpa_5.2+20150808-1.2build2_amd64.deb ...
Unpacking tftpd-hpa (5.2+20150808-1.2build2) ...
Selecting previously unselected package xinetd.
Preparing to unpack .../xinetd_1%3a2.3.15.3-1_amd64.deb ...
Unpacking xinetd (1:2.3.15.3-1) ...
Setting up xinetd (1:2.3.15.3-1) ...
Setting up tftpd-hpa (5.2+20150808-1.2build2) ...
Setting up tftp-hpa (5.2+20150808-1.2build2) ...
Processing triggers for man-db (2.10.2-1) ...
tronlong@tronlong-virtual-machine:~$
```

图 152

执行如下命令，打开"/etc/xinetd.conf"配置文件，查看文件内容是否与下面内容一致。如果不存在 xinetd.conf 配置文件，则需先新建此文件。如配置文件内容不一致，则将内容修改一致，并保存。

Host# sudo vi /etc/xinetd.conf

因我们的存在，让嵌入式应用更简单

```
tronlong@tronlong-virtual-machine:~$ sudo vi /etc/xinetd.conf
tronlong@tronlong-virtual-machine:~$
```

图 153

```
1 # Simple configuration file for xinetd
2 #
3 # Some defaults, and include /etc/xinetd.d/
4
5 defaults
6 {
7
8 # Please note that you need a log_type line to be able to use log_on_success
9 # and log_on_failure. The default is the following :
10 # log_type = SYSLOG daemon info
11
12 }
13
14 includedir /etc/xinetd.d
```

图 154

xinetd.conf 配置文件内容如下：

```
# Simple configuration file for xinetd
#
# Some defaults, and include /etc/xinetd.d/
#
defaults
{
# Please note that you need a log_type line to be able to use log_on_success
# and log_on_failure. The default is the following :
# log_type = SYSLOG daemon info
}
includedir /etc/xinetd.d
```

因我们的存在，让嵌入式应用更简单

执行如下命令，新建 TFTP 工作目录，并修改权限为可读可写可执行，允许其他服务器或者客户端对 TFTP 工作目录下载、上传文件。

```
Host# mkdir -p /home/tronlong/tftpboot
```

```
Host# sudo chmod 777 /home/tronlong/tftpboot
```

```
tronlong@tronlong-virtual-machine:~$ mkdir -p /home/tronlong/tftpboot
tronlong@tronlong-virtual-machine:~$ sudo chmod 777 /home/tronlong/tftpboot
tronlong@tronlong-virtual-machine:~$
```

图 155

打开 TFTP 服务器配置文件"/etc/default/tftpd-hpa"，配置 TFTP 服务器工作目录。

```
Host# sudo vi /etc/default/tftpd-hpa
```

```
tronlong@tronlong-virtual-machine:~$ sudo vi /etc/default/tftpd-hpa
tronlong@tronlong-virtual-machine:~$
```

图 156

配置文件内容如下：

```
# /etc/default/tftpd-hpa
```

```
TFTP_USERNAME="tftp"
```

```
TFTP_DIRECTORY="/home/tronlong/tftpboot" //TFTP 服务器工作目录
```

```
TFTP_ADDRESS=":69"
```

```
TFTP_OPTIONS="--secure -c"
```

```
# /etc/default/tftpd-hpa
TFTP_USERNAME="tftp"
TFTP_DIRECTORY="/home/tronlong/tftpboot"
TFTP_ADDRESS=":69"
TFTP_OPTIONS="--secure -c"
```

图 157

因我们的存在，让嵌入式应用更简单

执行如下命令，重启 tftpd-hpa 服务器和重启 xinetd 服务。

Host# sudo service tftpd-hpa restart

Host# sudo service xinetd reload

Host# sudo service xinetd restart

```
tronlong@tronlong-virtual-machine:~$ sudo service tftpd-hpa restart
tronlong@tronlong-virtual-machine:~$ sudo service xinetd reload
tronlong@tronlong-virtual-machine:~$ sudo service xinetd restart
tronlong@tronlong-virtual-machine:~$
```

图 158

12.2 TFTP 文件传输测试

执行如下命令，进入 TFTP 服务器工作目录，并新建一个 test 文件。在文件内输入测试内容并保存，然后修改 test 文件权限为可读可写。

Host# cd /home/tronlong/tftpboot/

Host# echo www.tronlong.com > test

Host# chmod 777 test

```
tronlong@tronlong-virtual-machine:~$ cd /home/tronlong/tftpboot/
tronlong@tronlong-virtual-machine:~/tftpboot$ echo www.tronlong.com > test
tronlong@tronlong-virtual-machine:~/tftpboot$ chmod 777 test
tronlong@tronlong-virtual-machine:~/tftpboot$ cat test
www.tronlong.com
tronlong@tronlong-virtual-machine:~/tftpboot$
```

图 159

执行如下命令可进入 TFTP 命令行模式。执行 help 命令可查看 TFTP 命令使用详情，按 q 可退出命令行模式。

Host# sudo tftp localhost

```
tronlong@tronlong-virtual-machine:~/tftpboot$ sudo tftp localhost
tftp> help
tftp-hpa 5.2
Commands may be abbreviated.  Commands are:

connect      connect to remote tftp
mode         set file transfer mode
put          send file
get          receive file
quit         exit tftp
verbose      toggle verbose mode
trace        toggle packet tracing
literal      toggle literal mode, ignore ':' in file name
status       show current status
binary       set mode to octet
ascii        set mode to netascii
rexmt        set per-packet transmission timeout
timeout      set total retransmission timeout
?            print help information
help         print help information
tftp> q
tronlong@tronlong-virtual-machine:~/tftpboot$
```

图 160

将评估板 ETH1 RGMII 网口和 PC 机通过路由器或者交换机互联，亦可通过网线直联。使用 `ifconfig` 命令分别查看评估板与 PC 机当前的 IP 地址是否在同一网段。具体为 IP 地址的前三个字段相同，最后一个字段不相同，以及子网掩码相同。

在下图中可见，PC 机 IP 地址为：192.168.13.46，评估板 IP 地址为：192.168.13.51，子网掩码都是 255.255.255.0，属于同一网段内。

```
tronlong@tronlong-virtual-machine:~/tftpboot$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.13.46 netmask 255.255.255.0 broadcast 192.168.13.255
    inet6 fe80::4cd1:8cce:b795:9ac prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:d0:6f:c2 txqueuelen 1000 (Ethernet)
    RX packets 65901 bytes 51951805 (51.9 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 45564 bytes 45798045 (45.7 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 168 bytes 15753 (15.7 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 168 bytes 15753 (15.7 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

tronlong@tronlong-virtual-machine:~/tftpboot$
```

图 161 PC 机 IP 地址

因我们的存在，让嵌入式应用更简单

```
root@RK3576-Tronlong:~# ifconfig
eth0      Link encap:Ethernet  HWaddr 52:1A:24:B0:52:51
          inet addr:192.168.13.51  Bcast:192.168.13.255  Mask:255.255.255.0
          inet6 addr: fe80::9084:97fb:57ba:e4a0/64  Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:485 errors:0 dropped:0 overruns:0 frame:0
          TX packets:71 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:45733 (44.6 KiB)  TX bytes:6270 (6.1 KiB)
          Interrupt:114

eth1      Link encap:Ethernet  HWaddr 56:1A:24:B0:52:51
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:116

eth2      Link encap:Ethernet  HWaddr 00:E0:99:E9:8A:62
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:3 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:270 (270.0 B)

eth3      Link encap:Ethernet  HWaddr 00:E0:99:E9:B7:43
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:12769 errors:0 dropped:0 overruns:0 frame:0
          TX packets:313 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:1460282 (1.3 MiB)  TX bytes:24094 (23.5 KiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128  Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:42 errors:0 dropped:0 overruns:0 frame:0
          TX packets:42 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:3084 (3.0 KiB)  TX bytes:3084 (3.0 KiB)

root@RK3576-Tronlong:~#
```

图 162 评估板 IP 地址

在 Ubuntu 执行 ping 命令测试 PC 机和评估板之间的网络连通性，按"Ctrl + C"命令退出。命令中 192.168.13.51 为评估板 IP 地址。

Host# ping 192.168.13.51

```
tronlong@tronlong-virtual-machine:~/tftpboot$ ping 192.168.13.51
PING 192.168.13.51 (192.168.13.51) 56(84) bytes of data.
64 bytes from 192.168.13.51: icmp_seq=1 ttl=64 time=1.18 ms
64 bytes from 192.168.13.51: icmp_seq=2 ttl=64 time=0.608 ms
64 bytes from 192.168.13.51: icmp_seq=3 ttl=64 time=0.627 ms
64 bytes from 192.168.13.51: icmp_seq=4 ttl=64 time=0.614 ms
^C
--- 192.168.13.51 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3091ms
rtt min/avg/max/mdev = 0.608/0.756/1.176/0.242 ms
tronlong@tronlong-virtual-machine:~/tftpboot$
```

图 163

在评估板执行如下命令从 TFTP 服务器工作目录中下载 test 测试文件，并查看内容。

Target# tftp -g -r test 192.168.13.46

因我们的存在，让嵌入式应用更简单

```
root@RK3576-Tronlong:~# tftp -g -r test 192.168.13.46
root@RK3576-Tronlong:~# ls
test
root@RK3576-Tronlong:~# cat test
www.tronlong.com
root@RK3576-Tronlong:~#
```

图 164

TFTP 命令参数解析如下：

表 13

参数	解析
192.168.13.46	代表 TFTP 服务器 IP 地址。
-g	get，代表从 TFTP 服务器工作目录中下载文件。
-r	remote，代表远程服务器。
test	代表 TFTP 服务器工作目录中的文件名称。

13. NFS 使用说明

NFS(Network File System)即网络文件系统，是基于 UDP/IP 协议的应用，它的主要功能是通过网络让不同的主机系统之间可共享文件或目录。NFS 客户端可通过挂载的方式将远程 NFS 服务器共享的数据目录挂载到 NFS 客户端本地系统中。在客户端来看，远程 NFS 服务器的目录就好像是自己的一个磁盘分区一样，在使用上相当便利。

评估板已支持 NFS 相关工具（如下图所示），本章节主要演示在 PC 机 Ubuntu 系统上搭建 NFS 服务器，然后在评估板上挂载服务器 NFS 共享目录并进行访问。

Target# find /sbin/ -name "*nfs"

```
root@RK3576-Tronlong:~# find /sbin/ -name "*nfs"
/sbin/mount.nfs
/sbin/umount.nfs
root@RK3576-Tronlong:~#
```

图 165

13.1 NFS 服务器搭建

进入 PC 机 Ubuntu 系统，执行如下命令安装 NFS 服务器安装包。

因我们的存在，让嵌入式应用更简单

Host# sudo apt-get install nfs-kernel-server -y

```
tronlong@tronlong-virtual-machine:~$ sudo apt-get install nfs-kernel-server -y
[sudo] password for tronlong:
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  keyutils libevent-core-2.1-7 libnfsidmap1 nfs-common rpcbind
Suggested packages:
  open-iscsi watchdog
The following NEW packages will be installed:
  keyutils libevent-core-2.1-7 libnfsidmap1 nfs-common nfs-kernel-server rpcbind
0 upgraded, 6 newly installed, 0 to remove and 41 not upgraded.
Need to get 615 kB of archives.
After this operation, 2,235 kB of additional disk space will be used.
Get:1 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy/main amd64 libevent-core-2.1-7 amd64 2.1.12-stable-1build3 [93.9 kB]
Get:2 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates/main amd64 libnfsidmap1 amd64 1:2.6.1-1ubuntu1.2 [42.9 kB]
Get:3 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy/main amd64 rpcbind amd64 1.2.6-2build1 [46.6 kB]
Get:4 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy/main amd64 keyutils amd64 1.6.1-2ubuntu3 [50.4 kB]
Get:5 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates/main amd64 nfs-common amd64 1:2.6.1-1ubuntu1.2 [241 kB]
Get:6 http://mirrors.tuna.tsinghua.edu.cn/ubuntu jammy-updates/main amd64 nfs-kernel-server amd64 1:2.6.1-1ubuntu1.2 [140 kB]
Fetched 615 kB in 1s (500 kB/s)
```

图 166

```
auth-rpcgss-module.service is a disabled or a static unit, not starting it.
nfs-idmapd.service is a disabled or a static unit, not starting it.
nfs-utils.service is a disabled or a static unit, not starting it.
proc-fs-nfsd.mount is a disabled or a static unit, not starting it.
rpc-gssd.service is a disabled or a static unit, not starting it.
rpc-statd-notify.service is a disabled or a static unit, not starting it.
rpc-statd.service is a disabled or a static unit, not starting it.
rpc-svcgssd.service is a disabled or a static unit, not starting it.
rpc_pipefs.target is a disabled or a static unit, not starting it.
var-lib-nfs-rpc_pipefs.mount is a disabled or a static unit, not starting it.
Setting up nfs-kernel-server (1:2.6.1-1ubuntu1.2) ...
Created symlink /etc/systemd/system/nfs-client.target.wants/nfs-blkmap.service
→ /lib/systemd/system/nfs-blkmap.service.
Created symlink /etc/systemd/system/multi-user.target.wants/nfs-server.service
→ /lib/systemd/system/nfs-server.service.
nfs-mountd.service is a disabled or a static unit, not starting it.
nfsdclld.service is a disabled or a static unit, not starting it.

Creating config file /etc/exports with new version

Creating config file /etc/default/nfs-kernel-server with new version
Processing triggers for man-db (2.10.2-1) ...
Processing triggers for libc-bin (2.35-0ubuntu3.9) ...
tronlong@tronlong-virtual-machine:~$
```

图 167

执行如下命令，创建 NFS 共享目录，在共享目录下新建 test 文件，并输入测试内容。

Host# sudo mkdir /home/tronlong/nfs_share/

Host# sudo chmod 777 nfs_share

Host# echo www.tronlong.com > nfs_share/test

因我们的存在，让嵌入式应用更简单

```
tronlong@tronlong-virtual-machine:~$ sudo mkdir /home/tronlong/nfs_share/
tronlong@tronlong-virtual-machine:~$ sudo chmod 777 nfs_share
tronlong@tronlong-virtual-machine:~$ echo www.tronlong.com > nfs_share/test
tronlong@tronlong-virtual-machine:~$ cat nfs_share/test
www.tronlong.com
tronlong@tronlong-virtual-machine:~$
```

图 168

打开"/etc/exports"配置文件，在配置文件中添加如下配置命令，设置服务器 NFS 共享目录及操作权限。

```
Host# sudo vi /etc/exports
```

```
tronlong@tronlong-virtual-machine:~$ sudo vi /etc/exports
tronlong@tronlong-virtual-machine:~$
```

图 169

```
/home/tronlong/nfs_share/ *(rw,nohide,insecure,no_subtree_check,async,no_root_squash)
```

```
1 # /etc/exports: the access control list for filesystems which may be exported
2 #           to NFS clients.  See exports(5).
3 #
4 # Example for NFSv2 and NFSv3:
5 # /srv/homes      hostname1(rw,sync,no_subtree_check) hostname2(ro,sync,no_subtree_check)
6 #
7 # Example for NFSv4:
8 # /srv/nfs4       gss/krb5i(rw,sync,fsid=0,crossmnt,no_subtree_check)
9 # /srv/nfs4/homes gss/krb5i(rw,sync,no_subtree_check)
10 #
11
12 /home/tronlong/nfs_share/ *(rw,nohide,insecure,no_subtree_check,async,no_root_squash)
```

图 170

配置命令参数解析如下：

表 14

参数	解析
/home/tronlong/nfs_share/	NFS 共享目录
*	允许任何网段 IP 的系统访问 NFS 共享目录
rw	访问者具有可读写权限
nohide	共享 NFS 目录的子目录

因我们的存在，让嵌入式应用更简单

insecure	NFS 通过 1024 以上的端口发送
no_subtree_check	若共享"/usr/bin"之类的子目录时，不检查父目录权限
async	资料不同步写入内存和硬盘
no_root_squash	访问者对 NFS 共享目录具有 root 权限

执行如下命令重启 NFS 服务器后，即可查询服务器本地 NFS 共享目录名称。

Host# sudo /etc/init.d/nfs-kernel-server restart

Host# showmount -e

```
tronlong@tronlong-virtual-machine:~$ sudo /etc/init.d/nfs-kernel-server restart
Restarting nfs-kernel-server (via systemctl): nfs-kernel-server.service.
tronlong@tronlong-virtual-machine:~$ showmount -e
Export list for tronlong-virtual-machine:
/home/tronlong/nfs_share *
```

图 171

13.2 NFS 共享目录挂载测试

将评估板 ETH1 RGMII 网口和 PC 机通过路由器或者交换机互联，亦可通过网线直联。使用 ifconfig 命令分别查看评估板与 PC 机当前的 IP 地址是否在同一网段。具体为 IP 地址的前三个字段相同，最后一个字段不相同，以及子网掩码相同。

在下图中可见，PC 机 IP 地址为：192.168.13.46，评估板 IP 地址为：192.168.13.51，子网掩码都是 255.255.255.0，属于同一网段内。

```
tronlong@tronlong-virtual-machine:~/tftpboot$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.13.46 netmask 255.255.255.0 broadcast 192.168.13.255
    inet6 fe80::4cd1:8cce:b795:9ac prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:d0:6f:c2 txqueuelen 1000 (Ethernet)
    RX packets 65901 bytes 51951805 (51.9 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 45564 bytes 45798045 (45.7 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 168 bytes 15753 (15.7 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 168 bytes 15753 (15.7 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

tronlong@tronlong-virtual-machine:~/tftpboot$
```

图 172 PC 机 IP 地址

```
root@RK3576-Tronlong:~# ifconfig
eth0      Link encap:Ethernet  HWaddr 52:1A:24:B0:52:51
          inet addr:192.168.13.51 Bcast:192.168.13.255 Mask:255.255.255.0
          inet6 addr: fe80::9084:97fb:57ba:e4a0/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:485 errors:0 dropped:0 overruns:0 frame:0
          TX packets:71 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:45733 (44.6 KiB)  TX bytes:6270 (6.1 KiB)
          Interrupt:114

eth1      Link encap:Ethernet  HWaddr 56:1A:24:B0:52:51
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:0 (0.0 B)
          Interrupt:116

eth2      Link encap:Ethernet  HWaddr 00:E0:99:E9:8A:62
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:0 errors:0 dropped:0 overruns:0 frame:0
          TX packets:3 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:0 (0.0 B)  TX bytes:270 (270.0 B)

eth3      Link encap:Ethernet  HWaddr 00:E0:99:E9:B7:43
          UP BROADCAST MULTICAST  MTU:1500  Metric:1
          RX packets:12769 errors:0 dropped:0 overruns:0 frame:0
          TX packets:313 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:1460282 (1.3 MiB)  TX bytes:24094 (23.5 KiB)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1 Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:65536  Metric:1
          RX packets:42 errors:0 dropped:0 overruns:0 frame:0
          TX packets:42 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:3084 (3.0 KiB)  TX bytes:3084 (3.0 KiB)

root@RK3576-Tronlong:~#
```

图 173 评估板 IP 地址

在 Ubuntu 执行 ping 命令测试 PC 机和评估板之间的网络连通性，按"Ctrl + C"命令退出。命令中 192.168.13.51 为评估板 IP 地址。

Host# ping 192.168.13.51

因我们的存在，让嵌入式应用更简单

```
tronlong@tronlong-virtual-machine:~$ ping 192.168.13.51
PING 192.168.13.51 (192.168.13.51) 56(84) bytes of data.
64 bytes from 192.168.13.51: icmp_seq=1 ttl=64 time=0.717 ms
64 bytes from 192.168.13.51: icmp_seq=2 ttl=64 time=0.676 ms
64 bytes from 192.168.13.51: icmp_seq=3 ttl=64 time=0.678 ms
64 bytes from 192.168.13.51: icmp_seq=4 ttl=64 time=0.687 ms
64 bytes from 192.168.13.51: icmp_seq=5 ttl=64 time=0.539 ms
^C
--- 192.168.13.51 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4091ms
rtt min/avg/max/mdev = 0.539/0.659/0.717/0.061 ms
tronlong@tronlong-virtual-machine:~$
```

图 174

在评估板文件系统执行如下命令，建立客户端 NFS 挂载目录，并修改其权限为可读可写。

```
Target#    mkdir -p /nfs/
Target#    chmod 777 /nfs/
```

```
root@RK3576-Tronlong:~# mkdir -p /nfs/
root@RK3576-Tronlong:~# chmod 777 /nfs/
root@RK3576-Tronlong:~# ls /
bin  busybox.fragment  dev  home  lib  libexec  lost+found  mnt  nfs  oem  proc  rockchip-test  root  sbin  sys  tmp  userdata  var  vendor
boot  data  etc  info  lib64  linuxrc  media  mnt  nfs  opt  rockchip-test  run  sdcard  system  tmp  udisk  usr  vendor
```

图 175

执行如下命令，将 PC 机 NFS 共享目录挂载到评估板 NFS 挂载目录。

```
Target#    mount -t nfs -o nolock 192.168.13.46:/home/tronlong/nfs_share /nfs/
```

```
root@RK3576-Tronlong:~# mount -t nfs -o nolock 192.168.13.46:/home/tronlong/nfs_share /nfs/
root@RK3576-Tronlong:~# df -h
Filesystem                                Size  Used Avail Use% Mounted on
/dev/root                                14G   1.3G   12G   10% /
devtmpfs                                966M   8.0K   966M    1% /dev
tmpfs                                    978M  140K   977M    1% /tmp
tmpfs                                    978M  992K   977M    1% /run
tmpfs                                    978M  196K   977M    1% /var/log
tmpfs                                    978M    0   978M    0% /dev/shm
/dev/mmcblk1p7                           123M   12M   108M   10% /oem
/dev/mmcblk1p8                           16G   704K   16G    1% /userdata
cgroup                                   978M    0   978M    0% /sys/fs/cgroup
192.168.13.46:/home/tronlong/nfs_share  196G  111G   76G   60% /nfs
root@RK3576-Tronlong:~#
```

图 176

NFS 挂载命令参数解析如下：

表 15

参数	解析
-t nfs	挂载类型为 NFS
-o no lock	不加文件锁，NFS 挂载时默认选项为文件锁(lock)
192.168.13.51	NFS 服务器 IP 地址
/home/tronlong/nfs_share	NFS 服务器共享目录
/nfs/	NFS 客户端挂载目录

查看 NFS 挂载目录内容，可看到内容与 PC 机 NFS 共享目录内容一致，并可正常读写共享目录的文件。

Target# ls /nfs/

```
root@RK3576-Tronlong:~# ls /nfs/
test
root@RK3576-Tronlong:~# cat /nfs/test
www.tronlong.com
root@RK3576-Tronlong:~#
```

图 177

```
tronlong@tronlong-virtual-machine:~$ ls nfs_share/
test
tronlong@tronlong-virtual-machine:~$ cat nfs_share/test
www.tronlong.com
tronlong@tronlong-virtual-machine:~$
```

图 178

在 PC 机 NFS 共享目录新建的文件，会在评估板的 NFS 挂载目录同步出现。在评估板的 NFS 挂载目录新建的文件，也会在 PC 机 NFS 共享目录出现。执行如下命令在评估板 NFS 挂载目录下创建 test1 测试文件，并可在 PC 机 NFS 共享目录看到。

Target# echo "Hello Tronlong" > /nfs/test1

Target# cat /nfs/test1

```
root@RK3576-Tronlong:~# echo "Hello Tronlong" > /nfs/test1
root@RK3576-Tronlong:~# cat /nfs/test1
Hello Tronlong
root@RK3576-Tronlong:~#
```

图 179

```
tronlong@tronlong-virtual-machine:~$ ls nfs_share/
test test1
tronlong@tronlong-virtual-machine:~$ cat nfs_share/test1
Hello Tronlong
tronlong@tronlong-virtual-machine:~$
```

图 180

如需卸载 NFS 挂载目录，可执行如下命令。卸载完成后，可使用 df 命令查看。

```
Target#    umount /nfs/
```

```
Target#    df -h
```

```
root@RK3576-Tronlong:~# umount /nfs/
root@RK3576-Tronlong:~# df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        14G  1.1G   13G   9% /
devtmpfs         966M   8.0K  966M   1% /dev
tmpfs            978M  140K  977M   1% /tmp
tmpfs            978M  992K  977M   1% /run
tmpfs            978M  196K  977M   1% /var/log
tmpfs            978M    0  978M   0% /dev/shm
/dev/mmcblk1p7  123M   12M  108M  10% /oem
/dev/mmcblk1p8   16G   504K   16G   1% /userdata
cgroup           978M    0  978M   0% /sys/fs/cgroup
root@RK3576-Tronlong:~#
```

图 181

14. 基于 TFTP + NFS 的系统启动说明

本章节主要演示评估板基于 TFTP + NFS 启动 Linux 系统(通过 TFTP 启动 Linux 内核，通过 NFS 挂载 Linux 文件系统)。请确保 Ubuntu 网络配置为桥接模式，且已正常安装 TFTP 和 NFS。

将产品资料“4-软件资料\Linux\Kernel\image\linux-6.1.115-[版本号]-[Git 系列号]\”目录下的内核文件 boot.img 拷贝至 Windows 下的 SharedFolders 共享目录，执行如下命令，将其拷贝至 Ubuntu 的 TFTP 服务器工作目录。

```
Host#    cp /mnt/hgfs/SharedFolders/boot.img /home/tronlong/tftpboot/
```

SharedFolders				在 Share...
名称	修改日期	类型	大小	
boot.img	2024/12/18/周三 18:49	光盘映像文件	42,042 KB	

图 182

```
tronlong@tronlong-virtual-machine:~$ cp /mnt/hgfs/SharedFolders/boot.img /home/tronlong/tftpboot/
tronlong@tronlong-virtual-machine:~$ ls /home/tronlong/tftpboot/
boot.img test
tronlong@tronlong-virtual-machine:~$
```

图 183

将产品资料“4-软件资料\Linux\Filesystem\buildroot-2024.02\image\buildroot-202402-[版本号]-[Git 系列号]”目录下的 rootfs.img 文件系统拷贝至 Windows 下的 SharedFolders 共享目录，执行如下命令，将其拷贝至 Ubuntu 的 RK3576 工作目录下。版本号、Git 系列号请以实际情况为准。

```
Host# cp /mnt/hgfs/SharedFolders/rootfs.img /home/tronlong/RK3576/
```

SharedFolders				在 Share...
名称	修改日期	类型	大小	
boot.img	2024/12/18/周三 18:49	光盘映像文件	42,042 KB	
rootfs.img	2024/11/27/周三 15:28	光盘映像文件	1,463,296...	

图 184

```
tronlong@tronlong-virtual-machine:~$ cp /mnt/hgfs/SharedFolders/rootfs.img /home/tronlong/RK3576/
tronlong@tronlong-virtual-machine:~$ ls /home/tronlong/RK3576/rootfs.img
/home/tronlong/RK3576/rootfs.img
tronlong@tronlong-virtual-machine:~$
```

图 185

执行如下命令将文件系统挂载至 NFS 共享目录，如下图所示。

```
Host# sudo mount -o loop /home/tronlong/RK3576/rootfs.img /home/tronlong/nfs_share/
```

```
tronlong@tronlong-virtual-machine:~$ sudo mount -o loop /home/tronlong/RK3576/rootfs.img /home/tronlong/nfs_share/
tronlong@tronlong-virtual-machine:~$ ls /home/tronlong/nfs_share/
bin      data  home  lib64  lost+found  oem  rockchip-test  sbin  system  userdata  vendor
boot     dev   info  libexec  media      opt  root          sdcard  tmp     usr
busybox.fragment  etc   lib   linuxrc  mnt        proc  run          sys    udisk   var
tronlong@tronlong-virtual-machine:~$
```

图 186

如需进入"/home/tronlong/nfs_share/root/"目录,请执行如下命令,切换为 root 用户,输入 exit 命令即可退出。

Host# sudo su

```
tronlong@tronlong-virtual-machine:~$ sudo su
root@tronlong-virtual-machine:/home/tronlong# cd /home/tronlong/nfs_share/root/
root@tronlong-virtual-machine:/home/tronlong/nfs_share/root# exit
exit
tronlong@tronlong-virtual-machine:~$
```

图 187

在 Ubuntu 执行如下命令查看 Ubuntu IP 地址,本次查看 IP 地址为 192.168.13.46。

Host# ifconfig

```
tronlong@tronlong-virtual-machine:~$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.13.46 netmask 255.255.255.0 broadcast 192.168.13.255
    inet6 fe80::4cd1:8cce:b795:9ac prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:d0:6f:c2 txqueuelen 1000 (Ethernet)
    RX packets 4201 bytes 563574 (563.5 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 314 bytes 128497 (128.4 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 124 bytes 10598 (10.5 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 124 bytes 10598 (10.5 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

tronlong@tronlong-virtual-machine:~$
```

图 188

将评估板 ETH1 RGMII 网口和 PC 机通过路由器或者交换机互联。评估板上电前长按"Ctrl + C"进入 U-Boot 命令行,执行如下命令进行网络配置。

U-Boot# setenv serverip 192.168.13.46 //设置 NFS 和 TFTP 服务器 IP 地址

U-Boot# setenv ipaddr 192.168.13.51 //设置评估板的 IP 地址,与服务器 I

因我们的存在,让嵌入式应用更简单

P 地址同网段

- U-Boot# setenv gatewayip 192.168.13.1 //设置网关 IP 地址
- U-Boot# setenv nfspath /home/tronlong/nfs_share //设置 NFS 服务器共享目录路径
- U-Boot# setenv bootcmd 'run net_boot' //设置启动方式为 NFS
- U-Boot# saveenv //保存环境变量
- U-Boot# reset //重启

```
=> setenv serverip 192.168.13.46
=> setenv ipaddr 192.168.13.51
=> setenv gatewayip 192.168.13.1
=> setenv nfspath /home/tronlong/nfs_share
=> setenv bootcmd 'run net_boot'
=> saveenv
Saving Environment to MMC...
Writing to MMC(0)... done
=> reset
```

图 189

将评估板重启，U-Boot 将自动从 TFTP 服务器工作目录下载内核镜像。

[illegible]

图 190

下载完成后，启动内核，并从 NES 服务器共享目录中挂载文件系统。

```
Starting kernel ...
[ 22.191587] Booting Linux on physical CPU 0x0000000000 [0x410fd034]
[ 22.191622] Linux version 5.15-r16-1158798-g8f4d9031278 (tronlong@tronlong-virtual-machine) (
aarch64-none-linux-gnu-gcc (GNU Toolchain for the A-profile Architecture 10.3-2021.07 (arm-10.29)) 10
3.1.20210621, GNU ld (GNU Toolchain for the A-profile Architecture 10.3-2021.07 (arm-10.29)) 2.36.1
20210621) #1 SMP Fri Feb 14 14:53:03 CST 2025
[ 22.194456] random: crng init done
[ 22.199013] Machine model: TL3576-EVM
[ 22.120099] earlycon: uart8250 at MMIO32 0x000000002ad40000 (options '')
[ 22.267717] printk: bootconsole [uart8250] enabled
[ 22.275299] efi: UEFI not found.
[ 22.281131] OF: fdt: Reserved memory: failed to reserve memory for node 'drm-cubic-lut@0': base 0x
0000000000000000, size 0 MiB
[ 22.343662] Zone ranges:
[ 22.346447] DMA [mem 0x0000000040200000-0x00000000bfffffff]
[ 22.353226] DMA32 empty
[ 22.356378] Normal empty
[ 22.359529] Movable zone start for each node
[ 22.362049] Early memory node ranges
[ 22.368117] node 0: [mem 0x00000000004020000-0x000000000483fffffff]
[ 22.374987] node 0: [mem 0x00000000049400000-0x000000000bfffffff]
[ 22.381858] Initmem setup node 0 [mem 0x00000000040200000-0x000000000bfffffff]
[ 22.390150] On node 0, zone DMA: 512 pages in unavailable ranges
[ 22.398080] On node 0, zone DMA: 4096 pages in unavailable ranges
```

图 191

因我们的存在，让嵌入式应用更简单

```
Welcome to RK3576 Buildroot
RK3576-Tronlong login: root (automatic login)

[ 53.953027] read descriptors
[ 53.953036] read strings
Warning: your password will expire in 0 days.

54.187574] android_work: did not send uevent (0 0 0000000000000000)
root@RK3576-Tronlong:~# [ 55.043139] Freeing drm_logo memory: 1384K
root@RK3576-Tronlong:~#
```

图 192

如出现根文件系统无法加载成功，进而导致系统卡住的情况，请执行如下命令，修改 Ubuntu 的 NFS 配置文件。

Host# sudo vim /etc/default/nfs-kernel-server

```
tronlong@tronlong-virtual-machine:~/nfs_share$ sudo vim /etc/default/nfs-kernel-server
```

图 193

修改内容如下：

```
RPCNFSDCOUNT="-V 2 8"
```

```
RPCMOUNTDOPTS="-V 2 --manage-gids"
```

```
RPCNFSDOPTS="--nfs-version 2,3,4 --debug --syslog"
```

```
1 # Number of servers to start up
2 RPCNFSDCOUNT="-V 2 8"
3
4 # Runtime priority of server (see nice(1))
5 RPCNFSDPRIOITY=0
6
7 # Options for rpc.mountd.
8 # If you have a port-based firewall, you might want to set up
9 # a fixed port here using the --port option. For more information,
10 # see rpc.mountd(8) or http://wiki.debian.org/SecuringNFS
11 # To disable NFSv4 on the server, specify '--no-nfs-version 4' here
12 RPCMOUNTDOPTS="-V 2 --manage-gids"
13
14 # Do you want to start the svcgssd daemon? It is only required for Kerberos
15 # exports. Valid alternatives are "yes" and "no"; the default is "no".
16 NEED_SVCSSD=""
17
18 # Options for rpc.svcgssd.
19 RPCSVCGSSDOPTS=""
20 RPCNFSDOPTS="--nfs-version 2,3,4 --debug --syslog"
```

图 194

执行如下命令重启 NFS 服务，然后将评估板断电重启即可。

Host# sudo systemctl restart nfs-kernel-server

因我们的存在，让嵌入式应用更简单

```
tronlong@tronlong-virtual-machine:~/nfs_share$ sudo systemctl restart nfs-kernel-server
tronlong@tronlong-virtual-machine:~/nfs_share$
```

图 195

执行如下命令，在 Ubuntu 的 NFS 共享目录新建 test.txt 文件。

Host# sudo touch nfs_share/test.txt

```
tronlong@tronlong-virtual-machine:~$ sudo touch nfs_share/test.txt
tronlong@tronlong-virtual-machine:~$ ls nfs_share/
bin      data  home  lib64  lost+found  oen  rockchip-test  sbin  system  udisk  var
boot     dev  info  libexec  media      opt  root          sdcard test.txt userdata vendor
busybox.fragment  etc  lib  linuxrc  mnt        proc  run          sys   tmp     usr
tronlong@tronlong-virtual-machine:~$
```

图 196

在评估板可查看 test.txt 文件，执行如下命令，往 test.txt 文件写入内容并在 Ubuntu 查看。

Target# echo Tronlong > /test.txt

Host# cat nfs_share/test.txt

```
root@RK3576-Tronlong:~# ls /
bin  busybox.fragment  dev  home  lib  libexec  lost+found  mnt  opt  rockchip-test  run  sdcard  system  tmp  userdata  var
boot data            etc  info  lib64  linuxrc  media      oem  proc  root          sbin  sys    test.txt  udisk  usr      vendor
root@RK3576-Tronlong:~#
```

图 197

```
tronlong@tronlong-virtual-machine:~$ cat nfs_share/test.txt
Tronlong
tronlong@tronlong-virtual-machine:~$
```

图 198

如需恢复 Linux 系统的默认启动模式，可在 U-Boot 命令行执行如下命令恢复环境变量。

U-Boot# env default -a -f

U-Boot# saveenv

U-Boot# reset

因我们的存在，让嵌入式应用更简单

```
=> env default -a -f
## Resetting to default environment
=> saveenv
Saving Environment to MMC...
Writing to MMC(0)... done
=> reset
```

图 199

15. Linux 设备驱动说明

如下为主要的 Linux 设备驱动说明：

表 16

设备名	Linux 内核驱动源码路径	Linux 设备文件或头文件
SOM LED 2/3 EVM LED 2/3/4	drivers/leds/leds-gpio.c	/sys/class/leds/user-ledX
USB2.0 HOST	drivers/phy/rockchip/phy-rockchip-inno-usb2.c	/sys/bus/usb/
USB3.2 HOST	drivers/usb/dwc3/core.c	/sys/bus/usb/
USB3.2 OTG	drivers/usb/dwc3/core.c drivers/usb/typec/tcpm/tcpci_husb311.c	/sys/bus/usb/
KEY4	drivers/input/keyboard/gpio_keys.c	/dev/input/eventX
KEY5	drivers/input/keyboard/adc-keys.c	/dev/input/eventX
RTC	drivers/rtc/rtc-ds1307.c	/dev/rtcX
HDMI OUT	drivers/gpu/drm/rockchip/dw_hdmi-rockchip.c	/dev/fbX
eMMC	drivers/mmc/host/dw_mmc-rockchip.c	/dev/mmcblkX
Micro SD	drivers/mmc/host/dw_mmc-rockchip.c	/dev/mmcblkX
MIPI LCD	drivers/gpu/drm/panel/panel-simple.c	/dev/fbX
MIPI LCD CAP TS	drivers/input/touchscreen/goodix.c	/dev/input/eventX
MIPI LCD BACKLIGHT	drivers/video/backlight/pwm_bl.c	/sys/class/backlight/
ETH1/2 RGMII	drivers/net/ethernet/stmicro/stmmac/dwmac- rk.c	/sys/class/net/ethX
ETH3/4(USB)	drivers/usb/gadget/udc/core.c	/sys/class/net/ethX

因我们的存在，让嵌入式应用更简单

RS485	drivers/tty/serial/8250/8250_dw.c	/dev/ttySX
RS232	drivers/tty/serial/8250/8250_dw.c	/dev/ttySX
CAN-FD1/2	drivers/net/can/rockchip/rk3576_canfd.c	/sys/class/net/canX
MIC IN/LINE IN/HP OUT	sound/soc/codecs/es8323.c sound/soc/rockchip/rockchip_sai.c	/dev/snd/
CAMERA1	drivers/media/i2c/ov13850.c	/dev/videoX
CAMERA2/3/4/5	drivers/media/i2c/imx219.c	/dev/videoX
LVDS	drivers/gpu/drm/panel/panel-simple.c	/dev/fbX
LVDS BACKLIGHT	drivers/video/backlight/pwm_bl.c	/sys/class/backlight/
LVDS RES TS	input/touchscreen/ads7846.c	/dev/input/eventX
M.2 PCIe NVMe	drivers/pci/controller/dwc/pcie-dw-rockchip.c	/sys/class/pci_bus/
ADC	drivers/iio/adc/rockchip_saradc.c	/sys/bus/iio/devices/iio:deviceX/in_voltageX_raw

备注: "X"表示可变化数值。

如需查询上表未列举出的驱动源码, 请参考如下方法查询。

以 ADS7846 驱动为例, 进入内核源码 drivers 目录, 执行如下命令查找对应的驱动源码, 指令中"ads7846"为查找的关键字。

Host# find | grep "ads7846"

```
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release/kernel/drivers$ find | grep "ads7846"
./input/touchscreen/ads7846.c
./input/touchscreen/.ads7846.o.cmd
./input/touchscreen/ads7846.o
tronlong@tronlong-virtual-machine:~/RK3576/rk3576_linux6.1_release/kernel/drivers$
```

图 200

更多帮助

销售邮箱: sales@tronlong.com

技术邮箱: support@tronlong.com

创龙总机: 020-8998-6280

技术热线: 020-3893-9734

创龙官网: www.tronlong.com

技术论坛: www.51ele.net

官方商城: tronlong.tmall.com